

**UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA**



**Metodología para la evaluación y validación de productos de
software desarrollados para EDDIE**

José Manuel Ibaceta Jaña

Profesor guía: Roberto González Ibáñez

Tesis para optar al título
de Ingeniera Civil en Informática

Santiago - Chile
2023

Resumen

Una de las consecuencias del estallido social y la pandemia que ocurrió en Chile entre los años 2019 a 2021 fue el retraso de proyectos. Uno de ellos fue augmEnteD reaDIng dEsk (EDDIE) del Grupo InTeractiOn del Departamento de Ingeniería Informática de la USACH, el cual no pudo ser verificado ni validado en un contexto real de operación debido al contexto nacional. Hoy en día, el proyecto se encuentra en una etapa madura, contando con una herramienta que posee diversos módulos. Para poner verificar el correcto funcionamiento del proyecto EDDIE se desarrolla una METodología para la evaluaCIÓN y VALidación de Productos de Software (MECIVA PS) con el objetivo de sistematizar el proceso de validez y confiabilidad de todo el *software* desarrollado para la herramienta. Esta sistematización no solo podrá ser implementada en los módulos actuales de EDDIE sino que también incluye a los módulos futuros a desarrollar.

A lo largo de este documento se presenta una propuesta de solución donde se profundizará el problema presentado, el enfoque y metodología a utilizar, además de una descripción del producto y las actividades a realizar. La metodología cuenta con tres fases, despliegue, reparación y diagnóstico, que se aplican a cada funcionalidad para ser integrada y validada para la herramienta EDDIE.

Palabras clave: metodología, lectura aumentada, validación de proyectos, verificación de proyectos, sistematización.

Tabla de contenidos

1. Introducción	1
1.1. Antecedentes y Motivación	1
1.2. Descripción del problema	1
1.3. Solución propuesta	2
1.3.1. Características de la solución	2
1.3.2. Propósito de la solución	2
1.4. Objetivos y alcance del proyecto	2
1.4.1. Objetivo general	2
1.4.2. Objetivos específicos	3
1.4.3. Alcances y limitaciones	3
1.5. Metodología y herramientas utilizadas	4
1.5.1. Metodología	4
1.5.2. Herramientas de desarrollo	4
1.6. Organización del documento	4
2. Marco Teórico	6
2.1. Marco conceptual	6
2.2. Estado del arte	7
2.2.1. Comparativa de EDDIE	7
2.2.2. Comparativa de MECIVA PS	9
2.3. Resumen	11
3. Análisis	12
3.1. Estado previo de EDDIE	12
3.2. Inicio: reconocimiento de texto	14
3.3. Sistema para el soporte de lectura no obstructiva	17
3.4. Módulo de búsqueda	20
3.5. Módulo de consistencia	22
3.6. Módulo de rastreo ocular	25
3.7. Rediseño	26
3.8. Resumen	28
4. MECIVA PS	29
4.1. Metodología	29
4.2. Fase de despliegue	31
4.2.1. Despliegue de EDDIE	31
4.2.2. Despliegue de la funcionalidad	34
4.3. Fase de reparación	38
4.4. Fase de diagnóstico	42
4.5. Resumen	45
5. Aplicación de la metodología	47
5.1. Error interno a EDDIE: Consistencia	48
5.2. Error interno a EDDIE: Rastreo Ocular	53
5.3. Error interno a EDDIE: Búsqueda	56
5.4. Error interno a EDDIE: Interacción Físico/Digital	59
5.5. Error interno a EDDIE: Otro	62
5.6. Error externo a EDDIE	63
5.7. Resumen	64

6. Evaluación	66
6.1. Búsqueda en enciclopedia	66
6.2. Rastreo Ocular (TheEyeTribe)	69
6.3. Reconocimiento de texto	73
6.4. Resumen	77
7. Conclusiones	78
7.1. Objetivos	78
7.1.1. General	78
7.1.2. Específicos	78
7.2. Resultados	79
7.3. Implicaciones y alcances	80
7.4. Implicaciones	80
7.5. Alcances	80
7.6. Trabajo futuro	81
7.7. Observaciones finales	81
Referencias	82
Anexo	83
A.1. GOMS	84
A.2. Tabla de errores totales	84
A.3. Tabla de errores de consistencia, búsqueda, otro y externo	88
A.4. Tablas de resultados y documentación de diagnostico	99
A.4.1. Búsqueda en enciclopedia	99
A.4.2. Rastreo Ocular	102
A.4.3. Reconocimiento de texto	105
A.5. Derechos de Autor	108

Índice de cuadros

3.1. Requisitos funcionales de Carrasco. Fuente: (Carrasco, 2017)	17
3.2. Requisitos funcionales de Contreras. Fuente: (Contreras, 2019)	19
3.3. Requisitos funcionales de Montanares. Fuente: (Montanares, 2019)	21
3.4. Requisitos funcionales de Pezoa. Fuente: (Pezoa, 2020)	24
3.5. Requisitos funcionales de Estay. (Estay, 2020)	26
3.6. Requisitos funcionales de Santibáñez. Fuente:(Santibáñez,2020)	28
4.1. Plantilla para la documentación de la fase de reparación. Fuente: (Autoría propia, 2023)	42
4.2. Plantilla para la documentación de la fase de diagnóstico. Fuente: (Autoría propia, 2023)	44
5.1. Requisitos funcionales del Módulo de consistencia. Fuente: (Autoría propia, 2023)	49
5.2. Requisitos funcionales del Módulo de rastreo ocular. Fuente: (Autoría propia, 2023)	53
5.3. Errores encontrados en el módulo de rastreo ocular de EDDIE. Fuente: (Autoría propia, 2023)	54
5.4. Requisitos funcionales del Módulo de búsqueda. Fuente: (Autoría propia, 2023)	57
5.5. Requisitos funcionales del Módulo de interacción físico/digital. Fuente: (Autoría propia, 2023)	59
5.6. Errores encontrados en el módulo de interacción físico/digital de EDDIE. Fuente: (Autoría propia, 2023)	60
5.7. Resumen de requisitos funcionales y errores encontrados. Fuente: (Autoría propia, 2023)	65
6.1. Resultados finales de las pruebas de reconocimiento de texto. Fuente: (Autoría propia, 2023)	76
A.1. Errores encontrados en EDDIE, primer semestre 2023. Fuente: (Autoría propia, 2023)	84
A.2. Errores encontrados en el módulo de consistencia de EDDIE. Fuente: (Autoría propia, 2023)	88
A.3. Errores encontrados en el módulo de búsqueda de EDDIE. Fuente: (Autoría propia, 2023)	93
A.4. Errores encontrados en la categoría interno-otro de EDDIE. Fuente: (Autoría propia, 2023)	96
A.5. Errores encontrados en la categoría Externo de EDDIE. Fuente: (Autoría propia, 2023)	98
A.6. Documentación de la funcionalidad 'Búsqueda en enciclopedia'. Fuente: (Autoría propia, 2023)	100
A.7. Resultados de las pruebas aplicadas en la búsqueda en enciclopedia. Fuente: (Autoría propia, 2023)	100
A.8. Documentación de la funcionalidad 'Rastreo Ocular'. Fuente: (Autoría propia, 2023)	102
A.9. Resultados de las pruebas aplicadas en el rastreo ocular. Fuente: (Autoría propia, 2023)	103
A.10.Resultados finales para diagnostico de rastreo ocular. Fuente: (Autoría propia, 2023)	103
A.11.Documentación de la funcionalidad 'Rastreo Ocular' utilizando una segunda posición. Fuente: (Autoría propia, 2023)	103
A.12.Resultados de las pruebas aplicadas en el rastreo ocular utilizando una segunda posición. Fuente: (Autoría propia, 2023)	104
A.13.Resultados finales para diagnostico de rastreo ocular utilizando una posición alternativa. Fuente: (Autoría propia, 2023)	105

A.14.Documentación de la funcionalidad 'Reconocimiento de texto'. Fuente: (Autoría propia, 2023)	105
A.15.Resultados de las pruebas de validez para reconocimiento de texto. Fuente: te: (Autoría propia, 2023)	106
A.16.Resultados de las pruebas de consistencia para reconocimiento de texto. Fuente: (Autoría propia, 2023)	106
A.17.Resultados finales de las pruebas de reconocimiento de texto. Fuente: (Autoría propia, 2023)	106

Índice de figuras

2.1. Funcionamiento de Magic Bookmark. Fuente: (Bairaktaris et al., 2021) . .	8
2.2. Diseño de la cuadrícula de realidad aumentada de e-mmersive book. Fuente: (Kim et al., 2018)	8
2.3. Ejemplo de la traducción de japonés a inglés utilizando Google Translate. Fuente: (Google, sf)	9
2.4. Flujograma del Marco de Evaluación Fuente: (Soto, 2022)	10
3.1. Esquema conceptual de la herramienta EDDIE. Fuente: (Santibáñez, 2020)	13
3.2. Estructura de la herramienta EDDIE, mostrando en azul los componen- tes donde se desarrollaron las nuevas interfaces por Santibáñez. Fuente: (Santibáñez, 2020)	14
3.3. Búsqueda de palabra en texto físico. Fuente: (Carrasco, 2017)	15
3.4. Resaltado de texto digital a físico. Fuente: (Carrasco, 2017)	15
3.5. Resaltado de texto físico a digital. Fuente: (Carrasco, 2017)	16
3.6. Reconocimiento gestual por cámara digital. Fuente: (Contreras, 2019) . .	18
3.7. Utilización de reconocimiento gestual por lápiz especial. Fuente: (Contre- ras, 2019)	18
3.8. Consistencia de figuras desde físico a digital. Fuente: (Pezoa, 2020) . . .	22
3.9. Consistencia de comentarios desde físico a digital. Fuente: (Pezoa, 2020)	23
3.10. Consistencia de marca páginas desde físico a digital. Fuente: (Pezoa, 2020)	23
3.11. Obtención de información desde documento digital a físico. Fuente: (Pe- zoa, 2020)	24
3.12. Diagrama de flujo para la interfaz de configuración. Fuente: (Santibáñez, 2020)	27
4.1. Diagrama de flujo: Implementación de MECIVA PS. Fuente: (Autoría pro- pia, 2023)	30
4.2. Sala de experimentación del Departamento de ingeniería informática. Fuente: (Autoría propia, 2023)	33
4.3. Computador de la sala de experimentación del Departamento de ingenie- ría informática Fuente: (Autoría propia, 2023)	33
4.4. Posicionamiento de la cámara digital. Fuente: (Autoría propia, 2023) . . .	34
4.5. Posicionamiento del eyetracker <i>GazePoint</i> . Fuente: (Autoría propia, 2023)	34
4.6. Diagrama de flujo de la fase despliegue de funcionalidad en EDDIE. Fuen- te: (Autoría propia, 2023)	37
4.7. Diagrama de flujo de la fase de reparación. Fuente: (Autoría propia, 2023)	41
4.8. Diagrama de flujo de la fase de diagnostico. Fuente: (Autoría propia, 2023)	46
5.1. Errores encontrados en el módulo de consistencia clasificados en SS (sin solución), SP (solución parcial) y SF (solución final). Fuente: (Autoría pro- pia, 2023)	52
5.2. Funcionalidades desplegadas del módulo de consistencia. Fuente: (Auto- ría propia, 2023)	53
5.3. Errores encontrados en el módulo de búsqueda clasificados en SS (sin solución), SP (solución parcial) y SF (solución final). Fuente: (Autoría pro- pia, 2023)	59
5.4. Errores encontrados en el módulo de interacción físico/digital (F/D) clasi- ficados en SS (sin solución), SP (solución parcial) y SF (solución final). Fuente: (Autoría propia, 2023)	62
5.5. Errores encontrados en la categoría Interno-Otro (I-O) clasificados en SS (sin solución), SP (Solución parcial) y SF (solución final). Fuente: (Autoría propia, 2023)	63
5.6. Errores encontrados en la categoría 'externo' clasificados en SS (sin solu- ción), SP (Solución parcial) y SF (solución final). Fuente: (Autoría propia, 2023)	64

6.1. Resultados para la validez en la funcionalidad 'búsqueda en enciclopedia' utilizando una comparativa entre la página de Wikipedia y los resultados obtenidos, utilizando diez pruebas en diez palabras distintas (PX , X siendo índice de la palabra). Fuente: (Autoría propia, 2023)	68
6.2. Resultados para la confiabilidad en la funcionalidad 'búsqueda en enciclopedia' utilizando los resultados arrojados por diez pruebas a diez palabras distintas (PX , X siendo índice de la palabra). Fuente: (Autoría propia, 2023)	69
6.3. Interfaz de demostración de <i>TheEyeTribe</i> .	70
6.4. Denominaciones del área definida para las pruebas de rastreo ocular. Fuente: (Autoría propia, 2023)	71
6.5. Resultados de las pruebas de validez de la funcionalidad 'rastreo ocular'. Fuente: (Autoría propia, 2023)	72
6.6. Resultados de las pruebas de confiabilidad de la funcionalidad 'rastreo ocular'. Fuente: (Autoría propia, 2023)	72
6.7. Posición alternativa para el <i>eyetracker</i> . Fuente: (Autoría propia, 2023)	73
6.8. Esquema del setup propuesto. Fuente: (Autoría propia, 2023)	73
6.9. Posición de palabras en la hoja física para la prueba de reconocimiento de texto. Siendo la parte superior (1,2,3) la zona más cercana a la cámara	75
6.10. Diferencia de calidad de captado de texto según la posición de la palabra. Fuente: (Autoría propia, 2023)	76
6.11. Detección de botón de destacado de texto. Fuente: (Autoría propia, 2023)	76

1. Introducción

1.1. Antecedentes y Motivación

Para la implementación de un proyecto es necesario garantizar que este satisfice las especificaciones y entrega las funcionalidades requeridas para el *software* deseado. Para esto, son necesarias la validación y la verificación del *software* (Sommerville,2005). La validación refiere a corroborar que el producto satisfice las necesidades del cliente (Sommerville,2005). La verificación corresponde a la correcta construcción del *software* basándose en sus requisitos funcionales y no funcionales (Sommerville,2005). Tanto la validación como la verificación son claves para los resultados de la investigación, ya que la primera está enfocada en el usuario final, mientras que la segunda, en el desarrollo.

Los errores son comunes en todos los sistemas de *software* desarrollados actualmente y, entre más tarde se detecten, mayor será su impacto. Diversos proyectos han fallado gracias a pequeños errores, lo que se traduce en pérdidas millonarias y el fracaso de investigaciones (Cuauhtémoc, sf). Para evitar este fallo, existen diversos procesos como las pruebas de validación para demostrar que el *software* cumple con las expectativas del cliente. Una vez encontrados los defectos, deben ser corregidos realizando las pruebas correspondientes(Sommerville,2005).

En las épocas del estallido social en Chile y la pandemia del Covid-19 se provocaron retrasos en una serie de desarrollos a nivel global. Entre estos se encuentra augmEnteD reaDIng dEsk (EDDIE). Esta es una herramienta no obstructiva, es decir, no necesita usar un artefacto especial como lentes de realidad virtual. Incluso cuenta con la posibilidad de mantener la misma información del documento físico y digital (secciones destacadas, dibujos y notas)(González, 2019).

Sin embargo, las funcionalidades realizadas por los tesisistas del Grupo InteractiOn no fueron probadas en su integración y su despliegue con la configuración del hardware real. Es por ello que se necesita un proceso que sistematice la validación y verificación del proyecto EDDIE, y que pueda ser aplicado tanto a su *software* actual como futuro. Lo cual lleva al desafío de desarrollar una metodología de evaluación y validación enfocada a los productos de *software* de EDDIE.

1.2. Descripción del problema

A la fecha actual no se cuenta con lo necesario para poder utilizar el proyecto EDDIE en investigaciones. Esto se debe a que las funcionalidades no han sido

validadas ya que no fue posible acceder a los recursos de hardware para los que fueron diseñados durante la implementación de sus últimos avances. Para el estado actual de EDDIE y su futuro *software* y *hardware* ¿De qué manera sistematizar la evaluación de EDDIE y sus extensiones actuales y futuras en términos de validez y confiabilidad?

1.3. Solución propuesta

1.3.1. Características de la solución

Se presenta un plan de validación de proyectos a través del desarrollo de una nueva metodología enfocada a EDDIE denominada MECIVA PS: MEtodología para la evaluaCIÓN y VALidación de Productos de *Software*. Esto incluye la implementación de *software* en el *hardware*, clasificación de sus errores asociados, documentación del proceso de su evaluación en EDDIE y una guía sobre cómo utilizar la metodología propuesta. Lo anterior apuntando a evaluar, mantener y sistematizar la validación y puesta en marcha de proyectos relacionados a la herramienta EDDIE, puesto que la metodología esta desarrollada tomando en cuenta solo elementos de esta herramienta y va a incrementar en conjunto con esta. MECIVA PS se establece desde un análisis técnico, puesto que no se revisará ningún tipo de aspecto económico, y se enfocará en las nuevas funcionalidades que no han sido probadas en el ambiente de desarrollo, validando de igual manera el correcto funcionamiento de toda la herramienta. Para esto se utilizan validaciones respaldadas en teoría y conceptos de validación ya existentes.

1.3.2. Propósito de la solución

La solución busca desarrollar MECIVA PS, verificando y validando los módulos desarrollados en la época del estallido social y pandemia, que fueron probados con hardware propios de los investigadores. Con esta propuesta se podrá utilizar EDDIE para la investigación sobre la comprensión lectora (Contreras, 2019), utilizando las funcionalidades de rastreo ocular (Estay, 2020), consistencia (Pezoa, 2020), búsqueda (Montanares, 2019) e interfaz de usuario (Santibáñez, 2020). La validación de funciones actuales y futuras permitirá la evolución de la herramienta.

1.4. Objetivos y alcance del proyecto

1.4.1. Objetivo general

Desarrollar una metodología para la sistematización del proceso de la validez y confiabilidad de los desarrollos de *software* en torno al proyecto EDDIE.

1.4.2. Objetivos específicos

1. Diseñar una metodología ad hoc para evaluar productos de *software* del proyecto EDDIE considerando aspectos de funcionamiento, validez y confiabilidad.
2. Integrar las implementaciones de EDDIE, tales como: interfaz de usuario (Santibáñez, 2020), consistencia (Pezoa, 2020), búsqueda (Montanares, 2019) y rastreo ocular (Estay, 2020).
3. Desplegar la versión integrada del *software* en el ambiente operativo con el *hardware* para el que fue diseñado.
4. Aplicar la metodología para evaluar EDDIE en su estado de desarrollo actual.
5. Validar la metodología utilizando el estado de desarrollo actual de EDDIE.
6. Resolver problemas y errores de la plataforma identificados en un ambiente operativo.

1.4.3. Alcances y limitaciones

El plan de validación para aplicar MECIVA PS a EDDIE contará con tres partes. La primera es el despliegue del *software* con el *hardware* que dispone el laboratorio de InTeractiOn. La segunda es realizar una revisión de las funcionalidades de la herramienta EDDIE, obteniendo los errores correspondientes a cada una. Finalmente, se realiza un proceso de reparación y diagnóstico de la herramienta. Todo el proceso será documentado y se realizará una guía para poder implementar MECIVA PS en futuros módulos de EDDIE. Para esto se utiliza la herramienta de control de versiones Github y los repositorios de Gitbook.

Otro aspecto para destacar es que se asume que todo el *hardware* necesario para la validación de EDDIE se encuentra disponible y completamente funcional en el laboratorio de InTeractiOn, así mismo, la información necesaria para utilizar los dispositivos está disponible en el mismo lugar o, en su defecto, se utilizará la entrega de manera online por las empresas correspondientes a dichos artefactos.

En caso de que una funcionalidad no sea posible de adaptar al *hardware*, no se realizará un código nuevo de la funcionalidad. Solo en caso de corrección de errores y adaptación del código existente al *hardware*, el *software* será modificado para adaptarlo a estos errores.

1.5. Metodología y herramientas utilizadas

1.5.1. Metodología

Considerando que los productos que se desean validar están destinados a la investigación, la metodología objetivo de esta tesis (MECIVA PS) utilizará como base el método GOMS (ver Anexo), herramienta de validación de la interfaz de usuario, que se enfoca tanto en la interacción usuario-*software* como en el modelamiento de tareas (Karat, 1988). La elección de este método se justifica en que no solo analiza las partes por separado, sino que es especialmente útil en su unión. Además, es altamente compatible y posee capacidad de adaptación, pudiendo ser aplicado a las necesidades de los distintos proyectos, o en este caso, a distintos módulos.

Para esta tesis, se decide utilizar la versión GOMS inicial (CMN-GOMS) (Jonh, 1996), ya que esta presenta una forma de descomponer una tarea en sub-tareas jerárquicamente, con métodos, operadores y reglas pertinentes, adaptable a tareas de *software*. Otras versiones de GOMS que no serán utilizadas en esta tesis, como NGOMSL y CPM-GOMS, reemplazan y agregan elementos enfocados en la interfaz de usuario. Por ejemplo, la versión KLM elimina los objetivos, métodos y reglas reemplazándolos con movimientos del ratón y teclas a presionar.

1.5.2. Herramientas de desarrollo

A diferencia de los trabajos anteriores, este trabajo es desarrollado con el *hardware* con el que se encuentra disponible en la sala de experimentación del Departamento de Ingeniería Informática. Estos son: cámara digital, computador, eyetrackers, proyectos, trípode, alargador, entre otros. Además, se utilizará un notebook hp omen 15-dc0xxx, Intel(R) Core(TM) i7-8750H, 16 GB RAM, SSD 825 GB, sistema operativo Windows 10 64 bits. En el apartado de *software*, Todo código implementado es en el lenguaje C#, puesto que es en lo que está desarrollado el proyecto. El repositorio online se encuentra en Git Hub, desde donde se obtiene la versión actual de EDDIE, mientras que el editor de código es Visual Studio. Por otra parte, se utilizan Overleaf y gitbook para la documentación.

1.6. Organización del documento

A lo largo del documento se presentan siete capítulos.

En el *primer capítulo* se expone los conceptos básicos para entender la problemática presentada.

En el *segundo capítulo* se presenta el marco teórico, donde se abarcan todos los conceptos necesarios para entender el trabajo realizado. Además, se presentan proyectos similares tanto a la herramienta EDDIE como a la metodología MECIVA PS y se destacan las diferencias con estos.

El *tercer capítulo* muestra el estado de EDDIE previo a la implementación de MECIVA PS.

El *cuarto capítulo*, MECIVA PS, explica la metodología que será utilizada para la validación y evaluación de EDDIE en su estado actual y en sus futuras funcionalidades.

En el *quinto capítulo*, aplicación, se resumen los resultados del proceso realizado a lo largo de la implementación de MECIVA PS, donde se introducen los problemas obtenidos, su estado y comentarios pertinentes a estos. Además, se presenta el estado de los requisitos funcionales y otros elementos pertinentes a la aplicación de la metodología.

En el *sexto capítulo* se explica y ejemplifica el proceso de MECIVA PS en el proyecto EDDIE a través de tres funcionalidades.

El *séptimo capítulo*, concluye esta tesis, analizando los resultados obtenidos a través de la implementación de la metodología y los factores que pueden ser trabajados a futuro.

2. Marco Teórico

2.1. Marco conceptual

- **Realidad aumentada:** el usuario utiliza un dispositivo externo en el cual se aprecia un ambiente virtual, por lo cual se puede interpretar como una experiencia visual que combina tanto elementos reales como virtuales. En la implementación de estos entornos no se suelen utilizar sensores externos que interactúen directamente con el usuario, pero sí de forma indirecta como sensores de posición o movimiento. Puesto que no se pierde la conexión con el ambiente real no es necesario determinar un área de seguridad (Navarro, 2018).
- **Validez:** denota que el resultado esté dentro de lo esperado. según Martínez (Martínez, 2006), hay tres tipos de validez: de construcción, interna y externa. A lo largo de este documento se refiere a la validez de construcción.
 - **Validez de construcción:** 'Se establece una medida operacional para los conceptos que van a ser utilizados'(Martínez, 2006).
- **Confiabilidad:** En el área de la investigación, confiabilidad corresponde a la posibilidad de realizar repetidamente un procedimiento obteniendo los mismos resultados. Asegura que un investigador, al seguir el mismo procedimiento que otro, llegue a los mismos resultados y conclusiones que se obtuvieron previamente. (Martínez, 2006)
- **Herramienta:** Corresponde a un elemento que sirve para facilitar la realización de una tarea, garantizando el aumento de productividad. Estos elementos pueden ser físicos, *software*, conceptos, entre otros(Gonzalo Ramos, 2014).
- **Integración continua:** Según Duvall, Matyas y Glover (Duvall et al., 2007), se refiere a una práctica de software realizada donde todos los miembros involucrados integran sus avances, los cuales son previamente verificados. Generalmente es utilizado para asegurar que se mantenga la integridad del software. Para cumplir con lo anterior se debe cumplir con lo siguiente:
 - Contar con control de versiones.
 - Tener pruebas de aceptación y que todas las funcionalidades integradas las aprueben.
 - El producto generado debe ser capaz de ser probado funcionalmente.

- Priorizar la reparación de problemas.
- Generación de reportes que cuenten con aspectos de construcción, prácticas de programación, entre otros aspectos.

2.2. Estado del arte

A continuación, se presentan dos subsecciones. La primera presenta una breve descripción de EDDIE y la comparación de esta herramienta con tecnologías que utilizan funcionalidades similares. La segunda muestra los paralelos y diferencias entre la metodología propuesta MECIVA PS y otras metodologías de validación de *software*.

2.2.1. Comparativa de EDDIE

- **EDDIE:** También conocido como 'augmEnteD reaDIng dEsk', es una herramienta que propone el aumento de la comprensión lectora utilizando realidad aumentada, esto de forma no obstructiva. Esto último, en otras palabras, es que el usuario no necesita utilizar algún implemento extra como lentes especiales. Además, permite la interacción con el material escrito utilizando el reconocimiento de los gestos de la mano del usuario, lápices y comandos de voz. Otra de las ventajas que ofrece es la consistencia entre el texto físico y digital, compartiendo el destacado, dibujos y notas (InTeractiOn, sf). Este *software* cuenta con el esfuerzo combinado del equipo de InTeractiOn para lograr la lectura utilizando realidad aumentada, realizando trabajos en: consistencia (Pezoa, 2020), rastreo ocular (Estay, 2020), búsqueda (Montanares, 2019), interacción físico/digital (Contreras, 2019) e interfaz de usuario (Santibáñez, 2020). El estado actual de este proyecto y una definición más detallada es presentada más adelante en el documento.

Existen tecnologías similares a lo que es el proyecto EDDIE las cuales también usan lectura aumentada. Entre las cuales se encuentran:

- **Magic Bookmark:** Tecnología que reconoce automáticamente una página de un libro real, entregando un aumento sin cambiar el comportamiento del usuario de manera significativa. Para el funcionamiento de esta tecnología, es necesaria una banda con sensores especiales que reconocen la información a través de un código binario único por cada página, enviándola al dispositivo móvil (Bairaktaris et al., 2021) (Figura 2.1). Sin embargo, dicha tecnología está diseñada para ser compatible solo con un tipo de libros especiales, puesto que deben tener el código previamente mencionado. Esto se diferencia de la herramienta EDDIE puesto que no es necesario alguna alteración al documento impreso.

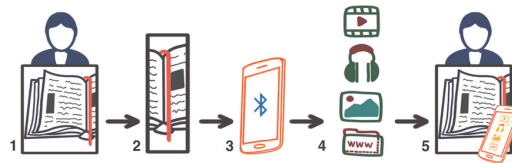


Figura 2.1: Funcionamiento de Magic Bookmark.
Fuente: (Bairaktaris et al., 2021)

- **e-mmersive Book:** Permite la lectura de libros simultáneamente para poder encontrar las respuestas deseadas. De esta manera, se utiliza una lectura sinóptica para la revisión y comparación de libros de un tópico determinado para tener una opinión sobre el tema o crear nuevas ideas. Además, utiliza los 'Microsoft Holo-lens' para aumentar el espacio visible del computador utilizando pantallas virtuales, siendo estas tablas de contenido, cuerpo y el número de páginas (Kim et al., 2018) (Figura 2.2). A diferencia de EDDIE, este requiere la implementación de la herramienta de Microsoft. En cambio, la herramienta EDDIE puede utilizar cualquier cámara o proyector para su implementación, además de no ser necesario que el usuario utilice directamente dichas herramientas, como lo es utilizar los lentes de realidad aumentada.

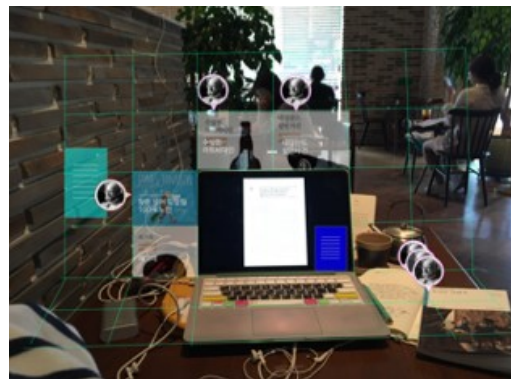


Figura 2.2: Diseño de la cuadrícula de realidad aumentada de e-mmersive book.
Fuente: (Kim et al., 2018)

- **Google Translate:** Este servicio de Google permite la traducción de texto escrito en tiempo real. Sin embargo, también puede ser utilizado para traducir los textos que se encuentren en una imagen a través de la cámara de un dispositivo smartphone. Otra de las ventajas que tiene es que no es necesaria la utilización de internet en el momento pues es posible descargar un archivo que permite la traducción a un idioma específico (depende del archivo que fue descargado, por ejemplo, se puede descargar un archivo que deje traducir desde el japonés hacia inglés). Finalmente, se permite la traducción a través de audio, la lectura de

un texto escrito y es capaz de reconocer el texto escrito por el usuario (Google, sf) (Figura 2.3). La herramienta EDDIE tiene varias similitudes: ambos son capaces de reconocer texto escrito a través de una imagen, son servicios no pagados, entre otros. Sin embargo, la herramienta EDDIE utiliza búsquedas de videos relacionados, acceso a Wikipedia y es capaz de reconocer los gestos de la mano del usuario.

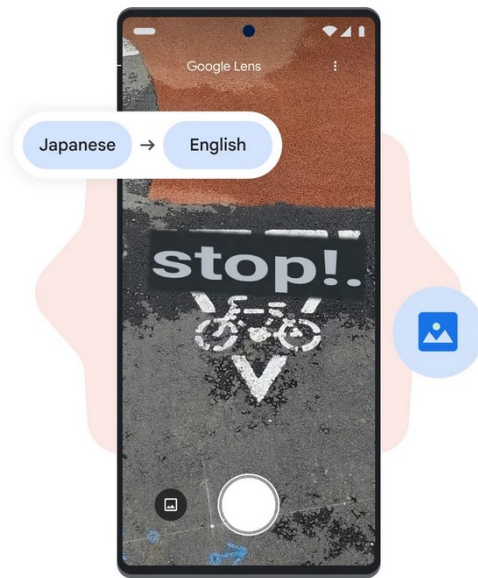


Figura 2.3: Ejemplo de la traducción de japonés a inglés utilizando Google Translate.
Fuente: (Google, sf)

2.2.2. Comparativa de MECIVA PS

Por otra parte, MECIVA PS no es la única metodología aplicada a la validación de *software*. A continuación se presentan algunas metodologías y servicios similares a la presentada en este documento:

- **Marco de trabajo para la evaluación de validez y confiabilidad de Multimodal Observer (MO):** Como su nombre lo indica, está dedicado a la evaluación de validez y confiabilidad, orientado a establecer cómo llevar a cabo la sistematización de procesos evaluativos para las nuevas extensiones del proyecto Multimodal Observer que, al igual que la herramienta EDDIE, es desarrollado por el equipo de InTeractiOn. Para lograr su objetivo, este marco de trabajo propone 7 pasos para evaluar y validar la nueva extensión (Soto, 2022), los cuales pueden ser apreciados de mejor manera en la Figura 2.4:

1. Identificación de la extensión.

2. Definición de métricas.
3. Elección de *ground truth*.
4. Despliegue pre-recolección.
5. Recolección y disponibilización.
6. Análisis estadístico.
7. Documentación de resultados.

Aunque ambas metodologías son desarrolladas por el equipo de InTeractiOn, el marco de trabajo está enfocado en la validación instrumental, mientras que MECI-VA PS se enfoca en distintos tipos de funcionalidades de una herramienta.

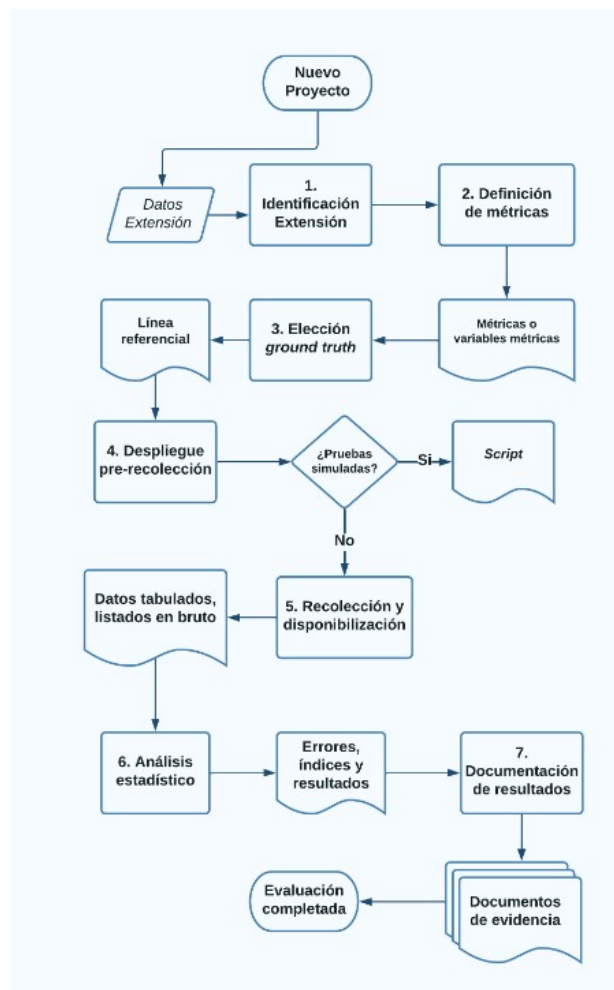


Figura 2.4: Flujograma del Marco de Evaluación
Fuente: (Soto, 2022)

- **SoftExpert:** Esta marca se dedica a la gestión empresarial. Cuenta con soluciones desde gestión de negocios y calidad empresarial hasta gestión ambiental. También

incluye un servicio de validación de *software* que asegura el cumplimiento de sus aplicaciones para la empresa objetivo, el cual es implementado de distintas maneras dependiendo del programa. Aún así, cuenta con procesos estándar para su implementación como lo son las pruebas de aceptación. Estas son desarrolladas con antelación, entregando el detalle necesario para garantizar que la prueba valide la aplicación (SoftExpert, sf). Otro aspecto importante para destacar es que cuenta con un servicio de integración. A diferencia de MECIVA PS, este servicio es pagado y se aplica de manera generica al *software*. MECIVA PS apunta a una forma gratuita de verificar la integración y validación correcta las funcionalidades de EDDIE.

- **Cercal Group:** Se dedican en ofrecer soluciones para cumplir con las exigencias de la industria farmacéutica, clínica, alimentaria y dispositivos médicos. Entre todos los servicios que ofrecen cuentan con uno de validación de *software* e Integridad de datos. Entre los servicios relacionados con la validación se encuentran: Análisis de riesgos y evaluación de sistemas, diseño e implementación de sistemas y equipos, validación y clasificación de sistemas y equipos, desarrollo de procedimientos y documentación, entre otros. Esto se implementa dependiendo del *software* deseado, adaptandose a sus requerimientos (Cercal Group, sf). La diferencia fundamental entre este servicio y MECIVA PS es que esta ultima se centra no solo en *software*, si no que tambien en otros aspectos como lo es el *hardware*.

2.3. Resumen

En este capítulo se presentan los conceptos necesarios para entender el desarrollo de la metodología presentada. Además, se abordan tecnologías similares a EDDIE y ejemplos de metodologías similares a MECIVA PS, realizando un análisis del porque utilizar esta metodología.

3. Análisis

3.1. Estado previo de EDDIE

Antes de profundizar en la metodología, es necesario conocer el estado de la herramienta EDDIE a finales del año 2022. EDDIE es un proyecto de lectura aumentada no obstructiva el cual ha sido desarrollado entre los años 2017 a 2022 por seis tesis, cuyo trabajo es importante separar en dos fases, debido a los ambientes de desarrollo utilizados. La primera fase fue desarrollada en el laboratorio por Carrasco (2017), Montañares (2019) y Contreras (2019). La segunda fase fue desarrollada en remoto por Santibáñez(2020), Estay (2020) y Pezoa (2020), quienes se vieron forzados a trabajar desde sus hogares a raíz de lo ocurrido en Chile como a nivel global.

El propósito de la herramienta EDDIE es su utilización en la investigación internacional sobre los efectos que tiene la lectura aumentada no obstructiva sobre la comprensión lectora del usuario. Con tal de lograr esto existen cuatro grandes funcionalidades unidas en este proyecto de *software*: rastreo ocular (Estay, 2020), consistencia (Pezoa, 2020), búsqueda (Montañares, 2019) y reconocimiento de texto físico (Carrasco, 2017). Gracias a sus avances, la herramienta EDDIE cuenta con la arquitectura que se presenta en el esquema conceptual de la Figura 3.1 y una estructura presentada en la Figura 3.2.

Para utilizar la herramienta EDDIE se requiere de varios pasos:

1. colocar el documento físico sobre una superficie donde será proyectada la interfaz del *software*.
2. utilización de hardware externo como lo son las dos cámaras digitales, un eyetracker y un proyector.
3. una vez está desplegado, se debe establecer la configuración de los elementos a utilizar: Las API's que se usaran para búsqueda, la cámara que se usará para el reconocimiento gestual, entre otros.
4. resaltar la palabra o figura deseada y ejecutar la acción deseada.

Todas las interacciones que son desde el documento digital hacia el físico son realizadas sin alterar el papel. Por ejemplo: cuando se quiere resaltar una palabra desde el texto digital hacia el físico, esta es resaltada utilizando la proyección de la interfaz en la superficie, resaltando el texto digitalmente en la versión física.

A continuación se presenta detalladamente el estado de la herramienta EDDIE en base a las tesis relacionadas, indicando las funcionalidades correspondientes

y los requisitos funcionales asociados.

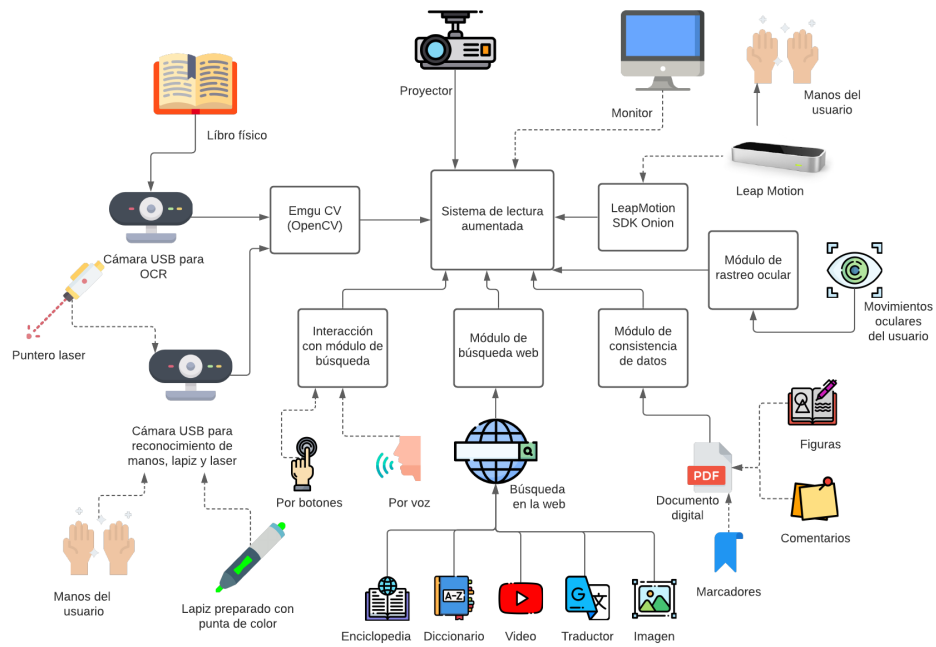


Figura 3.1: Esquema conceptual de la herramienta EDDIE.
Fuente: (Santibáñez, 2020)

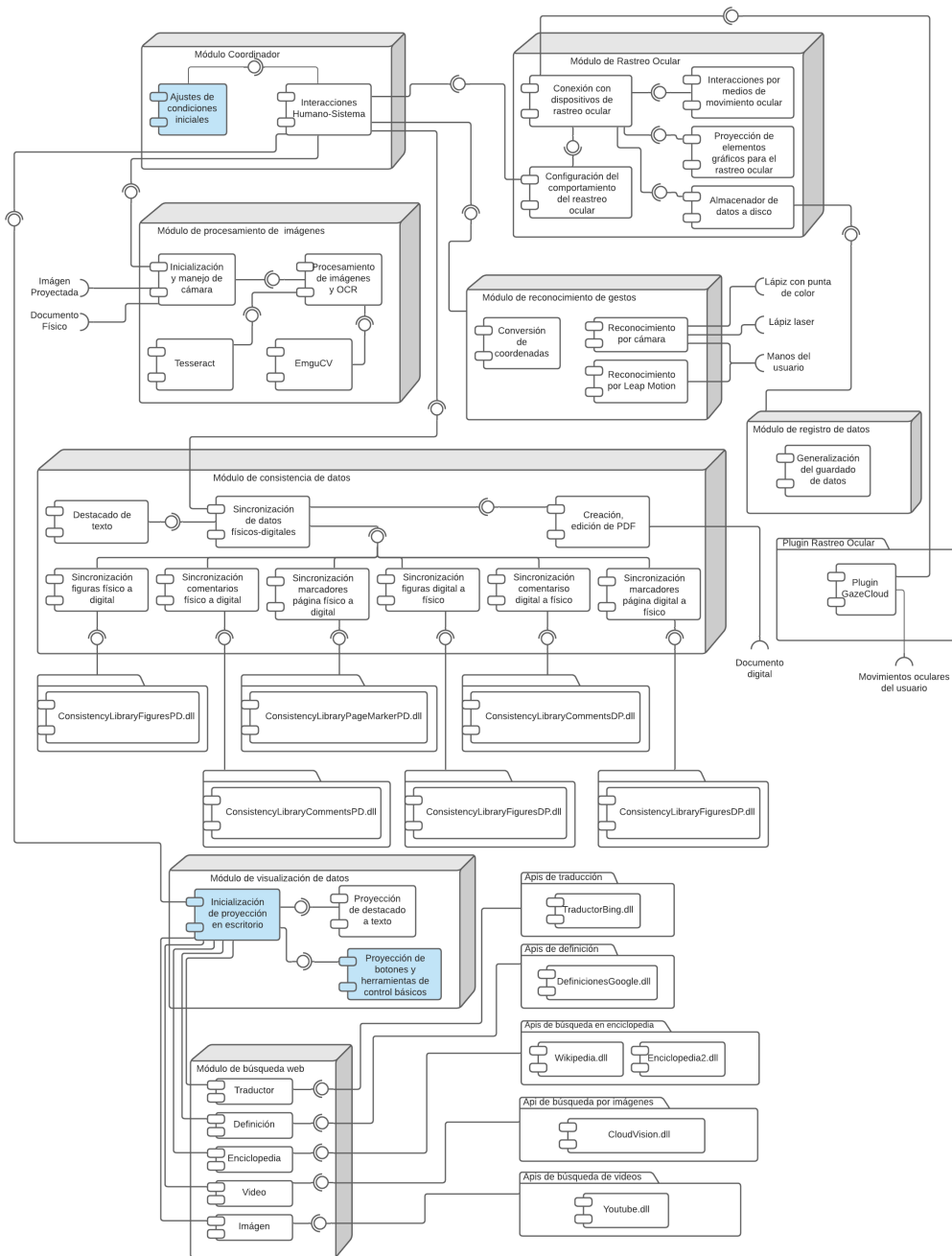


Figura 3.2: Estructura de la herramienta EDDIE, mostrando en azul los componentes donde se desarrollaron las nuevas interfaces por Santibáñez.

Fuente: (Santibáñez, 2020)

3.2. Inicio: reconocimiento de texto

Antes de que la herramienta estuviese siquiera pensada para su desarrollo, Carrasco desarrolló un sistema para el estudio de las interacciones del material impreso y digital (Carrasco, 2017). Lo construyó de manera modular para posibilitar la integración

de nuevas funcionalidades. A través de siete prototipos, se logró conectar la cámara a su software. Además, gracias a la utilización del conjunto de API's *Tess-Two* y la herramienta de *OCR Tesseract*, se implementó la capacidad de buscar palabras en un documento físico, resaltándola con colores para poder encontrar todas las coincidencias en papel, tal como se muestra en la Figura 3.3.

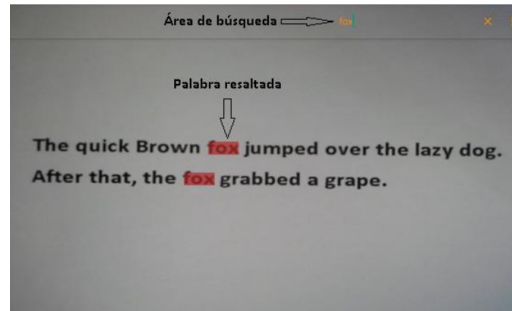


Figura 3.3: Búsqueda de palabra en texto físico.
Fuente: (Carrasco, 2017)

Otra de las aplicaciones es el resaltado de texto en digital a físico a través de la proyección, como se puede ver en la Figuras 3.4 y 3.5. Lo anterior deja ver las primeras señales de consistencia en la herramienta EDDIE. También se implementa la capacidad de crear comentarios, traspasándolos desde el documento físico hacia el digital, y viceversa (consistencia simétrica). Existen otras funciones aplicadas en esta versión, sin embargo, estas escapan a los alcances de esta tesis y no serán mencionadas. Si se desea obtener más información sobre este trabajo se recomienda revisar la página del laboratorio de InTeractiOn, en el apartado de publicaciones asociadas a EDDIE.

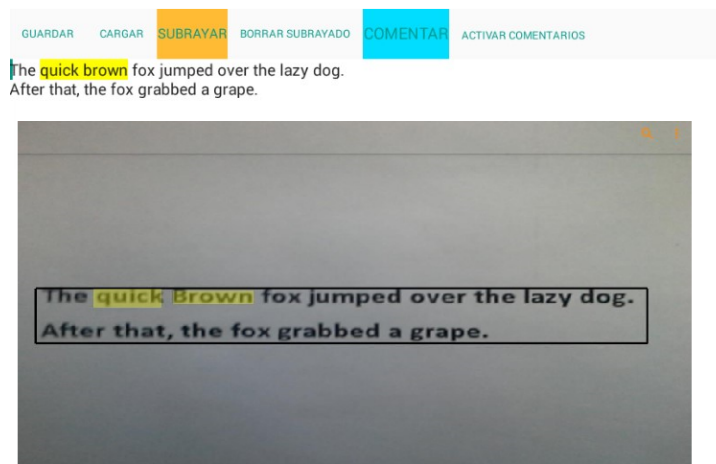


Figura 3.4: Resaltado de texto digital a físico.
Fuente: (Carrasco, 2017)

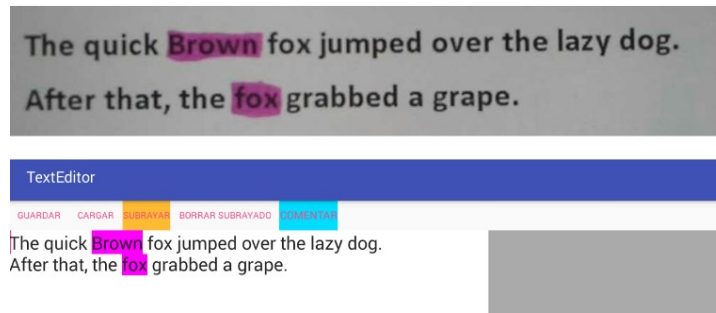


Figura 3.5: Resaltado de texto físico a digital.
Fuente: (Carrasco, 2017)

Para cerrar este apartado, se presentan los requisitos funcionales asociados a la tesis de Carrasco en la Tabla 3.1.

Tabla 3.1: Requisitos funcionales de Carrasco.
Fuente: (Carrasco, 2017)

ID	Descripción
RFCA1	Mantener consistencia de contenido.
RFCA2	Reconocer texto y crear documento digital.
RFCA3	Buscar y resaltar palabra en documento físico.
RFCA4	Resaltar palabra en documento físico y resaltarla en versión digital.
RFCA5	Resaltar palabra en documento digital y resaltarla en versión física.
RFCA6	Realizar comentarios en documento físico y mostrarlos en versión digital.
RFCA7	Realizar comentarios en documento digital y mostrarlos en versión física.
RFCA8	Capacidad de activar y desactivar comentarios.
RFCA9	Método para mantener consistencia entre ambos formatos.

3.3. Sistema para el soporte de lectura no obstructiva

En los inicios de EDDIE, se creó un sistema para el soporte de lectura no obstructiva (Contreras, 2019). A través de ocho prototipos se implementaron distintos tipos de reconocimientos de texto. Lo anterior se logró mediante la utilización de *OCR* (Optical Character Recognition) mediante la biblioteca de reconocimiento de caracteres *Tesseract*. Gracias a esto fue posible el desarrollo de funcionalidades relacionadas al reconocimiento de texto:

- **Manejo y búsqueda de datos en PDF:** Desarrollado en el segundo prototipo, mediante la utilización de la biblioteca *iTextShap* e indicando la página donde se buscará en el archivo, se resalta la palabra o frase que se desea encontrar en el documento físico.
- **Reconocimiento gestual por cámara digital:** Esta funcionalidad se logró gracias a la implementación de la biblioteca de reconocimiento de imágenes *Emgu CV*. La cámara capta la posición de la mano utilizando el color de piel del usuario. Con esta información se estima la posición de los dedos y utiliza el píxel superior para manipular el puntero del ratón. Lo anterior se puede ver representado en la Figura 3.6.

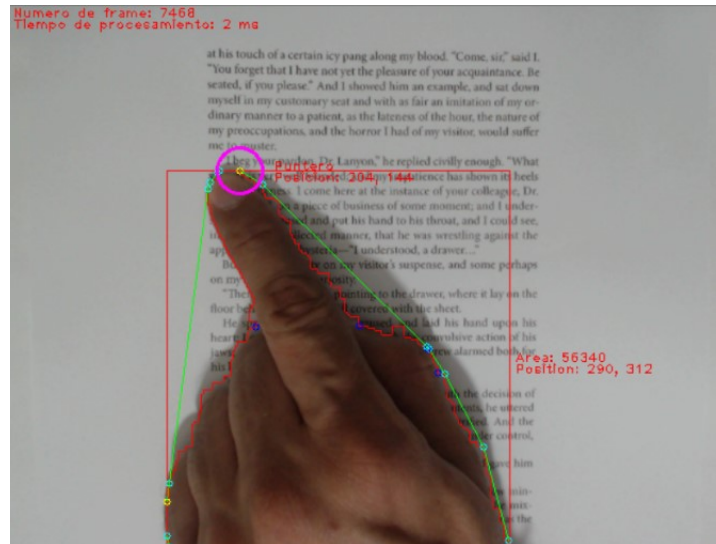


Figura 3.6: Reconocimiento gestual por cámara digital.
Fuente: (Contreras, 2019)

- **Reconocimiento gestual por lápiz especial:** El lápiz especial puede ser uno con un color característico en la punta o un puntero láser, como se puede ver en la Figura 3.7. Es importante destacar que la punta del lápiz debe estar en un rango de colores del espacio RGB, puesto que el programa es ajustable para detectarlo en este conjunto. Al igual que la detección gestual de la mano, se utiliza el color de la punta para estimar la posición de ésta para controlar el puntero.

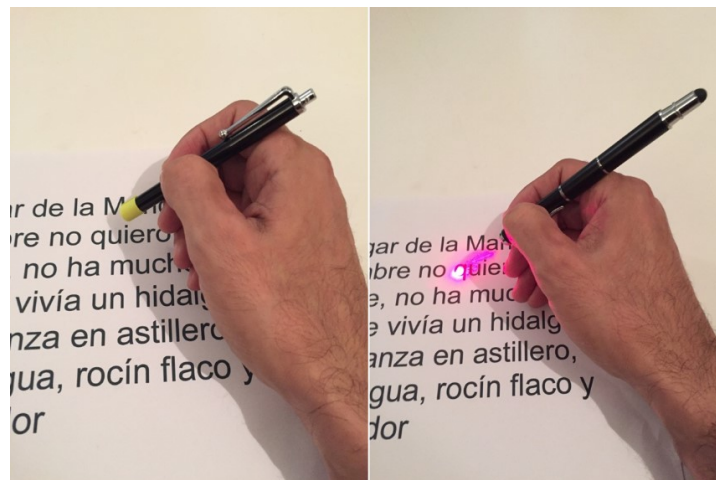


Figura 3.7: Utilización de reconocimiento gestual por lápiz especial.
Fuente: (Contreras, 2019)

A modo de resumen, se presentan los requisitos funcionales trabajados e implementados por Contreras en la Tabla 3.2.

Tabla 3.2: Requisitos funcionales de Contreras.
Fuente: (Contreras, 2019)

ID	Descripción
RFCO1	El sistema, por medio de un proyector, debe desplegar información relevante para un sesión de lectura aumentada. Esta proyección cubre el escritorio y el documento.
RFCO2	El sistema debe posibilitar la interacción entre el usuario y la proyección desplegada sobre el libro y el escritorio.
RFCO3	El sistema debe reconocer el documento físico y sincronizar la información del documento físico con el documento digital.
RFCO4	El sistema debe reconocer el texto de una página del documento físico y generar un texto equivalente en formato digital.
RFCO5	Por medio de un gesto manual sobre el documento físico, el sistema debe permitir seleccionar un fragmento de texto, proyectando un rectángulo de color sobre el texto para resaltarlo.
RFCO6	El sistema debe soportar consistencia de datos para el resaltado de texto entre el documento físico y el digital.
RFCo7	El sistema debe permitir hacer búsquedas de palabras o frases en el documento digital señalando la ubicación en el documento físico, siempre y cuando exista en el sistema un libro digital equivalente al físico.
RFCO8	El sistema debe permitir mostrar resultados de búsquedas de palabras o frases, proyectándolas sobre el escritorio junto al área donde se ubica el documento físico.
RFCO9	El sistema debe soportar reconocimiento gestual por medio del dispositivo de reconocimiento de manos Leap Motion.
RFCO10	El sistema debe soportar reconocimiento gestual por medio de una cámara USB.
RFCO11	El sistema debe soportar reconocimiento de posición de un lápiz especialmente preparado por medio de una cámara USB.
RFCO12	El sistema debe estar construido con una arquitectura modular para incorporar nuevos módulos que permitan la comunicación con otros dispositivos que permitan interactuar con el sistema.

ID	Descripción
RFCO13	El sistema debe soportar la incorporación de nuevos módulos que procesen datos provenientes del sistema y proyecten los resultados en el escritorio otorgando nuevas funcionalidades al sistema.

3.4. Módulo de búsqueda

Este conjunto de funcionalidades desarrolladas e integradas a EDDIE por Montanares, se encarga de aumentar la experiencia de lectura. Para ello, se utiliza un módulo de búsqueda en la web por distintos tipos de API's (Montanares, 2019). Para poder utilizar estas funcionalidades es necesario indicar previamente la palabra o imagen a buscar. Está construido de tal forma que permite la integración de nuevas API's que cumplan diferentes o las mismas funciones. Además, es posible para el usuario escoger la API que desea utilizar para cada función. Actualmente están implementadas cinco funcionalidades:

- **Búsqueda en enciclopedia:** Esta funcionalidad utiliza la API seleccionada por el usuario para buscar en la web una definición enciclopédica. En la actualidad se utiliza la búsqueda en Wikipedia.
- **Búsqueda de definiciones:** Esta funcionalidad se encarga de buscar la definición del diccionario utilizando la API seleccionada por el usuario. La implementación actual de la API de Google solo permite la búsqueda de definiciones en inglés.
- **Búsqueda de traducciones:** Esta funcionalidad, a través de la API seleccionada por el usuario, permite la traducción de la palabra previamente seleccionada. Actualmente se utiliza la API de traducción Bing de Microsoft, la cual tiene un límite de traducción de 20.000 caracteres mensuales por cuenta.
- **Búsqueda de videos:** Al igual que todas las anteriores, esta utiliza una API seleccionada por el usuario para buscar la palabra seleccionada para buscar los videos relacionados. Actualmente se utiliza la API de YouTube.
- **Búsqueda de imágenes:** A diferencia de las anteriores funcionalidades, esta requiere una imagen para buscar. La funcionalidad recibe una imagen, entregando contenido relacionado a esta. Actualmente se utiliza la API de búsqueda de imágenes CloudVision de Google.

Otros aspectos importantes para destacar son las funcionalidades de interacción por voz y lectura de resultados. Ambas funcionan usando la biblioteca *System.speech*, permitiendo que el usuario diga la función de búsqueda o lectura que desea realizar, siendo ejecutada por EDDIE. Estos comandos de voz son los siguientes:

- Buscar Definición.
- Buscar Traducción.
- Buscar Enciclopedia.
- Buscar video.
- Buscar Imagen.
- Lectura Definición.
- Lectura Traducción.
- Lectura Enciclopedia.

A modo de cierre, se presentan los requisitos funcionales asociados a la tesis de Montanares en la Tabla 3.3.

Tabla 3.3: Requisitos funcionales de Montanares.
Fuente: (Montanares, 2019)

ID	Descripción
RFM1	Acceder a recursos de información obtenidos desde múltiples fuentes en línea
RFM2	Acceder a traductor de contenido.
RFM3	Acceder a definiciones de contenido.
RFM4	Acceder a videos.
RFM5	Acceder a enciclopedia.
RFM6	Realizar búsqueda de contenido mediante imágenes.
RFM7	Escoger entre un grupo de API´s cuál utilizar para cada servicio proporcionado por el módulo de búsqueda web
RFM8	Visualizar los recursos de información digital obtenidos.
RFM9	Interactuar con los recursos de información digital obtenidos.
RFM10	Desplegar/ocultar los recursos de información digital obtenidos desde la web.
RFM11	Tener un registro de las búsquedas realizadas durante una sesión de lectura.
RFM12	Acceder a recursos de información en la web mediante comandos de voz.
RFM13	Acceso a un lector de información.

3.5. Módulo de consistencia

Uno de los aspectos que hace que EDDIE sea una propuesta distintiva para la investigación del efecto de la lectura aumentada en la comprensión lectora es el módulo de consistencia (Pezoa, 2020). Se desarrollaron seis prototipos para este módulo, implementando las funcionalidades encargadas de la sincronización de los elementos físicos y digitales del texto analizado, las cuales son:

- **Sincronización de figuras desde físico a digital:** A través de esta funcionalidad se detectan las figuras del documento físico, las cuales deben estar en color rosado. Una vez realizado esto, se aplica un filtro de rosado para obtener las figuras gracias a la biblioteca *Emgu CV*. Para terminar el proceso, se detectan las coordenadas y dimensiones de las figuras y se añaden al archivo .pdf, como se puede ver en la Figura 3.8.

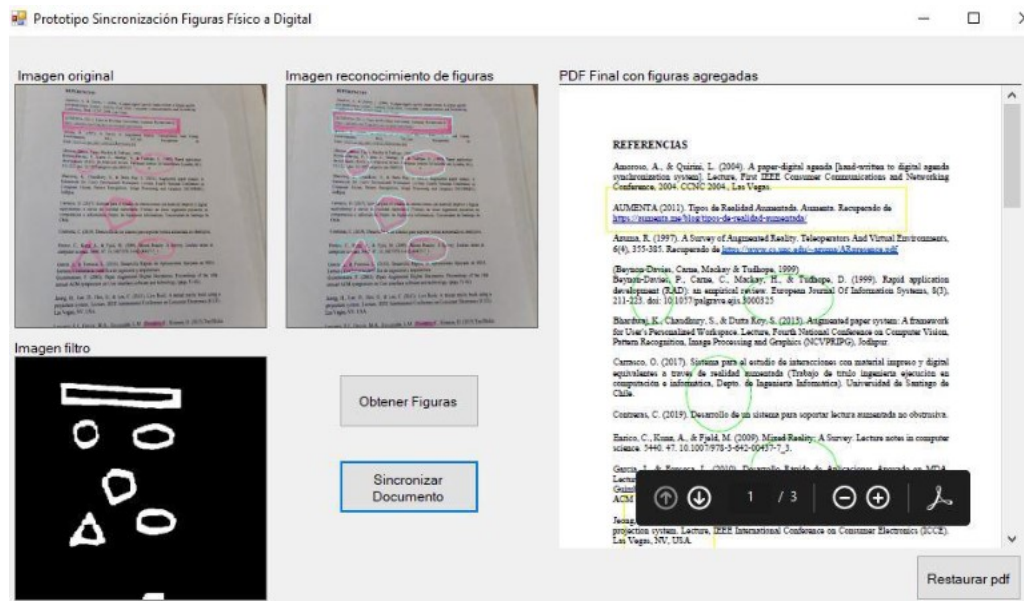


Figura 3.8: Consistencia de figuras desde físico a digital.

Fuente: (Pezoa, 2020)

- **Sincronización de comentarios desde físico a digital:** Para poder utilizar esto es necesario haber escrito previamente un comentario en una nota adhesiva amarilla y ponerla en el documento físico. Cuando se utiliza esta funcionalidad, el programa detecta el espacio amarillo y copia esta imagen en el documento digital, como se puede ver en la Figura 3.9.

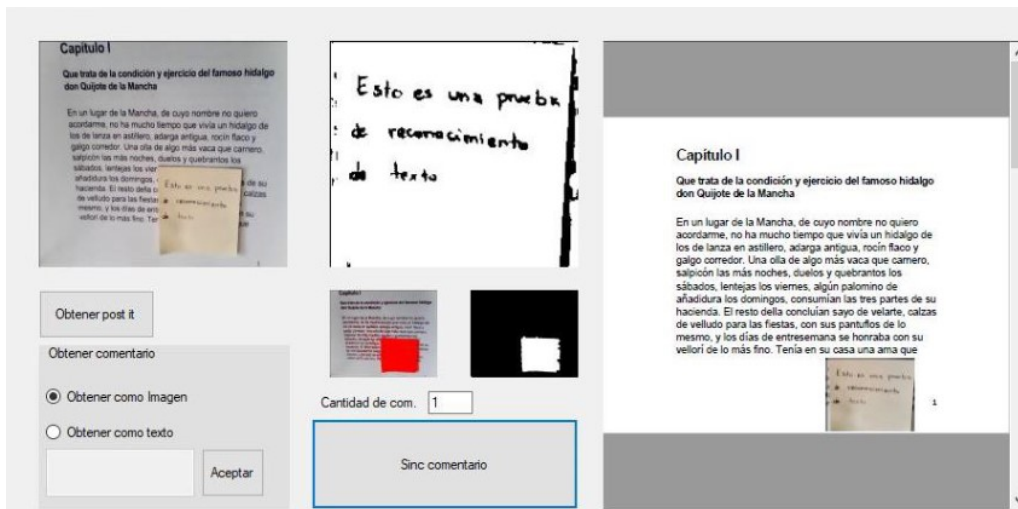


Figura 3.9: Consistencia de comentarios desde físico a digital.

Fuente: (Pezoa, 2020)

- **Sincronización de marca páginas desde físico a digital:** Al igual que en los comentarios, es necesario tomar una banderilla, en este caso azul, y posicionarla sobre el texto físico. Se utiliza un filtro de azules para poder reconocer el tamaño y posición del marcador y posicionarlo en el documento digital, como se puede ver en la Figura 3.10.

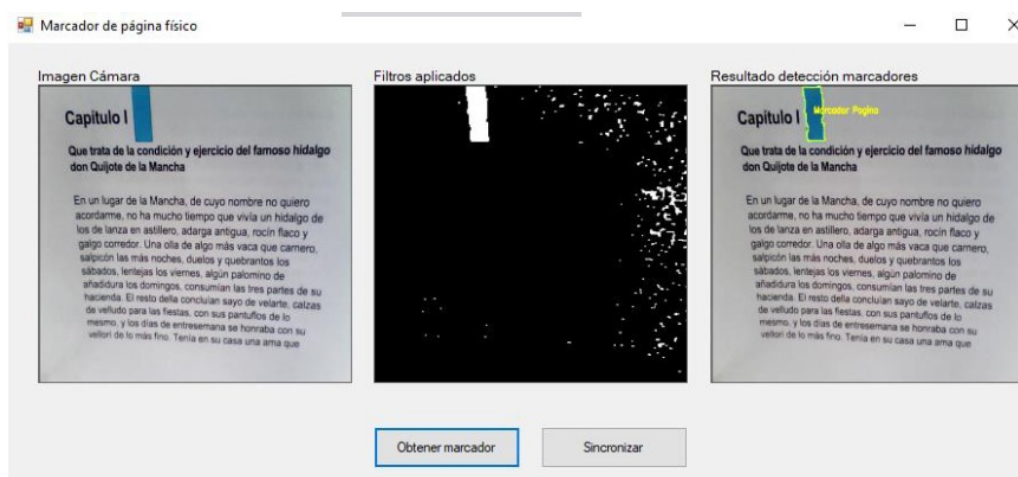


Figura 3.10: Consistencia de marca páginas desde físico a digital.

Fuente: (Pezoa, 2020)

- **Extracción de información de documento digital a físico:** Utilizando la biblioteca *ItexSharp*, se obtienen los comentarios y marca páginas, ubicándolos en la esquina superior izquierda del documento. Además, las figuras se ubican en una posición similar al documento digital, tal como se ve en la Figura 3.11.

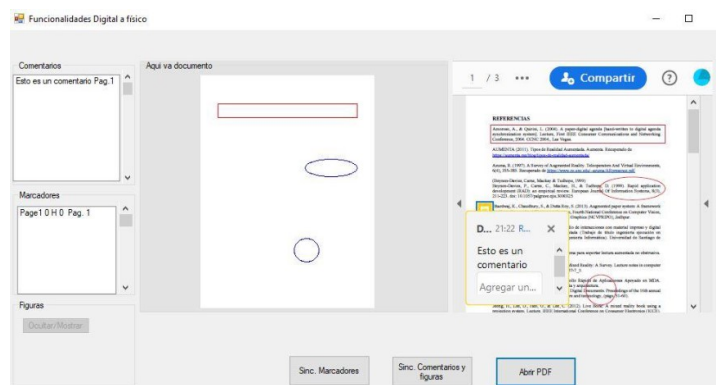


Figura 3.11: Obtención de información desde documento digital a físico.
Fuente: (Pezoa, 2020)

A modo de resumen, se presentan los requisitos funcionales presentados en la tesis de Pezoa, como se puede apreciar en la Tabla 3.4.

Tabla 3.4: Requisitos funcionales de Pezoa.
Fuente: (Pezoa, 2020)

ID	Descripción
RFP1	El sistema debe permitir detectar figuras realizadas en el documento físico y estas deben verse reflejadas en la versión digital del mismo.
RFP2	El sistema debe detectar comentarios en el documento físico y reflejarlos en su versión digital.
RFP3	El sistema debe permitir agregar un marcador de página al documento físico, por medio de una banderilla o post-it azul, el cual debe verse reflejado en la versión digital del documento.
RFP4	El sistema debe permitir eliminar un marcador de página del documento físico, quitando el marcador y reflejando esto en la versión digital.
RFP5	El sistema debe permitir detectar figuras en una página determinada del documento digital y estas se deben reflejar en la versión física del mismo.
RFP6	El sistema debe permitir detectar comentarios realizados en una página determinada del documento digital y reflejar estos en el formato físico.

ID	Descripción
RFP7	El sistema debe permitir detectar marcadores de página en el documento digital y desplegar la información asociada a los mismos.
RFP8	El sistema debe permitir activar o desactivar la visualización de figuras sobrepuestos en el documento físico.
RFP9	El sistema debe permitir eliminar un marcador de página creado en el documento digital actualizando la información en el documento digital.
RFP10	El sistema debe permitir eliminar un comentario realizado en el documento digital actualizando la información en el documento digital.

3.6. Módulo de rastreo ocular

Otro de los módulos de EDDIE es el de rastreo ocular desarrollado por Estay a lo largo de cinco prototipos (Estay, 2020). Es importante resaltar que este programa fue desarrollado sin un *eyetracker*, siendo probado con una cámara digital. Las funcionalidades desarrolladas son las siguientes:

- **Rastreo de movimientos oculares:** Como se mencionó antes, se desarrolló pensando en una cámara digital, por lo cual se utilizó la API *GazeCloudAPI* para representar los movimientos oculares del usuario como coordenadas X e Y, usando el punto inicial (0,0) como la esquina superior izquierda. Para implementar en EDDIE esta API utilizando una cámara digital, se utiliza la biblioteca *WatsonWebSocket* que habilita la comunicación entre *c#* y el navegador, donde se ejecuta el rastreo ocular.
- **Interacciones por medio de movimientos oculares:** Utilizando las coordenadas obtenidas se utiliza el puntero del *mouse* a través de los movimientos oculares. Además, utilizando la biblioteca *user32.dll*, se puede fijar la mirada sobre un botón de la aplicación, iniciando una cuenta regresiva. Cuando termina lo anterior se ejecuta un click.
- **Registro de datos de rastreo ocular:** Esta funcionalidad crea un archivo nombrado por el usuario, también la ruta donde se guarda este archivo es designada por el usuario. Este archivo guarda la información de los datos relativos a la captura del rastreo ocular: posición X, posición Y y un timestamp.

Finalmente, se presentan los requisitos funcionales desarrollados en la tesis de Estay en la Tabla 3.5.

Tabla 3.5: Requisitos funcionales de Estay. (Estay, 2020)

ID	Descripción
RFE1	El módulo debe ser capaz de rastrear los movimientos oculares del usuario.
RFE2	El módulo debe permitir la visualización de un componente gráfico que represente las fijaciones del usuario.
RFE3	El módulo debe permitir la interacción del usuario con el sistema por medio de sus movimientos oculares.
RFE4	El módulo debe permitir registrar los datos derivados del rastreo ocular.
RFE5	El módulo debe permitir la integración de distintos dispositivos de rastreo ocular.
RFE6	Creación de un módulo de registro de datos el cual pueda ser usado por cualquier componente de la aplicación.
RFE7	El módulo debe permitir al usuario de la plataforma seleccionar el dispositivo de rastreo ocular que será utilizado.

3.7. Rediseño

Finalmente, al ser el propósito de EDDIE su uso para la investigación con usuarios, es necesario dar énfasis a su interfaz de usuario. Para ello fue necesario un rediseño realizado por Santibáñez (2020) a través de siete prototipos. Además del completo cambio visual realizado, se implementaron cambios en las dos interfaces principales:

- **Interfaz de configuración:** Esta cuenta con una pestaña para cada funcionalidad de EDDIE: rastreo ocular, API para las búsquedas web, rastreo gestual, ingresar documento .pdf para consistencia y reconocimiento de texto. Uno de los cambios a destacar es que no es necesario que el usuario ingrese cada parámetro de cada funcionalidad para inicializar la proyección; en la nueva interfaz estos parámetros cuentan con un valor por defecto por lo cual se puede inicializar la proyección apenas se inicia el software. En caso de necesitarlo, el usuario puede ir a cada pestaña de configuración e ingresar manualmente los cambios que desee. Para implementar estos cambios es necesario guardar los cambios realizados. Esta estructura se puede ver en la Figura 3.12.

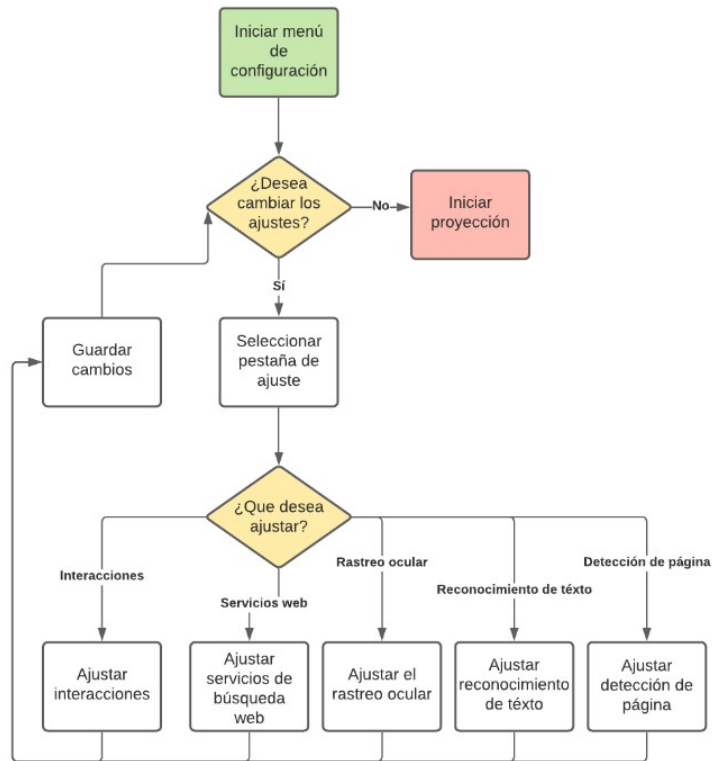


Figura 3.12: Diagrama de flujo para la interfaz de configuración.
Fuente: (Santibáñez, 2020)

■ **Interfaz de realidad aumentada:** Se realizaron cinco cambios significativos, estos son:

- Mensajes de advertencia cuando el usuario intente cerrar una ventana y para la eliminación del contenido.
- Mensajes que informan al usuario cuales fueron los servicios que este eligió.
- Animación de carga para el inicio de la proyección.
- Utilización de iconos que reflejan de mejor manera la funcionalidad que representan.
- Nueva paleta de colores.
- Líneas que trazan desde los botones de búsqueda web hasta el panel de desplegado de contenido, con el propósito de indicar donde se esta desplegando el resultado de su acción.

A modo de cierre, se presentan los requisitos funcionales del trabajo de Santibáñez, como se puede ver en la Tabla 3.6.

Tabla 3.6: Requisitos funcionales de Santibáñez.
Fuente:(Santibáñez,2020)

ID	Descripción
RFS1	Acceder a las funcionalidades de lectura aumentada.
RFS2	Acceder a las funcionalidades de la búsqueda web.
RFS3	Acceder a las funcionalidades de consistencia de documentos.
RFS4	Acceder a las funcionalidades de rastreo ocular.
RFS5	Configurar el funcionamiento del programa.
RFS6	Navegar por las configuraciones de cada módulo de EDDIE.
RFS7	Guardar todas las configuraciones del sistema.
RFS8	Reiniciar las configuraciones del sistema.

3.8. Resumen

Se presenta un análisis a la herramienta EDDIE, describiendo sus características, funcionamiento y los avances de cada integración realizada a lo largo de su desarrollo. Para mostrar todo lo anterior se expone la historia de la herramienta, definiendo cuales son las funcionalidades de cada producto desarrollado, como se utilizan y cuales son los requisitos funcionales asociados. En las seis tesis asociadas, EDDIE cuenta con cinco módulos y 60 requisitos funcionales.

4. MECIVA PS

4.1. Metodología

La Metodología para la evaluación y Validación de Productos de *Software* (MECIVA PS), propuesta en el presente trabajo de tesis, consta de tres fases inspiradas en la metodología GOMS (Karat, 1988), utilizando sus propiedades de descomposición y unión. MECIVA PS cuenta con una guía para futuros desarrolladores, la cual se encuentra en un documento en línea almacenado en el dominio de Gitbook ¹. Las fases son:

- **Despliegue:** En esta sección se presentan los pasos a seguir para descargar y ejecutar EDDIE para que el usuario sea capaz de implementar las nuevas funcionalidades dentro de la herramienta. Esta sección se divide en el despliegue de la herramienta EDDIE en su totalidad y el de la nueva funcionalidad.
- **Reparación:** Esta fase solo se realiza en caso de encontrar uno o más errores o aspectos por mejorar. Por cada desafío detectado, el desarrollador debe implementar una serie de pasos, definidos posteriormente en el documento, para identificar las causas que originaron los problemas. Lo anterior con el propósito de la mejor experiencia posible para el usuario y la obtención de datos.
- **Diagnóstico:** En esta fase se revisan todas las funcionalidades nuevas añadidas, generando una lista con todas ellas indicando su nivel de funcionamiento. En caso de que las funcionalidades puedan ser ejecutadas, estas deben ser sometidas a diversos métodos para su evaluación y validación, obteniendo una lista de mejoras en caso de que no cumplan con los estándares establecidos por el jefe de proyecto. La elección de estos métodos depende de la naturaleza de cada funcionalidad.

Ante el desarrollo de una nueva funcionalidad de EDDIE, se debe aplicar la metodología MECIVA PS, detallada en las siguientes subsecciones. Primero se realiza la fase de despliegue para poder integrar y manejar la nueva funcionalidad en EDDIE. Una vez realizada esta fase, en caso de que existan errores que impidan la ejecución de esta, se obtiene una lista de errores los cuales deben ser reparados y se debe pasar a la fase de reparación. En caso de que se presenten nuevos errores, se repite el proceso de la fase de reparación. Si no existen errores, tanto al terminar la fase de despliegue como de reparación, se debe pasar a la etapa de diagnóstico, donde se verifica el nivel de funcionalidad que presenta la nueva función. En caso de existir algún error, este

¹<https://usach-1.gitbook.io/meciva-ps/>

debe ser corregido pasando a la fase de reparación nuevamente. En caso contrario, se deben seleccionar e implementar métodos para la validación y evaluación, verificando si cumplen con las expectativas del jefe de proyecto. Si no lo hacen, se genera una lista de mejoras las cuales pasan a la fase de reparación para reparar la funcionalidad y que este al nivel que se desea para una investigación. Finalmente, si cumplen con las exigencias del jefe de proyecto, se documenta todo el proceso y se da por finalizado el desarrollo de estas funcionalidades. Todo este proceso se puede ver reflejado en la Figura 4.1.

Para cerrar esta sección, es importante destacar que esta metodología fue desarrollada mientras se realizó la revisión y solución de problemas de EDDIE , madurándola en conjunto a las situaciones encontradas, esto es detallado en el capítulo de 'Aplicación de la metodología'.

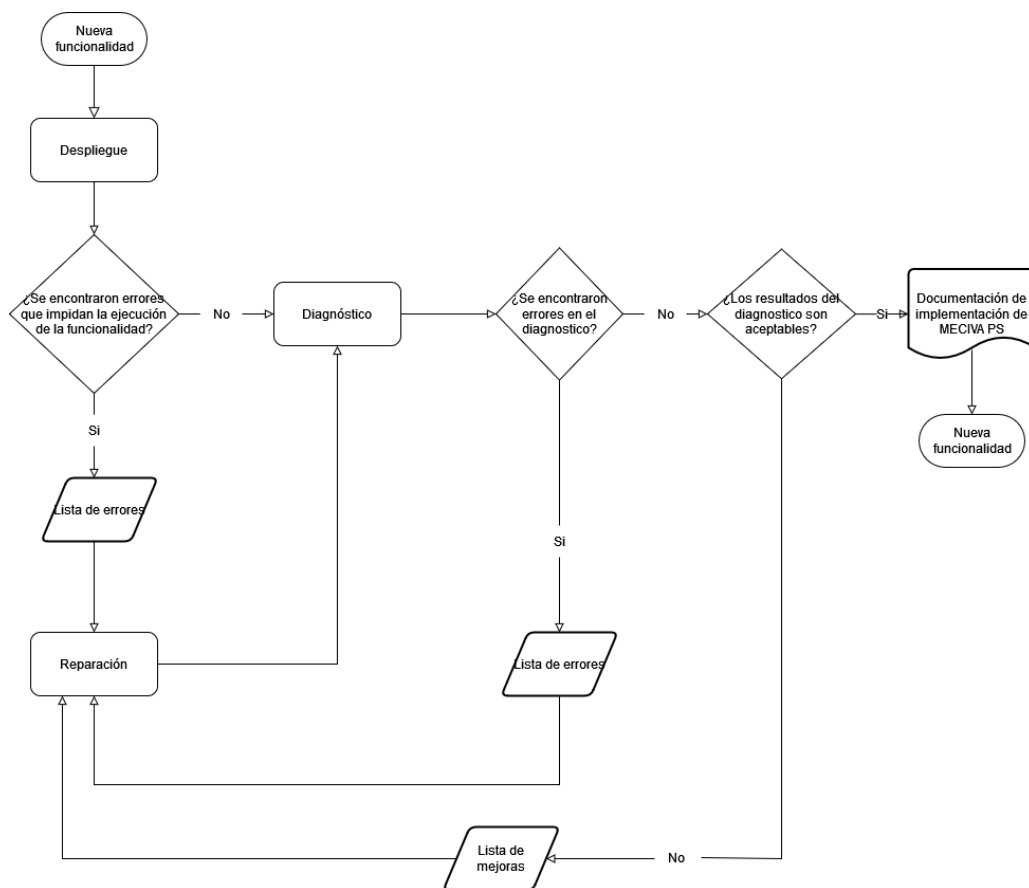


Figura 4.1: Diagrama de flujo: Implementación de MECIVA PS.

Fuente: (Autoría propia, 2023)

A continuación, se presenta el detalle de las fases de MECIVA PS, utilizando ejemplos para su mayor comprensión.

4.2. Fase de despliegue

4.2.1. Despliegue de EDDIE

Al iniciar el despliegue de la nueva funcionalidad de la herramienta EDDIE, es necesario que este software este previamente en el computador y que sea correctamente ejecutable, por lo cual se indican los pasos a seguir. Es prerequisite tener instalado Visual Studio (Microsoft, sf), en su versión 4.6.1 o superior, luego se debe abrir el código como solución. A la hora de ejecutar EDDIE se encontraron una serie de errores y problemas los cuales se podrían producir nuevamente dadas las características de Visual Studio, para prevenir lo anterior se deben seguir una serie de pasos:

- **Preparación del software:** Para ejecutar correctamente EDDIE es necesario realizar lo siguiente:
 - **Descarga de la herramienta:** Es necesario que el desarrollador descargue la última versión disponible del código. A la hora de la realización del presente documento, esta se encontraba en el repositorio de GitHub, perteneciente al laboratorio de InTeracTiOn, llamado 'EDDIE-NewGUI2-2020'².
 - **Actualizar referencias:** Es importante revisar de que todas las referencias utilizadas por EDDIE en Visual Studio estén correctamente asignadas. En caso de no estarlo, estas deben ser corregidas manualmente por el desarrollador.
 - **Verificar el correcto funcionamiento de los paquetes:** Es necesario verificar el funcionamiento de los paquetes utilizados por EDDIE, como lo es *serilog*, para esto es necesario ejecutar el programa para que Visual Studio indique su estado. En caso de no funcionar correctamente, el paquete debe ser reinstalado a través del administrador de paquetes nuget, a través de Visual Studio. Si el método anterior no funciona se debe instalar manualmente, esto a través de algún recurso del laboratorio de InTeracTiOn o página de internet.
 - **Verificación de archivos .dll:** Al ser EDDIE desarrollado de manera modular, algunos de sus módulos deben ser conectados a través de archivos .dll. Es necesario consultar la documentación disponible en el laboratorio de InTeracTiOn para saber cuáles son estos módulos, donde encontrar los archivos .dll y donde deben ser puestos dentro del software para que este funcione.

²<https://github.com/EDDIE-InTeracTiOn/EDDIE-NewGUI2-2020/tree/newGUI>

A modo de ejemplo: Para ejecutar las funcionalidades del módulo de consistencia es necesario obtener los archivos .dll correspondientes. Para esto se deben compilar todos sus plugins: *ConsistencyLibraryCommentsDP*, *ConsistencyLibraryCommentsPD*, *ConsistencyLibraryFiguresDP*, *ConsistencyLibraryFiguresPD*, *ConsistencyLibraryPageMarkerDP* y *ConsistencyLibraryPageMarkerPD*. Una vez realizado esto, por cada uno se debe ingresar a su carpeta correspondiente y seguir la ruta '*bin/Debug/ netstandard2.0*' donde se encuentran los archivos .dll. Finalmente, se ingresa a la carpeta del proyecto y se ingresa a la ruta '*bin/x86/Debug/PluginsConsistencia*', donde se depositan todos los archivos .dll generados.

- **Preparación de *hardware*:** El espacio a utilizar es la sala de experimentación del Departamento de Ingeniería Informática, el cual cuenta con el computador donde se despliega la herramienta, la mesa para trabajar y el soporte metálico alrededor de esta, estos elementos son visibles en las Figuras 4.2 y 4.3. Es necesario conectar 2 cámaras digitales, un *eyetracker* y un proyector. Una cámara digital es implementada para el reconocimiento de texto y la otra para el reconocimiento gestual. Es necesario desplegar el *hardware* de una manera específica, en caso contrario la herramienta no funcionara correctamente. El proyector debe ser colocado con un agarre a 1,15 metros de altura del escritorio. La cámara, tanto para el texto como para el reconocimiento gestual deben ser ubicadas sin obstruir la proyección ni el espacio donde opera el usuario con la interfaz, por lo tanto, deben ser colocadas en la parte trasera del escritorio a una altura de 25cm, como indica la Figura 4.4. El mismo caso es aplicable al *eyetracker*, como se puede apreciar en la Figura 4.5 donde se utiliza el *eye tracker GazePoint*. La utilización de imágenes reales es para dar una mejor perspectiva para el futuro desarrollador.



Figura 4.2: Sala de experimentación del Departamento de ingeniería informática.
Fuente: (Autoría propia, 2023)



Figura 4.3: Computador de la sala de experimentación del Departamento de ingeniería informática
Fuente: (Autoría propia, 2023)

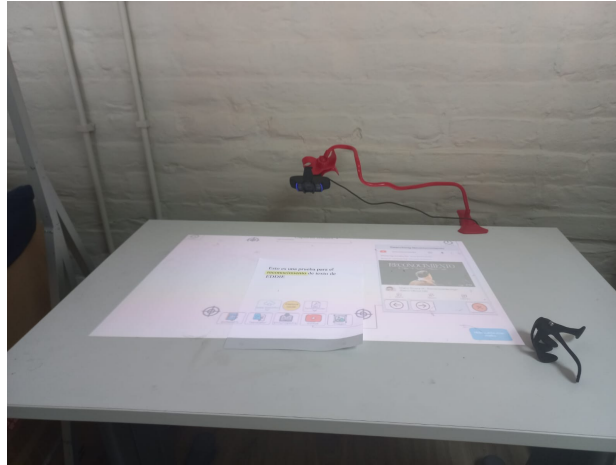


Figura 4.4: Posicionamiento de la cámara digital.
Fuente: (Autoría propia, 2023)

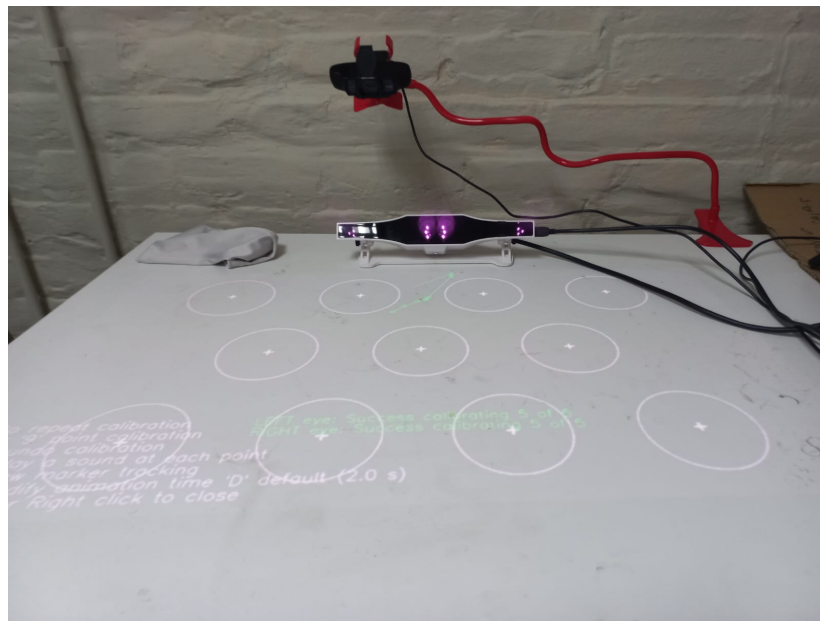


Figura 4.5: Posicionamiento del eyetracker *GazePoint*.
Fuente: (Autoría propia, 2023)

4.2.2. Despliegue de la funcionalidad

Una vez desplegado el software principal, es necesario reconocer ciertos aspectos de la funcionalidad que se desea aplicar a la herramienta EDDIE. Para cumplir con el anterior propósito se procede a desglosar los elementos necesarios para la implementación de la funcionalidad. Para lo anterior es necesario lo siguiente:

- **Identificar el módulo al que corresponde la funcionalidad:** Es necesario reconocer en totalidad las características de la funcionalidad, pues con esta informa-

ción el desarrollador puede seleccionar la mejor ubicación para desplegarla. Para su selección, se debe verificar si existe una interfaz que contenga funcionalidades de características similares o si es necesario desarrollar una nueva. En caso positivo, esta funcionalidad debe ser incluida en la sección pertinente, identificando la posición de la interfaz donde debe ser implementada y desplegada junto a las demás funcionalidades que ya se encontraban previamente en esa sección de EDDIE. Un ejemplo de lo anterior: Se quiere implementar una funcionalidad que, a través de una palabra destacada, busque en internet todas las imágenes relacionadas con dicha palabra. Esto es identificable como una funcionalidad que corresponde al módulo de búsqueda y debe ser desplegado junto a las demás funcionalidades de dicho módulo. En caso que la funcionalidad no pertenezca a un módulo preexistente, se debe implementar una nueva sección donde agregar la funcionalidad.

- **Determinar el tipo de conexión de la funcionalidad:** Es importante elegir la mejor manera para conectar la funcionalidad con la herramienta, puesto que esta es una nueva adición a la herramienta. Como la herramienta EDDIE está desarrollada de manera modular, existen diversas formas de implementar funcionalidades dependiendo de sus características. (ej: Si se conecta con alguna API, necesita un *hardware* específico, ser conectado con un *plugin*, entre otros.)
- **Identificar los componentes existentes relacionados a la funcionalidad:** EDDIE cuenta con diversos módulos implementados, por lo que es posible que elementos de la nueva funcionalidad ya estén siendo utilizados actualmente por la herramienta. Para esto es necesario conocer cuáles son las bibliotecas, APIs y variables que se utilizan en la interfaz de despliegue de la funcionalidad. Para verificar que se este realizando correctamente, el desarrollador debe tener conocimiento de la documentación de los trabajos anteriores. Aún así, se puede consultar con el jefe de proyectos. Un ejemplo de lo anterior es que, si la nueva funcionalidad necesita ejecutarse mediante la identificación de un elemento encontrado en la zona de proyección, esta puede utilizar la misma cámara que se usa en la identificación de texto o el reconocimiento gestual, utilizando la misma variable dentro del código. Con esto se reduce la cantidad de cámaras necesarias para utilizar EDDIE, mejorando la experiencia no obstructiva para el usuario.
- **Determinar el nivel de acoplamiento de la funcionalidad:** Este punto se refiere a tener el conocimiento sobre si la funcionalidad puede ser implementada individualmente o puede ser ejecutada en un conjunto de funciones, es decir, que

una acción del usuario desencadene la ejecución de múltiples funcionalidades. Para esto es necesario reconocer las características de las funciones que ya se encuentran en el módulo. Un ejemplo implementado actualmente en la herramienta EDDIE está en el módulo de consistencia, donde se encuentra la opción de mostrar todos los elementos de un archivo .pdf ingresado, siendo estos: figuras, comentarios y marca páginas. Esta función es un conjunto de tres funcionalidades que operan por separado pero desplegadas a la vez por el usuario: 1. identificación de figuras desde digital a físico, 2. identificación de comentarios desde digital a físico y 3. identificación de marca páginas desde digital a físico. Una vez identificado, se crea el elemento necesario para la ejecución de la o las funcionalidades, estos pueden ser botones, ejecución por voz o cualquier otra opción de despliegue pertinente.

- **Identificar el impacto en otras funcionalidades:** En algunos casos, al implementar una nueva funcionalidad, otras pueden verse parcial o completamente afectadas siendo necesaria su actualización. Un ejemplo de lo anterior es la funcionalidad de reconocimiento por voz, la cual ejecuta funcionalidades relacionadas al módulo de búsqueda a través de comandos de voz, por lo tanto, si se crea una nueva funcionalidad relacionada a este módulo, el reconocimiento de voz debe ser actualizado.

Una vez terminado este proceso, se despliega la nueva funcionalidad en la herramienta. Todo el proceso se puede ver reflejado en la Figura 4.6.

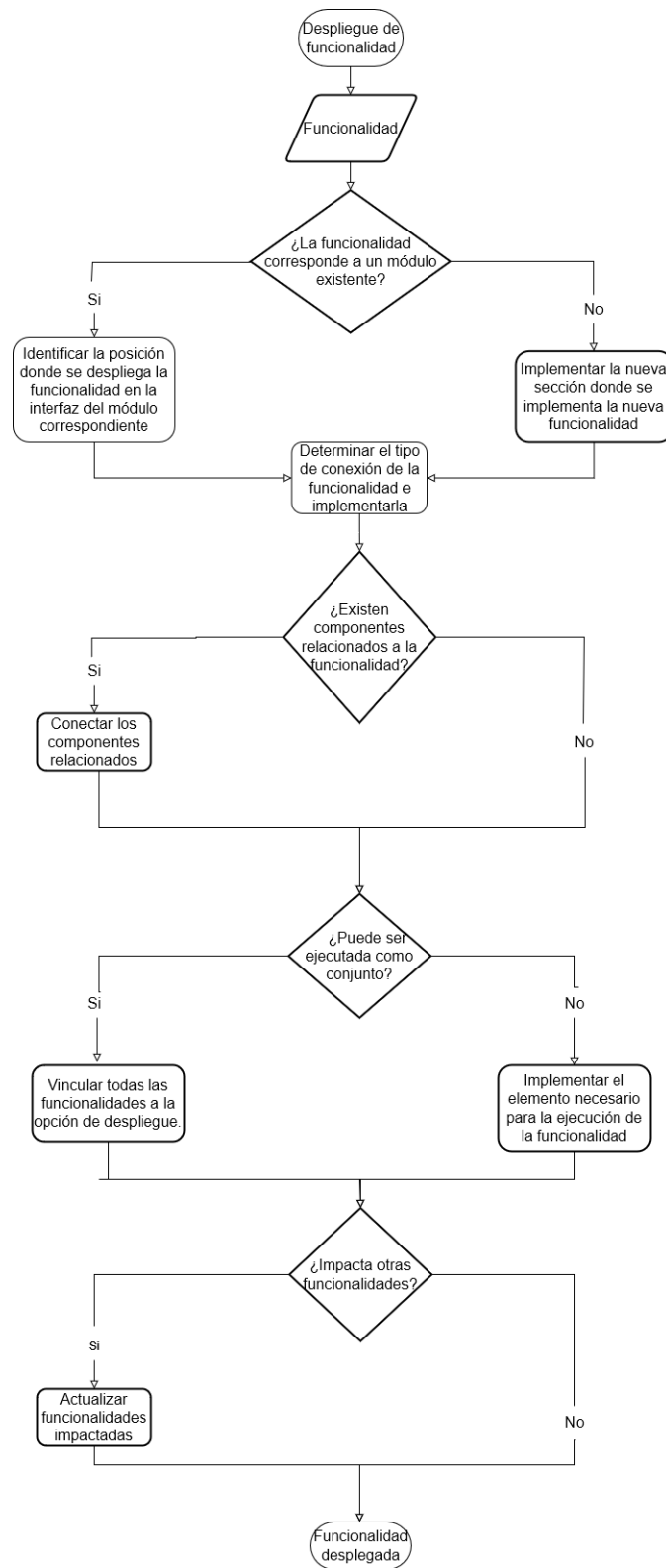


Figura 4.6: Diagrama de flujo de la fase despliegue de funcionalidad en EDDIE.
Fuente: (Autoría propia, 2023)

4.3. Fase de reparación

Es importante reparar errores, independientemente de si son o no lo suficientemente influyentes como para impedir la ejecución de la funcionalidad. Además, es necesario realizar mejoras para poder cumplir con los estándares de desarrollo. Para poder implementar esta fase, se aplica la descomposición de factores, identificando los distintos elementos asociados al problema y encontrando su solución. A esta fase entra una lista de problemas/mejoras, los cuales se abarcan de uno en uno. Una vez que se termine de corregir uno, se procede al siguiente. Para poder reparar el problema presente en una funcionalidad de la herramienta EDDIE se deben realizar los pasos a continuación:

- **Categorizar el error presentado:** Para poder comenzar a manejar el error, es importante conocer sus características. Para poder reducir el espacio donde buscar, se debe categorizar el error como interno o externo a EDDIE. Errores externos a EDDIE se subdividen en:
 - **Software:** Todos los problemas y errores encontrados relacionados al software externo a EDDIE.
 - **Hardware:** Todos los problemas y errores encontrados relacionados a los dispositivos que son necesarios para el uso de EDDIE. Un ejemplo de dispositivo es la cámara digital utilizada para el reconocimiento de texto.
 - **Otro:** Todos los problemas y errores encontrados relacionados a aspectos indirectamente vinculados a EDDIE. Un ejemplo es algún error relacionado con el ambiente de desarrollo, como lo es la falta de conexiones eléctricas.

Los internos se dividen en siete categorías:

- **Consistencia:** Todos los problemas y errores encontrados relacionados con el módulo de consistencia en EDDIE.
- **Rastreo ocular:** Todos los problemas y errores encontrados relacionados con el módulo de rastreo ocular en EDDIE.
- **Búsqueda:** Todos los problemas y errores encontrados relacionados con el módulo de búsqueda en EDDIE.
- **Interacción físico/digital:** Todos los problemas y errores encontrados relacionados con el módulo de interacción físico/digital en EDDIE.
- **Reconocimiento gestual:** Todos los problemas y errores encontrados relacionados con el módulo de reconocimiento gestual.

- **Interfaz de usuario:** Todos los problemas y errores encontrados relacionados a la interfaz de usuario.
- **Otro:** Todos los problemas y errores encontrados no relacionados un módulo específico de EDDIE.

Estas categorías son aplicables a la versión de EDDIE existente al momento de la realización este documento y son expandibles según crezca EDDIE. Una vez categorizado, se puede saber cuáles son las características que puede y no puede presentar el error.

- **Identificar las características del error:** Para la identificación de características se debe tener conocimiento de como se comporta la categoría, puesto que se deben reconocer las APIs, bibliotecas y variables utilizadas para saber como debe comportarse y de donde puede surgir el error. Por ejemplo: si falla una funcionalidad categorizada en el reconocimiento ocular, se debe revisar el correcto funcionamiento del *eyetracker*, las bibliotecas utilizadas en el rastreo ocular, que estén conectadas correctamente las variables entre las funciones asociadas, la implementación de los archivos .dll, entre otros.
- **Identificar los tipos posibles del error:** Al generarse un error, es importante identificar su tipo. Estos pueden ser evidentes como los que informa directamente VisualStudio al presentarse un error que impide la ejecución de la funcionalidad, o los que deben ser determinados por el desarrollador. Un ejemplo presentado durante la evaluación de la herramienta, en el marco de esta tesis, es que la funcionalidad de reconocimiento de texto funcionaba sin arrojar errores que impidan la ejecución, sin embargo, éste no reconoció correctamente el texto. Dos tipos posibles de errores fueron detectados: error de implementación de biblioteca de reconocimiento y error de posición de cámara.
- **Determinar los componentes relacionados al error:** Una vez analizadas las características y tipos posibles del error, se deben determinar los componentes que están directamente asociados. En esta etapa se hace un seguimiento del error y se determina si realmente se produce donde parece. Un problema ocurrido durante la revisión de la herramienta fue la no ejecución del reconocimiento de comentarios. El *software* indicó que una variable presentó valor null (tipo de error). Se realizó el seguimiento de esta variable, teniendo el conocimiento previo de que viene desde el módulo de consistencia (categorización), y que este error está relacionado con la cámara (característica). Se identificó que no se ejecutaba la detección de la cá-

mara, por lo cual no se asignaba un valor a la variable, obteniendo así el error de origen.

- **Determinar y seleccionar la fuente necesaria para abordar el error:** Al determinar el error de origen, se debe determinar cuál es la documentación disponible relacionada con ello. Para una herramienta de gran tamaño como lo es EDDIE, existen diversas fuentes donde se puede encontrar la solución como lo son: Trabajos de tesis anteriores, documentación de C#, comentarios en el código, entre otros. Estas fuentes no están limitadas a únicamente textos de documentación; como se puede apreciar en los ejemplos provistos, pueden encontrarse en el mismo código. Además, el error puede tener una característica física, por lo cual puede ser necesario buscar la solución dentro del personal de la universidad, el jefe de proyecto, entre otros. También es importante resaltar que puede ser un error de traspaso, así que es importante tener en consideración la revisión de versiones previas de EDDIE. Una vez seleccionada la documentación a examinar, se debe obtener una o más posibles soluciones. En caso de no encontrarla, se debe seleccionar una documentación distinta que no se utilizó previamente.
- **Aplicar la solución encontrada:** Cuando se encuentran las posibles soluciones, se deben aplicar una a una, hasta que el error sea corregido, dejando implementada únicamente la solución final. En caso de que una de las encontradas en el conjunto de soluciones posibles no resuelva el error, se debe proceder a consultar la documentación nuevamente.

Finalmente, si al solucionar un error surge uno nuevo, se debe repetir todo el proceso, en caso contrario se revisa el siguiente error. El proceso debe ser realizado para cada error de la lista de errores/mejoras que se tenían al iniciar la fase. Si no quedan más errores por reparar, se termina la fase y se procede a la fase de diagnóstico. El proceso presentado en la fase de reparación puede ser visto en la Figura 4.7.

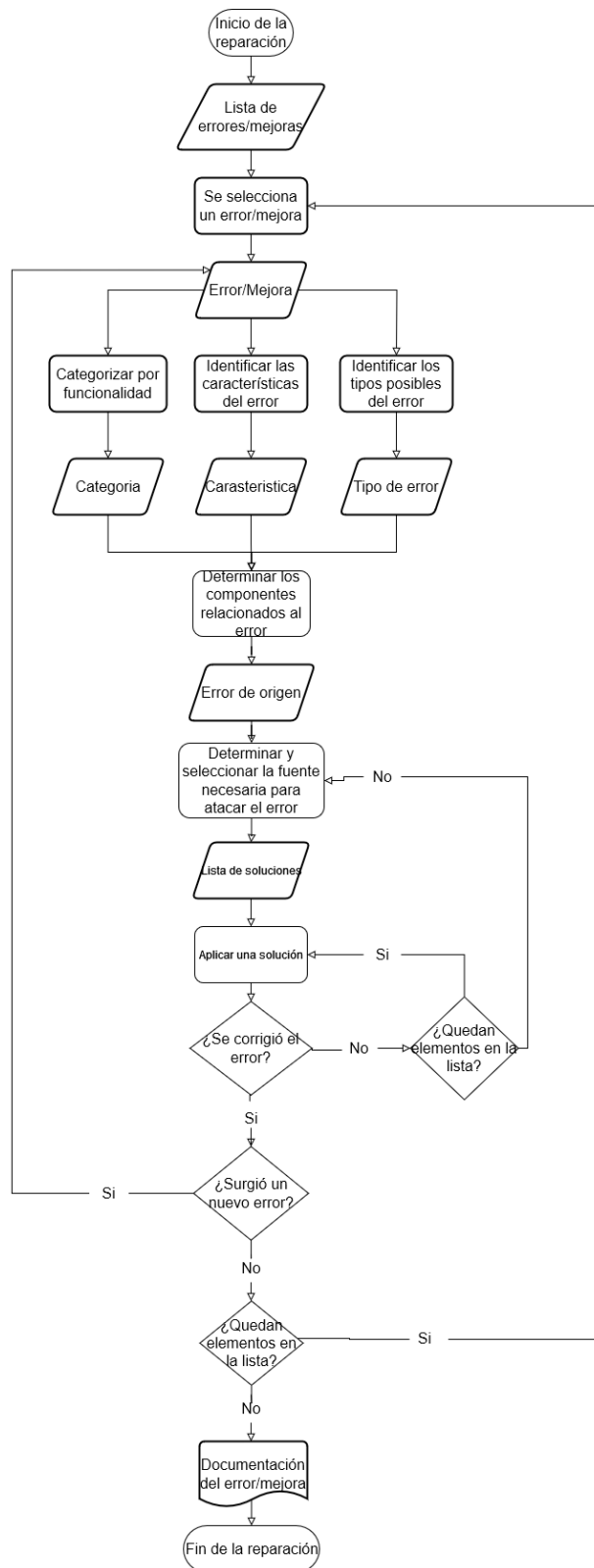


Figura 4.7: Diagrama de flujo de la fase de reparación.
Fuente: (Autoría propia, 2023)

Para documentar, se utiliza una tabla con la estructura de la Tabla 4.1. Todos los errores/mejoras se encuentran en una misma tabla, la cual almacena todos los que compartan la misma subcategoría. Los elementos de la tabla de documentación son los siguientes:

- **ID:** Dígito verificador del error, definido por el usuario.
- **Descripción:** En este parámetro el desarrollador debe describir cual es el error inicial.
- **Estado:** Se indica con un parámetro el nivel de resolución del error/mejora. Estos parámetros pueden tomar los siguientes valores:
 - **SS (0):** Sin solución. No se pudo solucionar el error/problema.
 - **SP (1):** Solución parcial. Se pudo resolver parcialmente el error/problema o se encontró la solución, pero escapa a los alcances de la tesis.
 - **SF (2):** Solución final. Se resolvió en su completitud el error/problema.
- **Justificación o Solución:** Se explica cuál es la justificación por la cual el error esta sin resolver completamente. Esto es especialmente útil para tener en cuenta el trabajo futuro. En caso de estar resuelto, se indica brevemente cual es la solución final. Esto es de ayuda para los desarrolladores futuros, puesto que pueden encontrar una solución a un problema similar.

Tabla 4.1: Plantilla para la documentación de la fase de reparación.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución

4.4. Fase de diagnóstico

Esta fase se enfoca en el proceso de validación y confiabilidad del *software*. Para esto es necesario seleccionar, justificar y adaptar distintos métodos para comprobar la validez y consistencia, dependiendo de las características de cada funcionalidad. Para poder lograrlo, el desarrollador de la herramienta EDDIE debe implementar los siguientes puntos:

- **Definir y descomponer las características de la funcionalidad:** Es necesario conocer las características de la funcionalidad que está siendo aplicada para encontrar la documentación necesaria y obtener los métodos que se deben aplicar.

Estas características pueden ser nuevas para la herramienta, como lo es la implementación de un nuevo *hardware*. Para ello, se debe determinar a cuál módulo pertenece la funcionalidad y descomponer sus características para encontrar cada elemento influyente en la respuesta. Por ejemplo: La funcionalidad utiliza la cámara y un *eyetracker* para funcionar.

- **Determinar fuentes de información alimentadoras:** Para poder identificar correctamente el resultado esperado, es necesario conocer las fuentes de información influyentes. Un ejemplo de una funcionalidad aplicada en la herramienta EDDIE, es la función de 'búsqueda en enciclopedia', donde es importante conocer que las fuentes de información que influyen en la respuesta obtenida son: El texto identificado y el contenido de la página de Wikipedia con respecto al texto identificado.
- **Determinar los tipos de datos de la respuesta:** Dadas las características de la funcionalidad y las fuentes de información, se determinan los tipos de datos que entrega finalmente la funcionalidad. Gracias a esto se podrá obtener el tipo de método aplicable para validez como para confiabilidad.
- **Seleccionar y justificar métodos para verificar la validez y confiabilidad:** Con lo anterior se seleccionan y justifican los métodos de validez y confiabilidad aplicados. Esto debe ser discutido con el jefe de proyectos para obtener su aprobación, sugerencias de alternativas o rechazo. Como ejemplo: En el caso de lo aplicado en la herramienta EDDIE, en la funcionalidad de 'búsqueda en enciclopedia', se obtienen datos en forma de texto los cuales no corresponden a datos estadísticos, por lo cual se propuso un método de prueba de usuario para verificar el nivel de validez: 'se evalúa el resultado de distintas palabras, verificando que cada una de estas sea el resultado que corresponde cada vez'. A lo anterior, el jefe de proyectos agregó un criterio de aceptabilidad a partir de la ejecución de cada prueba diez veces. Además, determinar la confiabilidad con diez palabras diferentes, dando un total de 100 pruebas para obtener una confiabilidad aceptable.
- **Determinar, describir y aplicar las pruebas:** Una vez seleccionados los métodos, se debe determinar cuales serán las pruebas de aceptación tanto para validez como para confiabilidad. Para esto, el desarrollador debe crear un conjunto de prueba, indicando sus características para su reproducibilidad. Por ejemplo: en caso de un cambio de interfaz, se deben realizar nuevamente las pruebas para confirmar que esta correctamente implementado, sirviendo como demostración de los resultados. Finalmente se aplican dichas pruebas.

- **Aprobación de los resultados:** se muestran los resultados al jefe de proyecto, quien dará su veredicto respecto al resultado final, documentado según la Tabla 4.2. En caso de ser rechazado, se entregará una lista de mejoras a la cual se le debe aplicar la fase de reparación de MECIVA PS. Una vez se corrijan, se repite la fase de diagnóstico.

Tabla 4.2: Plantilla para la documentación de la fase de diagnóstico.
Fuente: (Autoria propia, 2023)

Funcionalidad	
Descripción	
Módulo	
Resultado esperado	
Método implementado	
Pruebas	
Resultados	
Comentarios	
Nivel de funcionamiento	
Aprobado	
Comentarios jefe proyectos	

Para documentar este proceso se utiliza una tabla como la Tabla 4.2. En esta se presentan los siguientes elementos:

- **Funcionalidad:** En este campo se indica el nombre de la funcionalidad.
- **Descripción:** Se indican las características de la funcionalidad, su funcionamiento y las fuentes de información importantes.
- **Módulo:** Se indica el módulo de EDDIE al que pertenece.
- **Resultado esperado:** Corresponde al resultado que se espera después de la ejecución de la funcionalidad. Además de que se indica el tipo de dato obtenido.
- **Método implementado:** Corresponde al método que se implementa para realizar las pruebas de validez y confiabilidad. Estas deben estar debidamente justificadas, además de contar con la aprobación del jefe de proyectos.
- **Pruebas:** Se presentan los resultados individuales de cada prueba, detallando cual es el conjunto de prueba y los resultados de cada una.
- **Resultados:** Se presentan los resultados totales, en forma porcentual, de la validez y confiabilidad que se obtiene en las pruebas.
- **Comentarios desarrollador:** En este apartado el desarrollador deja registrado todos los elementos que considere pertinentes relacionados a la funcionalidad, tanto para el jefe de proyectos como para los futuros desarrolladores.

- **Aprobado:** Nivel de validación del jefe de proyectos. La funcionalidad puede estar completamente aprobada, aprobada con solicitud de mejoras y rechazada.
- **Comentarios jefe proyectos:** Todo comentario que el jefe de proyectos considere pertinente para el desarrollador actual como para el futuro.

La fase presentada puede verse reflejada en la Figura 4.8.

4.5. Resumen

A lo largo de este capítulo se presenta la metodología MECIVA PS, indicando sus distintas fases inspiradas en la metodología GOMS: despliegue, reparación y diagnóstico. La primera aborda tanto el despliegue de la herramienta EDDIE y la nueva funcionalidad desarrollada. La segunda se encarga del tratado de errores y mejoras, presentando un proceso de descomposición que permite al desarrollador conocer las características necesarias para encontrar una solución final. Finalmente, la tercera se encarga de presentar lo que debe realizar un desarrollador de productos de *software* de EDDIE para diagnosticar la validez y confiabilidad de la nueva funcionalidad.

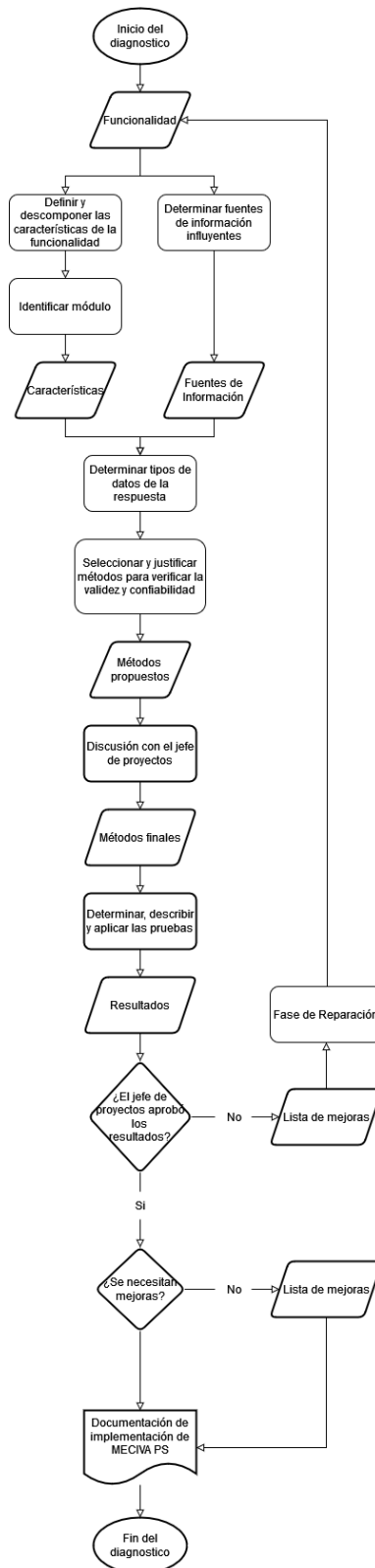


Figura 4.8: Diagrama de flujo de la fase de diagnostico.
Fuente: (Autoría propia, 2023)

5. Aplicación de la metodología

A lo largo de este capítulo se presentan los resultados obtenidos de la aplicación de la metodología MECIVA PS, además de indicar aspectos que influyen en la maduración de esta metodología.

Es importante resaltar que se encontró y trató un total de 67 errores en la herramienta EDDIE, los cuales se encuentran en la Tabla A.1. Esta cantidad excede en sobremanera las expectativas sobre errores en la implementación en el espacio otorgado. Estos errores se encuentran en varios aspectos de la herramienta, por ejemplo: Despliegue, prácticas de programación, falta de documentación (manual de uso), funcionalidades, conexión de API's, entre otros. Por todo esto aún quedan errores sin identificar y sin resolver. Sin embargo, estos son tratables implementando la metodología propuesta. Además, se toma la decisión de centrarse en el correcto funcionamiento de los módulos que se consideran más importantes, dando una menor cantidad de tiempo a los módulos de interfaz de usuario y reconocimiento gestual. Aun así, se enfocó en que al menos un aspecto de cada módulo sea funcional.

Para abordar este capítulo es importante recordar las categorías de errores encontrados, las cuales son:

- Interno a EDDIE:
 - **Consistencia**
 - **Rastreo ocular**
 - **Búsqueda**
 - **Interacción físico/digital**
 - **Reconocimiento gestual**
 - **Interfaz de usuario**
 - **Otro**
- Externo a EDDIE:
 - **Software**
 - **Hardware**
 - **Otro**

De este listado se abarcan cinco internos: consistencia, rastreo ocular, búsqueda, interacción físico/digital, otro. En el caso de los externos, estos son evaluados como conjunto

por sus características. Todos los errores encontrados están documentados y analizados, siendo explorados en las siguientes secciones a través de las dos grandes categorías: interno y externo a EDDIE. Es importante destacar el significado de cada elemento la tabla de errores, los cuales son utilizados con el mismo significado para los requisitos funcionales. Estos son:

- **SS (0):** Sin solución. No se pudo solucionar el error/problema.
- **SP (1):** Solución parcial. Se pudo resolver parcialmente el error/problema o se encontró la solución pero escapa a los alcances de la tesis.
- **SF (2):** Solución Final. Se resolvió en su completitud el error/problema.

Para nombrar los requisitos funcionales se utiliza la siguiente nomenclatura:

- **Primera letra de cada palabra de la categoría + Índice correspondiente.**

Un ejemplo de esto es en el rastreo ocular, el primer requisito es nombrado **RO1**. Para los errores se utiliza el mismo proceso, pero con la letra 'E' al inicio. Utilizando el ejemplo anterior, el primer error de rastreo ocular es llamado **ERO1**. Finalmente, analizando los 67 errores encontrados a lo largo del desarrollo, implementación y mejora de la metodología se obtienen los siguientes porcentajes: 68.65 % **SF**, 25.37 % **SP** y 5.98 % **SS**.

5.1. Error interno a EDDIE: Consistencia

Para comenzar el análisis de la aplicación de MECIVA PS en el módulo de consistencia es necesario exponer todos los requisitos funcionales y errores encontrados. Estos se encuentran en la Tabla 5.1 y la Tabla A.2, respectivamente. Siendo esta la categoría que más errores presenta y se pudo resolver una menor cantidad de requisitos funcionales.

Tabla 5.1: Requisitos funcionales del Módulo de consistencia.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado actual
C1	El sistema debe permitir detectar figuras realizadas en el documento físico y estas deben verse reflejadas en la versión digital del mismo. El requisito proviene del trabajo de Pezoa (RFP1).	SP
C2	El sistema debe detectar comentarios en el documento físico y que estos se vean reflejados en su versión digital. El requisito proviene del trabajo de Pezoa (RFP2).	SP
C3	El sistema debe permitir agregar un marcador de página al documento físico, por medio de una banderilla o post-it azul, el cual debe verse reflejado en la versión digital del documento. El requisito proviene del trabajo de Pezoa (RFP3).	SP
C4	El sistema debe permitir eliminar un marcador de página del documento físico, quitando el marcador y reflejando esto en la versión digital. El requisito proviene del trabajo de Pezoa (RFP4).	SS
C5	El sistema debe permitir detectar figuras en una página determinada del documento digital y estas se deben reflejar en la versión física del mismo. El requisito proviene del trabajo de Pezoa (RFP5).	SS
C6	El sistema debe permitir detectar comentarios realizados en una página determinada del documento digital y reflejar estos en el formato físico. El requisito proviene del trabajo de Pezoa (RFP6).	SS
C7	El sistema debe permitir detectar marcadores de página en el documento digital y desplegar la información asociada a los mismos. El requisito proviene del trabajo de Pezoa (RFP7).	SS

ID	Descripción	Estado actual
C8	El sistema debe permitir activar o desactivar la visualización de figuras sobrepuestos en el documento físico. El requisito proviene del trabajo de Pezoa (RFP8).	SS
C9	El sistema debe permitir eliminar un marcador de página creado en el documento digital actualizando la información en el documento digital. El requisito proviene del trabajo de Pezoa (RFP9).	SS
C10	El sistema debe permitir eliminar un comentario realizado en el documento digital actualizando la información en el documento digital. El requisito proviene del trabajo de Pezoa (RFP10).	SS
C11	Resaltar palabra en documento físico y resaltarla en versión digital. El requisito proviene del trabajo de Carrasco (RFCA4).	SP
C12	Resaltar palabra en documento digital y resaltarla en versión física. El requisito proviene del trabajo de Carrasco (RFCA5).	SS

Observando la Tabla A.2 se puede apreciar que la mayoría de errores se resolvieron en su completitud, en total 21 de los 24 encontrados en este módulo como se ve en la Figura 5.1, correspondiendo a un estado promedio de 1.79, lo que significa que el promedio de errores es más cercano a **SF** e indicando que la gran mayoría de los errores del módulo de consistencia son manejables. Sin embargo, esto no se asemeja en lo obtenido en los requisitos funcionales, esta diferencia se debe a que la mayoría de los errores encontrados y tratados con MECIVA PS corresponden a las funcionalidades de consistencia físico a digital de comentarios, marca páginas y figuras. Los errores correspondientes a estos tomaron un tiempo de entre uno a dos meses en su tratamiento, madurando la metodología en el proceso. Los errores más destacables, presentados en conjunto, corresponden a:

- **EC1, EC2 y EC3:** Estos errores tienen en común la falta de un archivo .pdf, los que fueron resueltos indicando al usuario que debe ingresar el archivo y evitando que el programa falle. Esto ayudó a la maduración de la metodología puesto a que gracias a estos se implementa el concepto de las características relacionadas al error. Esto

se debe a que se tuvo que identificar correctamente todos los elementos que se utilizan y revisar que este correctamente desplegado, verificando si existía alguna API o biblioteca que no funcionara correctamente.

- **EC17, EC18 y EC19:** Este conjunto representa el error de que no se traspasó, puesto que la conexión de la cámara que utiliza el módulo de búsqueda no fue implementado en el cambio de interfaz. La solución tomó la mayor parte del tiempo en ser encontrada y habilitó el uso de tres funcionalidades: consistencia físico a digital de marca páginas, comentarios y figuras.

Este conjunto de errores ayudo a la evolución de la metodología, inspirando el seguimiento de los componentes relacionados al error. Lo anterior se debe a que para encontrar la solución se tuvo que realizar un seguimiento de la variable, obteniendo el tipo de error, categorización y características de la misma, ubicando el componente que causa el conflicto, como se maneja y porque estaba fallando. Además, ayudo a la mejora de la fase de despliegue, puesto que es una funcionalidad que requiere la activación de una la cámara digital la cual fue reutilizada pero no conectada en el avance previo a la herramienta EDDIE, considerando necesario destacar que el nuevo desarrollador determine el nivel de individualidad de la funcionalidad e identifique los componentes existentes relacionados. Puesto que si lo anterior fuera considerado a la hora del desarrollo, estos errores no se presentarán. Otro aspecto importante a destacar es que al utilizar la cámara que esta siendo usada por otra aplicación, siendo asociada a EDDIE o no, evita el correcto funcionamiento de la aplicación, por lo cual se sugiere utilizar una cámara por cada módulo que la requiera.

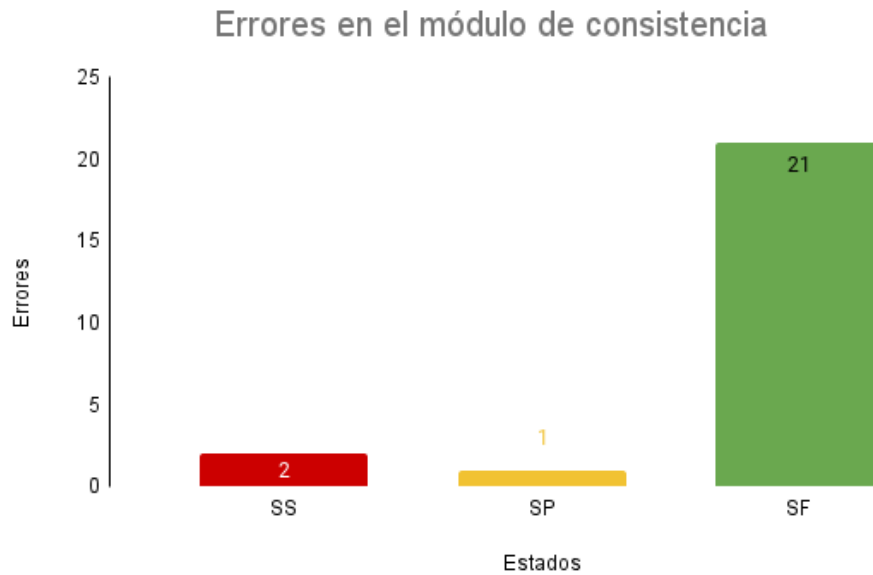


Figura 5.1: Errores encontrados en el módulo de consistencia clasificados en **SS** (sin solución), **SP** (solución parcial) y **SF** (solución final).
Fuente: (Autoría propia, 2023)

Los requisitos funcionales que se abordan, aunque no se refleje en la Tabla 5.1, corresponden a cuatro de siete de las funcionalidades desplegadas en la interfaz de consistencia en la herramienta EDDIE, como se puede apreciar en la Figura 5.2 donde las destacadas en verde están funcionando parcialmente, en rojo no funciona y en morado depende de la que esta en rojo. Es importante resaltar que los requisitos C4, C8, C9, C10 y C12, como están implementados, dependen de que los requisitos C7, C8 y C9 y, si uno de estos falla, todos fallan. En la situación actual los errores con estado **SS** están relacionados directamente con los requisitos C7, C8 y C9.



Figura 5.2: Funcionalidades desplegadas del módulo de consistencia.

Fuente: (Autoría propia, 2023)

5.2. Error interno a EDDIE: Rastreo Ocular

En el caso del rastreo ocular, se encuentra un total siete requisitos funcionales y de seis errores los cuales se listan en las Tablas 5.2 y 5.3, respectivamente.

Tabla 5.2: Requisitos funcionales del Módulo de rastreo ocular.

Fuente: (Autoría propia, 2023)

ID	Descripción	Estado actual
RO1	El módulo debe ser capaz de rastrear los movimientos oculares del usuario. El requisito proviene del trabajo de Estay (RFE1).	SP
RO2	El módulo debe permitir la visualización de un componente gráfico que represente las fijaciones del usuario. El requisito proviene del trabajo de Estay (RFE2).	SF
RO3	El módulo debe permitir la interacción del usuario con el sistema por medio de sus movimientos oculares. El requisito proviene del trabajo de Estay (RFE3).	SF

ID	Descripción	Estado actual
RO4	El módulo debe permitir registrar los datos derivados del rastreo ocular. El requisito proviene del trabajo de Estay (RFE4).	SF
RO5	El módulo debe permitir la integración de distintos dispositivos de rastreo ocular. El requisito proviene del trabajo de Estay (RFE5).	SP
RO6	Creación de un módulo de registro de datos el cual pueda ser usado por cualquier componente de la aplicación. El requisito proviene del trabajo de Estay (RFE6).	SF
RO7	El módulo debe permitir al usuario de la plataforma seleccionar el dispositivo de rastreo ocular que será utilizado. El requisito proviene del trabajo de Estay (RFE7).	SF

Tabla 5.3: Errores encontrados en el módulo de rastreo ocular de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
ERO1	La aplicación falla cuando se abre nuevamente la vista de rastreo ocular luego de ingresar los parámetros.	SF	■ Se cierra la pestaña de parámetros, eliminando la configuración establecida. ■ Se evita que se cierre la pestaña.
ERO2	No se encuentran los plugins relacionados al rastreo ocular.	SF	■ Se compila la funcionalidad. Con la ayuda del jefe de proyectos se encuentra la locación del archivo generado y se coloca en la localización necesaria para su funcionamiento.

ID	Descripción	Estado	Justificación o Solución
ERO3	En la funcionalidad de 'controlar <i>mouse</i> mediante el <i>eyetracker</i> ', la variable <i>controller</i> no se establece como objeto	SF	■ Se cierra la pestaña de parámetros, eliminando la configuración establecida. ■ Se evita que se cierre la pestaña.
ERO4	En la funcionalidad de 'controlar <i>mouse</i> mediante el <i>eyetracker</i> ', la variable <i>logging</i> no se establece como objeto	SF	■ Se cierra la pestaña de parámetros, eliminando la configuración establecida. ■ Se evita que se cierre la pestaña.
ERO5	En la funcionalidad de 'Guardar registro de rastreo ocular', la variable <i>controller</i> no se establece como objeto.	SF	■ Se cierra la pestaña de parámetros, eliminando la configuración establecida. ■ Se evita que se cierre la pestaña.
ERO6	En la funcionalidad de 'Guardar registro de rastreo ocular', la variable <i>logging</i> no se establece como objeto.	SF	■ Se cierra la pestaña de parámetros, eliminando la configuración establecida. ■ Se evita que se cierre la pestaña.

Aunque la cantidad de errores no lo refleja, el nivel de complejidad de los mismos es tal que tomo la totalidad del tiempo de desarrollo para corregirlos. Sin aplicar la metodología, los errores ERO1, ERO3, ERO4, ERO5 y ERO6 no son abarcables, puesto que no se puede definir la raíz del problema sin un análisis integral del código, dada la magnitud de este es necesaria la implementación de la metodología. Cuando se utilizó, se llegó a la realización de que el error de origen entre ellos es único gracias al siguiente análisis:

- **Categorizar el error presentado:** Se identifica como un error interno dentro del módulo de rastreo ocular.
- **Identificar las características del error:** El *eyetracker* funciona correctamente y es identificado por la aplicación. Se identifica que tanto las variables *logging* y *controller* tienen un valor nulo, aunque la aplicación reporta que estas son inicializadas.
- **Identificar posibles tipos de error:** La aplicación indica que las variables previamente mencionadas no están establecidas como objetos. Sin embargo, se plantean dos teorías: El error se encuentra en la inicialización de las variables y que el error corresponde a uno de despliegue.
- **Determinar los componentes relacionados al error:** Utilizando la información obtenida se realiza un seguimiento a las variables, ubicando su función de origen sin encontrar un error en esta. Por lo tanto, se rastrea la función donde se inicializan buscando un error de despliegue, el cual se encuentra puesto que se establece que se detenga una vez termina la inicialización.
- **Determinar y seleccionar la fuente necesaria para abordar el error:** En este caso, es necesario identificar cual de los archivos que inicializan corresponden al correcto, puesto que el código cuenta tanto con la versión nueva como la antigua de la herramienta EDDIE. Utilizando la documentación de Santibáñez (Santibáñez, 2020) se encuentra el archivo correcto y, al ser idénticos, se descubre que el error fue producido a la falta de un *eyetracker* al desarrollarlo.
- **Aplicar la solución encontrada:** Como se tiene conocimiento del archivo donde se encuentra el error de origen, la razón por la cual ocurre y su localización dentro del código, se procede a comentar la sección que termina el funcionamiento. Luego de esto se verifica el estado de la funcionalidad, tras lo cual se comprueba que el error es solucionado y no surge uno nuevo.

Con todo lo anterior, el módulo se encuentra completamente funcional, con acotaciones para mejorar su rendimiento.

5.3. Error interno a EDDIE: Búsqueda

En el caso del modulo de búsqueda se encuentran 13 requisitos funcionales y diez errores, los cuales se ven en las Tablas 5.4 y A.3, respectivamente.

Tabla 5.4: Requisitos funcionales del Módulo de búsqueda.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado actual
B1	Acceder a recursos de información obtenidos desde múltiples fuentes en línea. El requisito proviene del trabajo de Montanares (RFM1).	SP
B2	Acceder a traductor de contenido. El requisito proviene del trabajo de Montanares (RFM2).	SP
B3	Acceder a definiciones de contenido. El requisito proviene del trabajo de Montanares (RFM3).	SP
B4	Acceder a videos. El requisito proviene del trabajo de Montanares (RFM4).	SP
B5	Acceder a enciclopedia. El requisito proviene del trabajo de Montanares (RFM5).	SP
B6	Realizar búsqueda de contenido mediante imágenes. El requisito proviene del trabajo de Montanares (RFM6).	SP
B7	Escoger entre un grupo de API´s cuál utilizar para cada servicio proporcionado por el módulo de búsqueda web. El requisito proviene del trabajo de Montanares (RFM7).	SF
B8	Visualizar los recursos de información digital obtenidos. El requisito proviene del trabajo de Montanares (RFM8).	SF
B9	Interactuar con los recursos de información digital obtenidos. El requisito proviene del trabajo de Montanares (RFM9).	SP
B10	Desplegar/ocultar los recursos de información digital obtenidos desde la web. El requisito proviene del trabajo de Montanares (RFM10).	SF
B11	Tener un registro de las búsquedas realizadas durante una sesión de lectura. El requisito proviene del trabajo de Montanares (RFM11).	SF

ID	Descripción	Estado actual
B12	Acceder a recursos de información en la web mediante comandos de voz. El requisito proviene del trabajo de Montanares (RFM12).	SF
B13	Acceso a un lector de información. El requisito proviene del trabajo de Montanares (RFM13).	SF

En el presente caso, gran cantidad del trabajo fue realizado con éxito, obteniendo solamente un error, EB4, que posteriormente fue solucionado (ver Figura 5.3). El estado de error tuvo un valor promedio de 1.33, siendo más cercano a **SP**, es decir, los errores son manejables temporalmente o necesitan implementación de nuevos elementos para ser resueltos. En el caso de este módulo, existe aún trabajo a futuro por desarrollar cuyos problemas identificados ya cuentan con soluciones propuestas. Considerando los requisitos funcionales, se puede ver que ninguno de estos obtiene la calificación mínima, por lo cual solo falta la implementación de las soluciones propuestas para que este completamente funcional.

En este caso, el factor común a analizar es la utilización de API's para el desarrollo. Al desarrollar la metodología fue necesario tomar en cuenta los aspectos como este, por lo cual ayudó a el desarrollo de la parte de despliegue, tanto para EDDIE como para la funcionalidad. Lo anterior debido a que es necesario destacar la necesidad de tener en conocimiento el estado de cada API en EDDIE para su correcto funcionamiento y la forma de conectarlas a la herramienta para el despliegue de una nueva funcionalidad. También, el tener presente este tipo de factores ayuda a la búsqueda eficiente de la documentación, ayudando a desarrollar la fase de reparación de MECIVA PS.

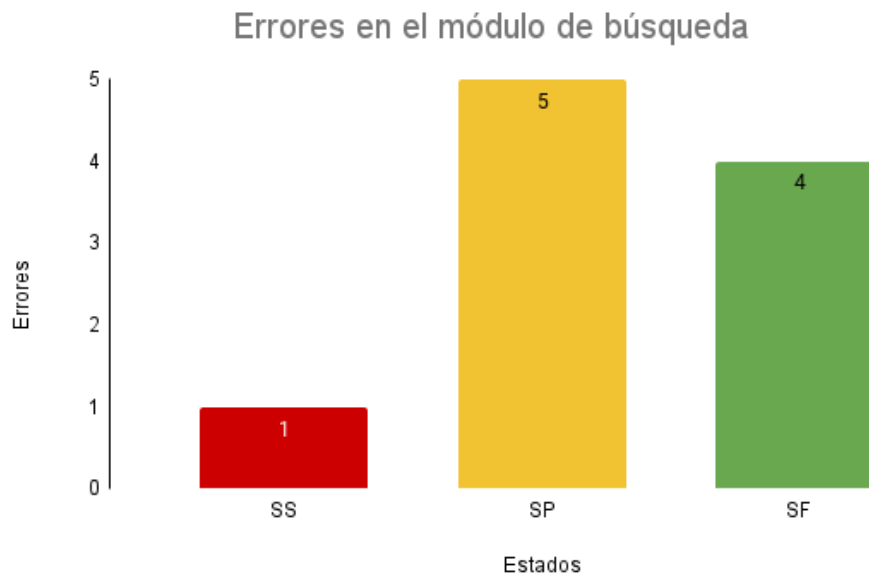


Figura 5.3: Errores encontrados en el módulo de búsqueda clasificados en **SS** (sin solución), **SP** (solución parcial) y **SF** (solución final).
Fuente: (Autoría propia, 2023)

5.4. Error interno a EDDIE: Interacción Físico/Digital

En esta sección se presenta un requisito funcional y cuatro errores, representados en las Tablas 5.5 y 5.6, respectivamente.

Tabla 5.5: Requisitos funcionales del Módulo de interacción físico/digital.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado actual
FD1	Reconocimiento de texto. El requisito proviene del trabajo de Carrasco (RFCA2).	SP

Tabla 5.6: Errores encontrados en el módulo de interacción físico/digital de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
EFD1	No se reconoce el texto del documento físico.	SF	■ Se comprueba que todos los elementos de software relacionados funcionen correctamente. ■ Se prueba con distintas posiciones de la cámara digital. ■ Se determina que el texto es reconocido solo si se coloca al revés.
EFD2	Al reconocer el texto la cámara enfoca otros elementos como botones de la interfaz	SF	■ La altura de la cámara digital se ajusta para poder reconocer el texto y que solo capte el área donde este se encuentra. ■ Se sugiere un cambio de color para los botones.
EFD3	Luego de un uso prolongado el reconocimiento de texto deja de funcionar correctamente, únicamente captando una imagen totalmente negra o blanca	SF	■ Se detectan errores en la inicialización de la cámara cuando se inicia la proyección. ■ Se asegura la correcta inicialización de la cámara.

ID	Descripción	Estado	Justificación o Solución
EFD4	No se reconoce correctamente el texto cada vez que se utiliza el reconocimiento.	SP	■ Solución posible: Refactorizar la interfaz para que el texto este en la parte superior de la misma.

La razón por la cual este módulo tiene solo un requisito funcional se debe a que la mayoría de errores fueron descartados durante la evolución la herramienta EDDIE. Un ejemplo de lo anterior es '*Visión estereoscópica para headset de realidad aumentada.*', este fue descartado puesto que la herramienta actualmente esta diseñada para ser no obstructiva. Al no utilizarse un *headset*, este requisito es descartado. En el caso de los errores, se obtiene un estado promedio de 1.75 siendo más cercano a **SF**, por lo que la mayoría de los errores son solucionables con las herramientas actuales. Con esto se obtiene una aplicación completa dentro de los problemas encontrados, teniendo una solución para todos menos uno, dejando uno **SP** cuya solución esta propuesta. Lo anterior se puede ver reflejado en la Figura 5.4.

Con este módulo se madura la metodología en términos de documentación, puesto que gran parte de las soluciones se encontraron con documentación alternativa, como lo es la consulta al jefe de proyectos.

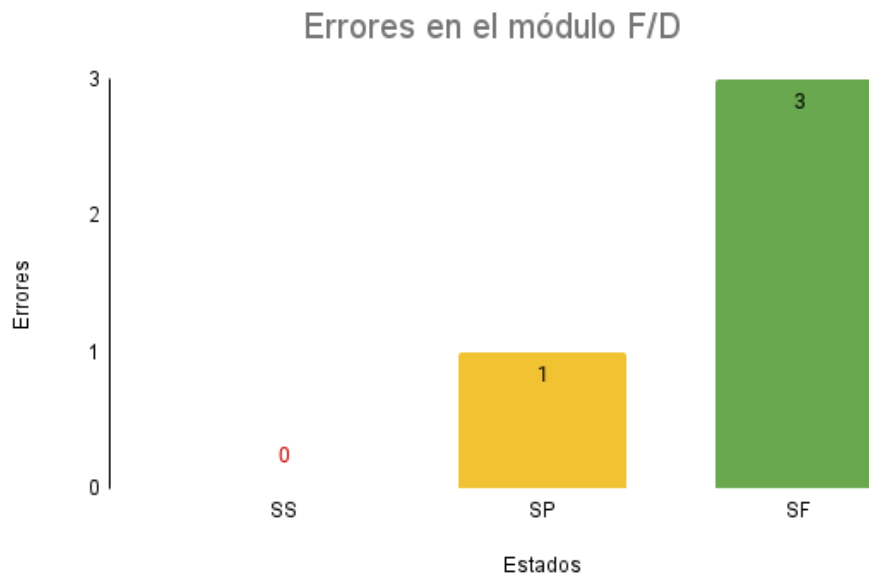


Figura 5.4: Errores encontrados en el módulo de interacción físico/digital (F/D) clasificados en **SS** (sin solución), **SP** (solución parcial) y **SF** (solución final).
Fuente: (Autoría propia, 2023)

5.5. Error interno a EDDIE: Otro

En esta sección se presentan 12 errores, los cuales corresponden a los expuestos en la Tabla A.4.

La mayoría de los errores presentados corresponden a errores de despliegue, siendo la excepción EO6, EO10, EO11 y EO12. Estos errores ayudaron a la maduración de la metodología, pues todos estos fueron utilizados como base para el diseño de la fase de despliegue de MECIVA PS. Además, se toma en cuenta para la documentación en la fase de reparación de errores el consultar a los desarrolladores pasados, puesto que sin este recurso no hubiera sido posible desplegar la herramienta. Considerando los errores, se encuentra con un promedio de estado de 1.66 estando cerca del punto medio entre **SP** y **SF** indicando que aunque muchos errores son manejables actualmente, se requieren nuevos componentes para solucionarlo. Estos problemas surgen puesto a que los errores EO11 y EO12 no se pudieron resolver debido a su dificultad. Esta proporción se ve reflejada en la Figura 5.5



Figura 5.5: Errores encontrados en la categoría Interno-Otro (I-O) clasificados en **SS** (sin solución), **SP** (Solución parcial) y **SF** (solución final).
Fuente: (Autoría propia, 2023)

5.6. Error externo a EDDIE

En esta sección se presentan diez errores, los cuales están documentados en la Tabla A.5.

Esta sección destaca de las demás puesto que presenta un aspecto que se procura cubrir con la metodología, el cual se considera vital para los futuros desarrolladores. Aunque parezca evidente, existen problemas fuera del ámbito de desarrollo que pueden influir en el trabajo del desarrollador y resolverlos es fundamental, aunque sea de manera temporal. Un ejemplo de esto es el error EE1, puesto que la falta de internet impide al desarrollador tener acceso completo a todos los recursos y documentación. Aunque este error no impide el desarrollo, lo dificulta y provoca retrasos en su producción.

El promedio de estado de error es de 1.3, la cercanía al estado **SP** corresponde a que encontrar una solución definitiva, la mayoría de estos errores cuentan con características las cuales imposibilitan encontrar una solución permanente. Lo anterior se puede ver reflejado en la Figura 5.6.

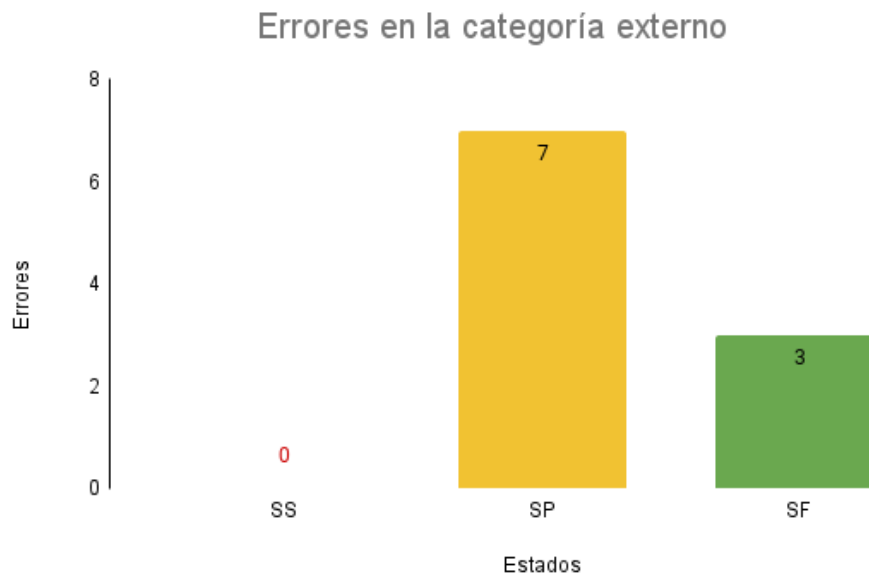


Figura 5.6: Errores encontrados en la categoría 'externo' clasificados en **SS** (sin solución), **SP** (Solución parcial) y **SF** (solución final).
Fuente: (Autoría propia, 2023)

5.7. Resumen

En el presente capítulo se muestra la aplicación de MECIVA PS. Más específicamente, muestra los errores encontrados a lo largo del proceso de desarrollo de la metodología, mostrando en detalle los errores encontrados en la categoría 'interno' de consistencia, rastreo ocular, búsqueda, integración físico/digital y otro, además de la categoría 'externo' la cual fue presentada como conjunto.

Para el tratamiento de cada categoría se utiliza el sistema de 'estado' establecido en la fase de reparación de MECIVA PS, mostrando en que estado se encuentran los errores encontrados y los requisitos funcionales, realizando una comparativa entre ambos. Además, se presenta un análisis de importancia por los errores encontrados indicando como evoluciona la metodología a raíz de estos. Además del valor que tiene el tratamiento de estos utilizando MECIVA PS. Finalmente, se presenta un resumen de los requisitos funcionales y los errores encontrados en la Tabla 5.7, donde RF se refiere a 'requisitos funcionales' y F/D a 'físico/digital'.

Tabla 5.7: Resumen de requisitos funcionales y errores encontrados.
Fuente: (Autoría propia, 2023)

Categoría	Estado RF			Estado errores		
Consistencia	SS	SP	SF	SS	SP	SF
	8.33 %	4.17 %	87.50 %	8.33 %	4.17 %	87.50 %
Rastreo ocular	SS	SP	SF	SS	SP	SF
	0.00 %	0.00 %	100.00 %	0.00 %	0.00 %	100.00 %
Búsqueda	SS	SP	SF	SS	SP	SF
	10.00 %	50.00 %	40.00 %	10.00 %	50.00 %	40.00 %
Interacción F/D	SS	SP	SF	SS	SP	SF
	0.00 %	25.00 %	75.00 %	0.00 %	25.00 %	75.00 %
Otro	SS	SP	SF	SS	SP	SF
	16.67 %	0.00 %	83.33 %	16.67 %	0.00 %	83.33 %
Externos	SS	SP	SF	SS	SP	SF
	0.00 %	70.00 %	30.00 %	0.00 %	70.00 %	30.00 %

6. Evaluación

A lo largo de esta sección, se evalúan tres funciones representativas a través de la aplicación de la fase de diagnóstico de MECIVA PS. Esta información se presenta indicando tanto lo realizado como la documentación, considerando el trabajo del desarrollador.

6.1. Búsqueda en enciclopedia

Esta funcionalidad tiene como objetivo la búsqueda de contenido en una enciclopedia elegida por el usuario. Se diagnostica utilizando como base la API de Wikipedia, esto pues es la única disponible al momento de aplicar la metodología. Con esto se realiza el siguiente procedimiento:

- **Definir y descomponer las características de la funcionalidad:** Lo primero que se hace es reconocer que pertenece al módulo de búsqueda, por lo cual esta utiliza APIs para su funcionamiento. Además, está ligada directamente con el módulo de reconocimiento de texto y es necesaria la conexión a internet para su correcto funcionamiento.
- **Determinar fuentes de información alimentadoras:** Las fuentes de información importantes son el texto identificado por el usuario y la página de Wikipedia correspondiente a dicho texto. Se identifica el resultado esperado, siendo este 'El despliegue de la primera sección de la enciclopedia elegida, basada en el texto destacado por el usuario a través del reconocimiento de la cámara.'
- **Determinar los tipos de datos de la respuesta:** El tipo de dato que retorna la funcionalidad es texto plano.
- **Seleccionar y justificar métodos para verificar la validez y confiabilidad:** Considerando las características del resultado esperado y la fuente de información, se opta por un método no estadístico. Por lo cual se usan los siguientes métodos:
 - **Validez:** Se ejecuta la funcionalidad con diez palabras distintas, comparando el resultado de cada una con la información entregada en wikipedia. Para este último se utiliza el formato: 'https://es.wikipedia.org/wiki/' + TextoResaltado.
 - **Confiabilidad:** Se ejecuta la funcionalidad con diez palabras distintas, diez veces cada una. Se compara si el resultado es el mismo por las diez veces que se ejecuta el resultado.

- **Determinar, describir y aplicar las pruebas:** Para el conjunto de prueba se escogen 10 palabras, para su selección se utilizan palabras que cuenten con todo tipo de elementos, algunas con tildes y mayúsculas para comprobar que estas no afectan el comportamiento de la funcionalidad. Las palabras utilizadas son las siguientes:

1. **P1:** control
2. **P2:** arte
3. **P3:** lectura
4. **P4:** Jerusalén
5. **P5:** esponja
6. **P6:** pastilla
7. **P7:** caramelo
8. **P8:** metodología
9. **P9:** consistencia
10. **P10:** cama

Es importante destacar que 'metodología' esta escrito sin la tilde, esto para comprobar como afecta a la funcionalidad.

Los resultados obtenidos de la realización de las pruebas se pueden ver reflejados en la Tabla A.7. Gráficamente se pueden ver en las Figuras 6.1 y 6.2, gracias a esto y tomando en cuenta los errores encontrados se deduce lo siguiente:

- La API no cuenta con un control de respuesta, por lo cual al ingresar una palabra que no cuenta con una página correspondiente en Wikipedia, esta provoca un fallo en la herramienta EDDIE.
- Elementos como las tildes no afectan a la búsqueda en Wikipedia mediante url, a diferencia de la API la cual no corrige la ortografía o algún error de escritura. Es clave considerar el reconocimiento correcto de la palabra antes de la ejecución. En caso contrario, se entrega el mensaje: 'No se ha encontrado el termino buscado'.
- La búsqueda a través de la API debe ser exacta con las categorías. Por ejemplo, al ingresar la palabra 'lectura' por la API esta retorna 'No se ha encontrado el termino buscado', esto se debe a que no se corrige lo ingresado. Su símil en la búsqueda por url corresponde a 'Lectura (educación)' el cual es corregido por la propia página. Esto puede ser corregido utilizando un nuevo servicio, puesto que el actual no realiza las correcciones necesarias.

Estas conclusiones son importantes a la hora de implementar una nueva enciclopedia, puesto que se debe escoger una que no cuente con dichas falencias. Finalmente, la documentación relacionada con este diagnóstico se encuentra en la Tabla A.6.



Figura 6.1: Resultados para la validez en la funcionalidad 'búsqueda en enciclopedia' utilizando una comparativa entre la página de Wikipedia y los resultados obtenidos, utilizando diez pruebas en diez palabras distintas (**PX**, **X** siendo índice de la palabra).
Fuente: (Autoría propia, 2023)



Figura 6.2: Resultados para la confiabilidad en la funcionalidad 'búsqueda en enciclopedia' utilizando los resultados arrojados por diez pruebas a diez palabras distintas (**PX**, **X** siendo índice de la palabra).

Fuente: (Autoría propia, 2023)

6.2. Rastreo Ocular (TheEyeTribe)

El objetivo de esta funcionalidad es detectar el movimiento de la retina del usuario y reflejarlos en una posición equivalente en la pantalla. Para el diagnóstico se utiliza el *eyetracker TheEyeTribe*, puesto que es el único que contaba con una implementación y conexión con la herramienta EDDIE al momento de aplicar la metodología. El procedimiento realizado es el siguiente:

- **Definir y descomponer las características de la funcionalidad:** El módulo al que corresponde es el de rastreo ocular, por lo cual es necesario utilizar y ubicar el *eyetracker* seleccionado en una posición específica, dependiendo de las características del usuario. En este caso, para poder reconocer la retícula del desarrollador se utilizó una inclinación de 30 grados y a una altura de 4.5cm respecto al escritorio, estando ubicado al final de la misma.
- **Determinar fuentes de información alimentadoras:** Se sabe que la fuente de información importante son los movimientos de retícula captados por *TheEyeTribe*. Por lo cual el resultado esperado es: 'La posición identificada que corresponde a la zona de la interfaz que está viendo el usuario.'
- **Determinar los tipos de datos de la respuesta:** Los tipos de datos corresponden

a datos del tipo coordenadas x,y . Sin embargo, no se encuentra disponible la funcionalidad de registro a la hora de la realización del diagnóstico, así que se utiliza la posición que se muestra en la pantalla a través de una imagen de un ojo que se posiciona sobre la herramienta en un equivalente a lo detectado por el *eyetracker*.

- **Seleccionar y justificar métodos para verificar la validez y confiabilidad:** Para la selección de métodos se consideran las características de los resultados esperados y de la interfaz provista por *TheEyeTribe*. Dado que ambas solo entregan resultados visuales se implementan métodos no estadísticos. Para el desarrollo de los métodos se utiliza como base la publicación: 'Eye tracking y usabilidad en ambientes informacionales digitales: revisión teórica y propuesta de procedimiento de evaluación' (Roa-Martínez and Vidotti, 2020). Además, se utiliza la interfaz de demostración de *TheEyeTribe* y posiciones equivalentes en la pantalla, reflejadas en las Figuras 6.3 y 6.4 respectivamente, mirando las mismas posiciones de la pantalla utilizando el *software* provisto por *TheEyeTribe* y la herramienta EDDIE, comparando las posiciones resultantes en ambos casos.

Validez: Se establece un área de estímulo, basada en la interfaz ofrecida por *TheEyeTribe*, y una lista de tareas a realizar, basadas en el área definida y las fijaciones entregadas por la calibración del *eyetracker*. Luego de ejecutar las tareas, se comparan los datos con la posición entregada por el *software* de *TheEyeTribe*.

Confiabilidad: Se ejecuta la lista de tareas definidas diez veces, comparando los resultados de cada tarea entre sí.



Figura 6.3: Interfaz de demostración de *TheEyeTribe*.

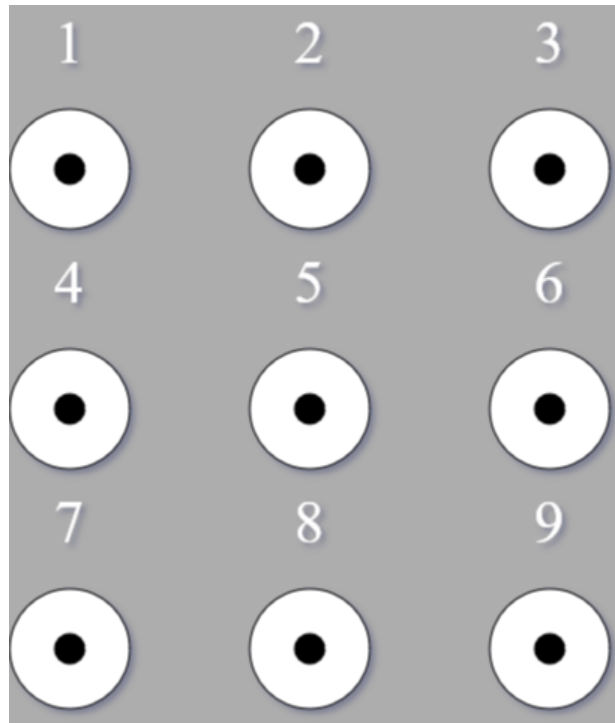


Figura 6.4: Denominaciones del área definida para las pruebas de rastreo ocular.
Fuente: (Autoría propia, 2023)

- **Determinar, describir y aplicar las pruebas:** Para el conjunto de prueba se decide utilizar como referencia la secuencia de calibración de *TheEyeTribe*, puesto que está validado por el dispositivo. Las fijaciones utilizadas son: 9, 2, 4, 3, 1, 7, 8, 5 y 6, equivalentes a la Figura 6.4. Para la ejecución, primero se observa la secuencia equivalente utilizando el *software* de *TheEyeTribe*, para luego repetir el proceso con la herramienta EDDIE.

Los resultados tanto para validez como confiabilidad se pueden ver en las Figuras 6.5 y 6.6, respectivamente. Además, la documentación pertinente se encuentra en la Tabla A.11. A raíz de estas se pueden desprender las siguientes sugerencias:

- Considerando que la zona superior es la única que cuenta con un 100 % de validez y confianza, además de los bajos resultados obtenidos en la zona inferior, se recomienda una refactorización de la interfaz de EDDIE. Para ello se sugiere posicionar los botones en el área superior o, en su defecto, en el área izquierda inferior y central, pues estas últimas cuentan con el segundo mejor porcentaje en validez.
- Se recomienda utilizar la aplicación duplicando las pantallas, puesto que la funcionalidad esta pensada solo para una pantalla.

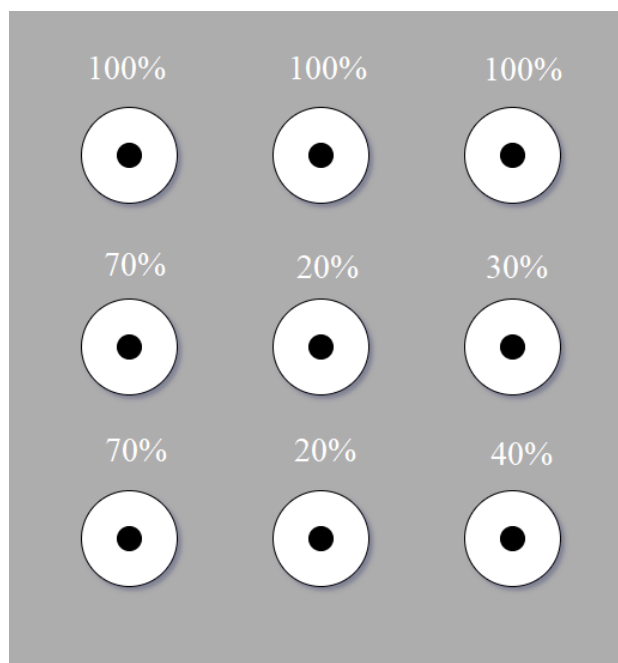


Figura 6.5: Resultados de las pruebas de validez de la funcionalidad 'rastreo ocular'.
Fuente: (Autoría propia, 2023)

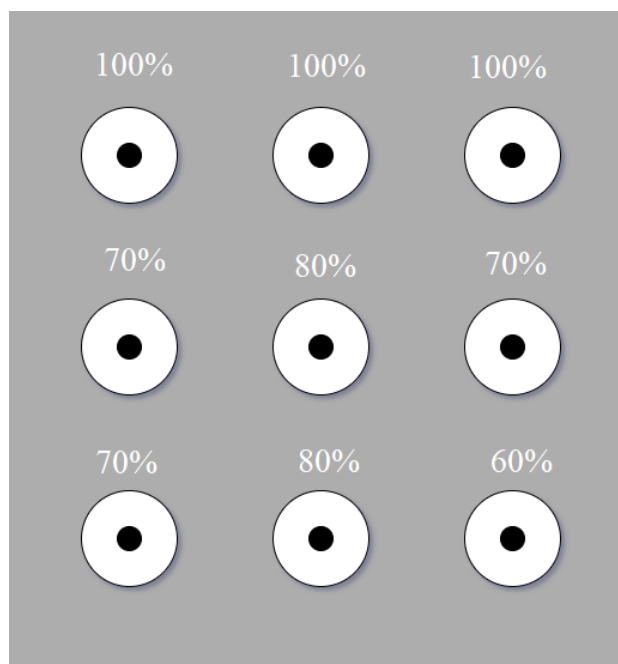


Figura 6.6: Resultados de las pruebas de confiabilidad de la funcionalidad 'rastreo ocular'.
Fuente: (Autoría propia, 2023)

Tomando en cuenta todo lo anterior, se contrasta con la misma prueba pero aplicada para el *eyetracker* en la posición más cercana al usuario en la mesa, a 45

grados como se puede ver en la Figura 6.7. La documentación se puede ver en la Tabla A.11, donde se puede apreciar que funciona completamente en esta nueva posición, tanto en validez como en confiabilidad, eliminando el problema de las áreas que no pueden ser detectadas. Por lo anterior, y considerando el análisis realizado, se propone como solución final implementar la nueva posición para la utilización futura de EDDIE, implementando el setup de la Figura 6.8.

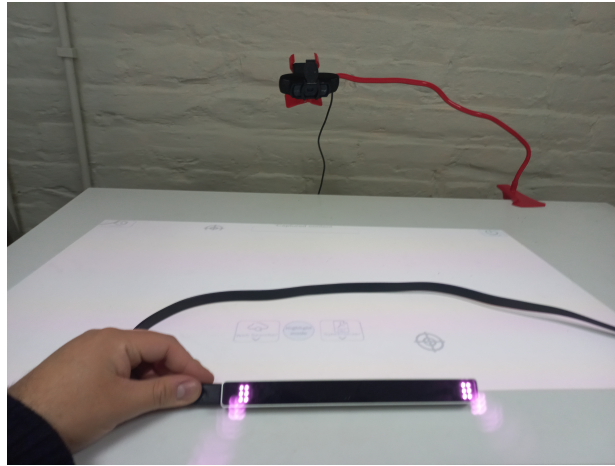


Figura 6.7: Posición alternativa para el *eyetracker*.
Fuente: (Autoría propia, 2023)

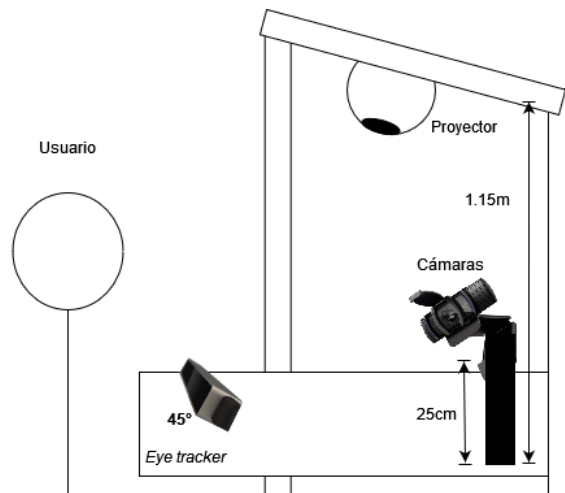


Figura 6.8: Esquema del setup propuesto.
Fuente: (Autoría propia, 2023)

6.3. Reconocimiento de texto

Esta es una de las funcionalidades más importantes de la herramienta EDDIE y, como su nombre lo indica, tiene como objetivo reconocer el texto de un documento

físico y obtener su equivalente digital. El procedimiento aplicado es el siguiente:

- **Definir y descomponer las características de la funcionalidad:** Primero se establece a que pertenece a su propio módulo, el de reconocimiento de texto. Por lo anterior es necesario utilizar una cámara digital que esté colocada de tal forma que reconozca el texto al revés.
- **Determinar fuentes de información alimentadoras:** La fuente de información influyente es el documento físico utilizado por el usuario, específicamente la fracción del texto que este mismo destaca.
- **Determinar los tipos de datos de la respuesta:** Los datos entregados corresponden a texto plano equivalente al destacado.
- **Seleccionar y justificar métodos para verificar la validez y confiabilidad:** Para la selección de métodos se opta por no estadísticos, puesto que los resultados obtenidos deben ser idénticos al material original. Por lo tanto, se decide utilizar lo siguiente:

Validez: Se utiliza un conjunto de 10 palabras de largo distinto y ubicadas en nueve posiciones diferentes. Esto para poder verificar los efectos de la posición y proximidad de las palabras. Las posiciones son utilizadas respecto a la Figura 6.9, dando como total un conjunto de 90 iteraciones.
- **Confiabilidad:** Se utiliza una palabra y se itera diez veces por cada posición, comparando la igualdad de los datos.

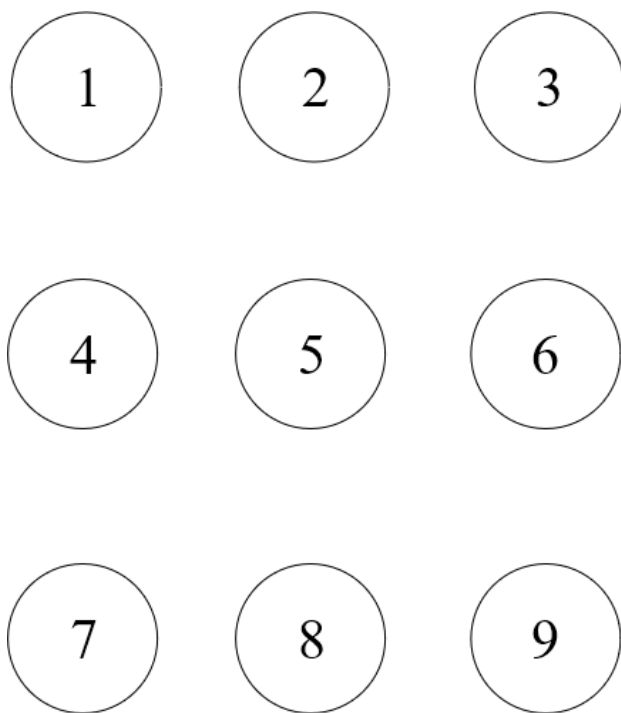


Figura 6.9: Posición de palabras en la hoja física para la prueba de reconocimiento de texto. Siendo la parte superior (1,2,3) la zona más cercana a la cámara

Fuente: (Autoría propia, 2023)

- **Determinar, describir y aplicar las pruebas:** Para la validez se utiliza un conjunto de palabras el cual cumple con tener distintos largos, este conjunto es: Control, Arte, Lectura, Jerusalén, Esponja, Pastilla, Caramelo, Metodología, Consistencia y Cama. En el caso de consistencia solo se utiliza la palabra Control.

Una vez realizado el diagnóstico, se descubrieron problemas importantes relacionados, como se puede ver en la documentación de la Tabla A.14 y los resultados en la Tabla 6.1. Lo importante a destacar es lo siguiente:

- Entre más lejos este la cámara del texto, peor es el reconocimiento. Lo anterior se debe a la calidad que detecta la cámara es inferior, como se puede ver en la Figura 6.10. Se sugiere que se refactorice la interfaz para poder adaptarse a este problema, implementar inteligencia artificial, utilizar la cámara en otra posición de la mesa y tratar la imagen o utilizar una cámara con mejor resolución a la actual.

Superior

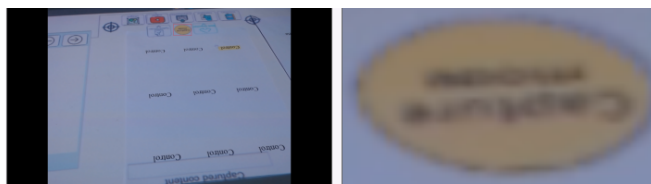
Inferior



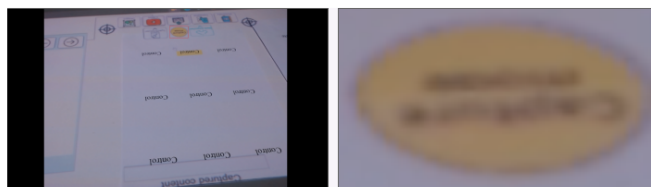
Figura 6.10: Diferencia de calidad de captado de texto según la posición de la palabra.
Fuente: (Autoría propia, 2023)

- Que el botón para destacar texto tenga el mismo color que el destacado entra en conflicto con el reconocimiento de texto de la cámara, como se puede ver en la Figura 6.11. Se sugiere un cambio de color.

posición 7



posición 8



posición 9

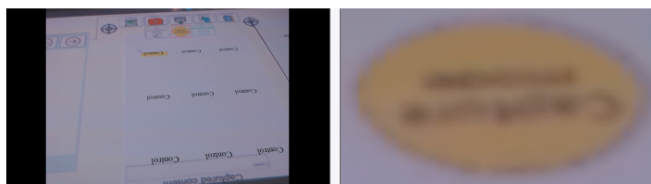


Figura 6.11: Detección de botón de destacado de texto.
Fuente: (Autoría propia, 2023)

Tabla 6.1: Resultados finales de las pruebas de reconocimiento de texto.
Fuente: (Autoría propia, 2023)

Posición	Validez	Confiabilidad
1	60 %	100 %

Posición	Validez	Confiabilidad
2	90 %	50 %
3	100 %	100 %
4	60 %	60 %
5	30 %	70 %
6	70 %	100 %
7	10 %	100 %
8	10 %	100 %
9	30 %	100 %

6.4. Resumen

En esta sección se muestra el proceso de la fase de diagnóstico de MECI-VA PS aplicado en tres funcionalidades de módulos distintos: Búsqueda en enciclopedia, rastreo ocular (*TheEyeTribe*) y reconocimiento de texto. Para todas se muestra paso a paso el proceso realizado y cuales son las conclusiones que se obtienen al respecto, también la documentación pertinente. Cada una cuenta con métodos distintos para diagnosticar la validez y confiabilidad, según sus características.

7. Conclusiones

En este capítulo se presentan las conclusiones relacionadas al trabajo realizado y la aplicación de MECIVA PS.

7.1. Objetivos

Para abarcar esta sección se presenta el objetivo general y los específicos, presentando como fueron abordados y el estado de estos.

7.1.1. General

Desarrollar una metodología para la sistematización del proceso de la validez y confiabilidad de los desarrollos de *software* en torno al proyecto EDDIE:

Se diseña, aplica y verifica una metodología de evaluación y validación aplicable a la herramienta EDDIE y adaptable para futuros módulos integrados, evolucionando en conjunto con la aplicación. Sin embargo, fue necesario adaptar inmediatamente la metodología para cubrir los distintos tipos de problemas que se presentaron.

7.1.2. Específicos

1. Diseñar una metodología ad hoc para evaluar productos de *software* del proyecto EDDIE considerando aspectos de funcionamiento, validez y confiabilidad:

Considerando el capítulo cuatro, 'MECIVA PS', se presenta una metodología dedicada a la herramienta EDDIE que cuenta con 3 fases. La primera es el despliegue para que el desarrollador pueda descargar y ejecutar la herramienta, integrando la funcionalidad que desarrolla. La segunda corresponde a la reparación de errores la cual verifica el correcto funcionamiento del nuevo producto. Finalmente, la tercera se encarga del diagnóstico, el cual verifica la validez y confiabilidad del nuevo software. Todas estas fases fueron desarrolladas en base a lo experimentado en el despliegue e integración de EDDIE.

2. Integrar las implementaciones de EDDIE, tales como: interfaz de usuario (Santibáñez, 2020), consistencia (Pezoa, 2020), búsqueda (Montanares, 2019) y rastreo ocular (Estay, 2020):

Como se puede apreciar en el capítulo cinco: 'Aplicación', se detallan los errores encontrados a la hora del despliegue y aplicación de EDDIE, con esto se verifica que la mayoría de las funcionalidades de EDDIE están disponibles para su uso. Sin embargo, aun falta integrar la capacidad de utilizar el *eyetracker* *TheGazePoint*.

3. Desplegar la versión integrada del *software* en el ambiente operativo con el *hardware* para el que fue diseñado:

Como se puede apreciar por las imágenes presentadas en la sección 4.2.1 y el diagnóstico presentado en el capítulo cinco, se despliega la última versión de la herramienta EDDIE a la fecha de elaboración de este documento. Para ello fue necesario verificar no solo factores de software interno, sino también el trato de *hardware*, como es la posición de la cámara digital y el *eyetracker*. Otros factores a considerar son el ambiental y el software externo.

4. Aplicar la metodología para evaluar EDDIE en su estado de desarrollo actual:

En el capítulo cinco y seis, 'Aplicación' y 'Evaluación' respectivamente, se puede ver como la metodología fue aplicada al estado actual de EDDIE. Considerando los errores encontrados en el despliegue de la herramienta y en las funcionalidades, y el diagnóstico aplicado.

5. Resolver problemas y errores de la plataforma identificados en un ambiente operativo:

Para este punto es necesario referirse al capítulo cinco: 'Aplicación', donde se detallan los errores encontrados y como se trataron en su mayoría. La metodología se adaptó a las características de EDDIE a medida que se encontraban errores. Aún así faltan problemas por resolver, pero la mayoría cuentan con soluciones sugeridas para su futura implementación.

7.2. Resultados

Considerando los resultados obtenidos, se logra exitosamente abarcar toda la herramienta EDDIE. Sin embargo, se encontró una cantidad excesiva de errores respecto a lo esperado. Si bien no todos los errores encontrados tienen una solución implementada, la herramienta actualmente es parcialmente funcional en todos sus módulos gracias a la implementación de una metodología que cuenta con etapas para el despliegue, reparación y diagnóstico.

Esta metodología, al igual que la herramienta, es incremental. Esto significa que esta va a ir madurando y evolucionando a la par de la herramienta. Lo anterior se debe a que, actualmente, esta diseñada para el estado actual de EDDIE y contando solo los problemas encontrados actualmente, por lo cual va a ser necesario una futura revisión a la misma. Esto se ve reflejado a través del documento, puesto que la metodología, al ser creada, fue adaptándose a los problemas que se encontraron. Esto se puede ver

especialmente reflejado en el capítulo cinco, donde se presenta el como maduro la metodología en conjunto de la solución de problemas.

Se encontraron, trataron y documentaron un total de 67 errores, lo que permitió que la mayoría de las funcionalidades de la herramienta EDDIE de encuentren sin fallos en su ejecución. Además, se muestra el diagnóstico de tres funcionalidades las cuales arrojan cambios necesarios para la aplicación, las cuales no serían descubiertas sin este proceso.

7.3. Implicaciones y alcances

7.4. Implicaciones

La existencia de esta metodología implica los siguientes puntos:

1. **Trabajo dirigido:** Al tener MECIVA PS para trabajar, los desarrolladores tendrán una guía para dirigir mejor su modo de trabajo, teniendo una referencia a la hora de desarrollar un nuevo componente o funcionalidad para EDDIE.
2. **Reforzar el concepto de desarrollo incremental:** Con la metodología, el nuevo desarrollador tendrá en cuenta que EDDIE debe ser trabajado como un todo y no solo como el desarrollo de una nueva funcionalidad.
3. **Despliegue en el ambiente de trabajo:** Tras las dificultades surgidas por el contexto mundial durante el desarrollo de la herramienta EDDIE, MECIVA PS se encarga de respaldar al nuevo desarrollador con una forma de desplegar no solo su funcionalidad, sino además de la herramienta completa en el laboratorio de InTeracTiOn, tanto *software* como *hardware*.

7.5. Alcances

El trabajo realizado está destinado al desarrollo de una metodología para la evaluación y validación de productos de software desarrollados para EDDIE, indicando como implementarla y documentar los resultados de la misma.

Las limitaciones están en que se desarrolla utilizando como base el estado actual de la herramienta, madurando la misma a través de los problemas encontrados y diagnósticos realizados, siendo posible su incremento en el futuro. No se considera el desarrollo de código nuevo para nuevas funcionalidades o recrear una funcionalidad que ya no se encuentre disponible o en funcionamiento.

7.6. Trabajo futuro

Considerando el estado final de la herramienta EDDIE al finalizar la primera versión de MECIVA PS, se requiere de la implementación de nuevas APIs, credenciales, refactorizaciones de interfaces, entre otros aspectos, para tener la herramienta completamente funcional. Muchas de las mismas ya están propuestas como solución a lo largo de este documento. En el caso de MECIVA PS, está diseñada para ser incremental, evolucionando en conjunto con EDDIE, por lo cual se pueden implementar nuevos factores:

- **Fase de despliegue:** Considerar nuevo *hardware* o *software* externo, en caso de ser implementado.
- **Fase de reparación:** Dependiendo de los módulos y funcionalidades futuros, se pueden agregar pasos para nuevas descomposiciones necesarias para tener el panorama completo del error/mejora y resolverlo.
- **Fase de diagnóstico:** Rendimiento y/o trabajo con usuarios reales.
- **Crear un manual de usuario de EDDIE en conjunto a la metodología.**
- **Nueva fase de la metodología para cubrir nuevos aspectos.**

Finalmente, esta metodología esta hecha solo para el trabajo individual, en otras palabras solo para un desarrollador a la vez. Por lo cual es necesario realizar futuros análisis para poder utilizarla en un ambiente de desarrollo de mayor magnitud. Lo anterior debido a que al ser EDDIE una herramienta que se desea utilizar para la investigación y tener una larga linea de desarrolladores previos, va a ser necesario un trabajo colaborativo. Por esto, la metodología aun puede ser mejorada más allá de la maduración que ofrece la herramienta como tal.

7.7. Observaciones finales

El desarrollo de una metodología y la aplicación a la herramienta en estado actual conllevó diversas dificultades: desarrollo de tres módulos en el laboratorio y tres fuera de su ambiente de desarrollo, trabajar con una herramienta de desarrollo incremental realizada por seis desarrolladores distintos, y que la documentación de su uso e implementación esta distribuida entre varios documentos distintos. A raíz de esto surgieron desafíos al intentar conocer y comprender las funcionalidades (qué realiza, cómo lo hace y dónde se encuentra cada una), puesto que cada módulo cuenta con errores distintos y estilos de programación diferentes, así como las prácticas de programación utilizadas. Esta experiencia fue utilizada para crear una metodología transversal que puede ser adaptable para nuevas funcionalidades de la herramienta.

Referencias

- Agency for Healthcare Research and Quality (s.f.). Goals, operators, methods, and selection rules. Recuperado de: <https://digital.ahrq.gov/health-it-tools-and-resources/evaluation-resources/workflow-assessment-health-it-toolkit/all-workflow-tools/goms>.
- Bairaktaris, G., Siderov, H., Celebiler, D., Kolli, C., Frohlich, D., and Sporea, R. (2021). Magic bookmark: A nonintrusive electronic system for functionalizing physical books. *Advanced Intelligent Systems*, 2100138. doi: 10.1002/aisy.2021.2100138.
- Carrasco, O. (2017). Sistema para el estudio de interacciones con material impreso y digital equivalentes a través de realidad aumentada. Universidad de Santiago de Chile.
- Cercal Group (s.f.). Cercal group. Recuperado de: <https://cercal.cl/>.
- Contreras, C. (2019). Desarrollo de un sistema para soportar lectura aumentada no obstrusiva. Universidad de Santiago de Chile.
- Cuauhtémoc, M. (s.f.). Errores de software. Recuperado de: <http://www.ii.unam.mx/es-mx/AlmacenDigital/CapsulasTI/Paginas/erroresdesoftware.aspx>.
- Duvall, P. M., Matyas, S., and Glover, A. (2007). *Continuous integration: improving software quality and reducing risk*. Pearson Education.
- Estay, G. (2020). Módulo de interacciones basadas en movimientos oculares para ambiente de lectura aumentada no obstrusiva. Universidad de Santiago de Chile.
- Gonzalo Ramos (2014). Definición de herramienta. Recuperado de: <https://enciclopedia.net/herramienta/>.
- González, R. (2019). Eddie (augmented reading desk). Recuperado de: <https://informatica.usach.cl/proyecto/eddie-augmented-reading-desk/>.
- Google (s.f.). Entiende el mundo que te rodea y comunícate en distintos idiomas. Recuperado de: <https://translate.google.com/about/?hl=es>.
- InteracTiOn (s.f.). Eddie-interaction/eddie-newgui2-2020. Recuperado de: <https://github.com/EDDIE-IntEracTiOn/EDDIE-NewGUI2-2020>.
- InTeractiOn (s.f.). Proyectos. Recuperado de: <https://interaction-lab.info/projects>.
- Jonh, B. E. (1996). Using goms for user interface design and evaluation: which technique? Recuperado de: <https://dl.acm.org/doi/10.1145/235833.236050>.
- Karat, J. (1988). Software evaluation methodologies en handbook of human-computer interaction. Elsevier.
- Kim, S., Park, J., and Kim, J. (2018). e-mmersive book: the ar book that assists the syntopical reading. In *Proceedings of the 9th Augmented Human International Conference*, pages 1–2.
- Martínez, M. (2006). Validez y confiabilidad en la metodología cualitativa. *Paradigma*, 27:07 – 33.
- Microsoft (s.f.). Github copilot y visual studio 2022. Recuperado de: <https://visualstudio.microsoft.com/es/>.

- Montanares, D. (2019). Desarrollo de un módulo de búsqueda web para el software eddie. Universidad de Santiago de Chile.
- Navarro, F. (2018). *Realidad virtual y realidad aumentada*. Grupo Editorial RA-MA.
- Pezoa, D. (2020). Módulo de consistencia en formato físico y digital para sistema de lectura aumentada no obstrusivo. Universidad de Santiago de Chile.
- Roa-Martínez, S. M. and Vidotti, S. A. B. G. (2020). Eye tracking y usabilidad en ambientes informacionales digitales: revisión teórica y propuesta de procedimiento de evaluación. *Transinformação*, 32.
- Santibáñez, F. (2020). Rediseño centrado en el usuario de las interfaces de eddie. Universidad de Santiago de Chile.
- SoftExpert (s.f.). Soluciones para excelencia en la gestión y conformidad empresarial. Recuperado de: <https://www.softexpert.com/es/>.
- Sommerville, I. (2005). Ingeniería del software. séptima edición. PEARSON EDUCACIÓN.
- Soto, M. (2022). Marco de trabajo para la evaluación de validez y confiabilidad de multi-modal observer (mo). Universidad de Santiago de Chile.

Anexo

A.1. GOMS

El nombre de este modelo es un acrónimo el cual contiene sus componentes: Goals, Operators, Methods y Selection Rules (Jonh, 1996). El primero corresponde a los objetivos del usuario final. El segundo son las acciones que el software le permite ejecutar al usuario. El tercero son las operaciones y subobjetivos que pueden lograr un objetivo. El cuarto son las reglas que usa el usuario para saber que método utilizar en cada ocasión. La primera versión es la llamada CMD-GOMS, esta tiene por objetivo el como establecer objetivos en subobjetivos de una manera jerárquica. Los subobjetivos son capaces de producir predicciones tanto cualitativas como cuantitativas del cómo los usuarios utilizaran el sistema. Los pasos a seguir para utilizar GOMS son los siguientes(Agency for Healthcare Research and Quality, sf):

- Definir los objetivos del usuario.
- Descomponer los objetivos.
- Determinar los operadores.
- Determinar y describir los métodos.
- Describir las reglas de selección.

A.2. Tabla de errores totales

Tabla A.1: Errores encontrados en EDDIE, primer semestre 2023.
Fuente: (Autoría propia, 2023)

ID	Tipo de error/problema	Descripción	Estado
E1	Externo	Internet del laboratorio inestable.	SP
E2	Externo	Problemas de compatibilidad entre el software de control de versiones (Gitkraken) y permisos del código.	SF
E3	Interno-otro	No se pudo encontrar el dll leapC.	SF
E4	Externo	No se identifica la rama correcta para obtener la última versión del código.	SP
E5	Interno-otro	El programa no puede inicializar debido a referencias de Visual Studio erróneas.	SF
E6	Interno-otro	El programa no puede inicializar debido a falta de dll correspondiente a serilog.	SF
E7	Interno-otro	El programa no se puede inicializar debido a falta de cámara disponible.	SF
E8	Interno-otro	El programa no tiene un correcto funcionamiento al no encontrarse 2 cámaras disponibles.	SF
E9	Externo	Falta de conexiones eléctricas en el laboratorio.	SP
E10	Externo	Falta de seguridad en el computador del laboratorio.	SF

ID	Tipo de error/problema	Descripción	Estado
E11	Interno-consistencia	Función de sincronización provoca que falle el programa cuando no se ingresa un archivo .pdf.	SF
E12	Externo	No existe un manual de instalación para EDDIE.	SS
E13	Externo	No existe un manual de uso para EDDIE.	SS
E14	Interno-otro	Los mensajes de error están en un idioma distinto al resto de la aplicación.	SF
E15	Externo	No se reconoce el eyetracker en el computador.	SF
E16	Externo	Conexión de la cámara digital es inestable.	SP
E17	Interno-otro	No se encuentra el paquete Nets-tantdart.library.	SF
E18	Interno-otro	No se encuentra Netframework v4.8.	SF
E19	Interno-otro	No se pueden procesar los recursos de las vistas debido a que los archivos están en internet o en una zona restringida, o bien tiene la marca de la web.	SF
E20	Interno-otro	La interfaz no tiene el tamaño correcto cuando se utiliza el proyector.	SF
E21	Interno-consistencia	El botón de búsqueda hace que el programa falle si no se sube un archivo .pdf.	SF
E22	Interno-consistencia	La función para revisar el contenido de pdf hace que el programa falle si no existe un archivo .pdf.	SF
E23	Interno-rastreoOcular	La aplicación falla cuando se abre nuevamente la vista de rastreo ocular luego de ingresar los parámetros.	SF
E24	Interno-interacción físico/digital	No se reconoce el texto del documento físico.	SF
E25	Interno-búsqueda	La función de diccionario no busca las palabras reconocidas.	SP
E26	Interno-búsqueda	La función de enciclopedia no muestra resultados con las palabras reconocidas.	SP
E27	Interno-otro	Malas prácticas: faltan comentarios dentro del código para reconocer que hace cada funcionalidad.	SS
E28	Interno-otro	Malas prácticas: variables y funciones no cuentan con nombres representativos.	SS
E29	Interno-consistencia	No se reconoce el contenido del archivo .pdf ingresado.	SS

ID	Tipo de error/problema	Descripción	Estado
E30	Interno-consistencia	La función de contenido de pdf provoca que el programa falle si es que no se indica una página del archivo .pdf a analizar.	SF
E31	Interno-interacción físico/digital	Al reconocer el texto la cámara enfoca otros elementos como botones de la interfaz.	SF
E32	Interno-interacción físico/digital	Luego de un uso prolongado el reconocimiento de texto deja de funcionar correctamente, únicamente captando una imagen totalmente negra o blanca.	SF
E33	Interno-búsqueda	No se detecta el cambio de la selección de la API para la enciclopedia.	SF
E34	Interno-búsqueda	Al ser detectado símbolos tales como '(', al utilizar la API de la Wikipedia en la enciclopedia el programa falla.	SS.
E35	Interno-interacción físico/digital	No se reconoce correctamente el texto cada vez que se utiliza el reconocimiento.	SP
E36	interno-búsqueda	En caso de no haber conexión a internet, la búsqueda de Wikipedia hace fallar el programa.	SF
E37	Interno-búsqueda	No se reconoce la API para la función de traducción.	SP
E38	Interno-consistencia	No se encuentra el plugin <i>consistencyLibraryFiguresDP.dll</i> .	SF
E39	Interno-búsqueda	No se puede utilizar la funcionalidad de 'comandos de voz' para acceder a la búsqueda de diccionario.	SF
E40	Interno-consistencia	En el destacado de texto de físico a digital, no se destaca en la misma posición ni largo de la palabra destacada.	SS
E41	Interno-consistencia	En el destacado de texto de digital a físico, no se destaca en la misma posición ni largo de la palabra destacada.	SS
E42	Interno-consistencia	En la consistencia de comentarios, si no se especifica la página en donde se realiza, el programa falla.	SF
E43	Interno-consistencia	En la consistencia de figuras, si no se especifica la página en donde se realiza, el programa falla.	SF
E44	Interno-consistencia	En la consistencia de marca páginas, si no se especifica la página en donde se realiza, el programa falla.	SF

ID	Tipo de error/problema	Descripción	Estado
E45	Interno-consistencia	En la consistencia de contenido, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF
E46	Interno-consistencia	En la consistencia de figuras, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF
E47	Interno-consistencia	En la consistencia de marca páginas, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF
E48	Interno-consistencia	En la consistencia de comentarios, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF
E49	Externo	Utilizando <i>The eyetracker</i> , no se puede detectar la parte inferior de la proyección.	SS
E50	Interno-consistencia	No se encuentra la carpeta <i>Plugins-Consistencia</i> .	SF
E51	Interno-rastreoOcular	No se encuentran los plugins relacionados al rastreo ocular.	SF
E52	Interno-consistencia	En la funcionalidad de 'Obtener Figuras', se presenta una ruta al computador de desarrollo original del desarrollador.	SP
E53	interno-interfaz usuario	Botón 'save settings' no representa su funcionalidad.	SF.
E54	Interno-consistencia	En Figuras, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF
E55	Interno-consistencia	En marca páginas, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF
E56	Interno-consistencia	En comentarios, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF
E57	interno-consistencia	En la función de consistencia de marca páginas físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF
E58	interno-consistencia	En la función de consistencia de figuras físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF
E59	interno-consistencia	En la función de consistencia de comentarios físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF

ID	Tipo de error/problema	Descripción	Estado
E60	interno-consistencia	En el archivo <i>FiguresDP</i> , existe una ruta local de la anterior desarrolladora, Dennise.	SF
E61	interno-consistencia	En 'búsqueda de enciclopedia', si la palabra no puede ser encontrada el programa falla	SP
E62	interno-consistencia	En 'búsqueda de enciclopedia', si la palabra no esta exactamente igual en la pagina de Wikipedia, la respuesta no es válida.	SP
E63	Interno-búsqueda	La funcionalidad de 'búsqueda de figuras' no puede ser ejecutada	SP
E64	Interno-rastreoOcular	En la funcionalidad de 'controlar <i>mouse</i> mediante el <i>eyetracker</i> ', la variable <i>controller</i> no se establece como objeto	SF
E65	Interno-rastreoOcular	En la funcionalidad de 'controlar <i>mouse</i> mediante el <i>eyetracker</i> ', la variable <i>logging</i> no se establece como objeto	SF
E66	Interno-rastreoOcular	En la funcionalidad de 'Guardar registro de rastreo ocular', la variable <i>controller</i> no se establece como objeto.	SF
E67	Interno-rastreoOcular	En la funcionalidad de 'Guardar registro de rastreo ocular', la variable <i>logging</i> no se establece como objeto.	SF

A.3. Tabla de errores de consistencia, busqueda, otro y externo

Tabla A.2: Errores encontrados en el módulo de consistencia de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
EC1	Función de sincronización provoca que falle el programa cuando no se ingresa un archivo .pdf.	SF	■ Se identifica la variable que maneja el archivo .pdf. ■ Se revisan todos los componentes relacionados para verificar la ruta que sigue el archivo .pdf. ■ Se agrega una excepción para evitar que se ejecute si no existe un archivo.

ID	Descripción	Estado	Justificación o Solución
EC2	El botón de búsqueda hace que el programa falle si no se sube un archivo .pdf.	SF	■ Se identifica la variable que maneja el archivo .pdf. ■ Se revisan todos los componentes relacionados para verificar la ruta que sigue el archivo .pdf. ■ Se agrega una excepción para evitar que se ejecute si no existe un archivo.
EC3	La función para revisar el contenido de pdf hace que el programa falle si no existe un archivo .pdf.	SF	■ Se identifica la variable que maneja el archivo .pdf. ■ Se revisan todos los componentes relacionados para verificar la ruta que sigue el archivo .pdf. ■ Se agrega una excepción para evitar que se ejecute si no existe un archivo.
EC4	No se reconoce el contenido del archivo .pdf ingresado.	SS	■ No se reconoce el aspecto de figuras. ■ Al ser todas las funcionalidades de consistencia de digital a físico, al fallar una fallan todas.
EC5	La función de contenido de pdf provoca que el programa falle si es que no se indica una página del archivo .pdf a analizar.	SF	■ Se identifica donde se debe introducir el texto. ■ Se aplica una transformación para hacer que la variable se transforme en número.
EC6	En el destacado de texto de físico a digital, no se destaca en la misma posición ni largo de la palabra destacada.	SP	■ Se verifican características y se identifica que la cámara que se usa es la misma que la de reconocimiento de texto. ■ Causas posibles: La posición de la cámara o la iluminación.

ID	Descripción	Estado	Justificación o Solución
EC7	En la consistencia de comentarios, si no se especifica la página en donde se realiza, el programa falla.	SF	■ Se identifica el tipo de problema: la variable correspondiente equivale a nula. ■ Se identifica la variable y se agrega una excepción. ■ Sugerencia: Añadir un indicador de donde se encuentra la celda en la interfaz donde se debe indicar la página.
EC8	En la consistencia de comentarios, si no se especifica la página en donde se realiza, el programa falla.	SF	■ Se identifica el tipo de problema: la variable correspondiente equivale a nula. ■ Se identifica la variable y se agrega una excepción. ■ Sugerencia: Añadir un indicador de donde se encuentra la celda en la interfaz donde se debe indicar la página.
EC9	En la consistencia de figuras, si no se especifica la página en donde se realiza, el programa falla.	SF	■ Se identifica el tipo de problema: la variable correspondiente equivale a nula. ■ Se identifica la variable y se agrega una excepción. ■ Sugerencia: Añadir un indicador de donde se encuentra la celda en la interfaz donde se debe indicar la página.
EC10	En la consistencia de marca páginas, si no se especifica la página en donde se realiza, el programa falla.	SF	■ Se identifica el tipo de problema: la variable correspondiente equivale a nula. ■ Se identifica la variable y se agrega una excepción. ■ Sugerencia: Añadir un indicador de donde se encuentra la celda en la interfaz donde se debe indicar la página.

ID	Descripción	Estado	Justificación o Solución
EC11	En la consistencia de contenido, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SS	Se revisan e identifican las características del fallo, descubriendo que existen diversos problemas asociados. Siendo estos: EC4, EC15 y EC23.
EC12	En la consistencia de figuras, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF	Se revisan e identifican las características del fallo, descubriendo que existen diversos problemas asociados. Siendo estos: EC15, EC16, EC17 y EC21.
EC13	En la consistencia de marca páginas, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF	Se revisan e identifican las características del fallo, descubriendo que existen diversos problemas asociados. Siendo estos: EC15, EC18 y EC20.
EC14	En la consistencia de comentarios, ingresando correctamente los elementos necesarios para su ejecución, el programa falla.	SF	Se revisan e identifican las características del fallo, descubriendo que existen diversos problemas asociados. Siendo estos: EC15, EC19 y EC22.
EC15	No se encuentra la carpeta <i>PluginsConsistencia</i> .	SF	Se identifica y rastrea la localización que debería tener la carpeta, se crea, asigna el nombre apropiado y se ingresan los archivos .dll, probando el correcto funcionamiento de la aplicación.
EC16	En la funcionalidad de 'Obtener Figuras', se presenta una ruta al computador de desarrollo original del desarrollador.	SF	- Se realiza un seguimiento de la ruta presentada, obteniendo la ruta equivalente. - Se asigna la nueva ruta.

ID	Descripción	Estado	Justificación o Solución
EC17	En Figuras, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF	Se procede a identificar y rastrear las características, encontrando que el error de origen se encuentra en el módulo de interacción físico/digital. Rescatando una función que no fue implementada en el traspaso a la nueva interfaz e implementándola.
EC18	En Marca páginas, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF	Se procede a identificar y rastrear las características, encontrando que el error de origen se encuentra en el módulo de interacción físico/digital. Rescatando una función que no fue implementada en el traspaso a la nueva interfaz e implementándola.
EC19	En comentarios, la variable auxcapture presenta el valor null, haciendo fallar la aplicación.	SF	Se procede a identificar y rastrear las características, encontrando que el error de origen se encuentra en el módulo de interacción físico/digital. Rescatando una función que no fue implementada en el traspaso a la nueva interfaz e implementándola.
EC20	En la función de consistencia de marca páginas físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF	- Se realiza un seguimiento de la ruta presentada, obteniendo la ruta equivalente. - Se asigna la nueva ruta.
EC21	En la función de consistencia de figuras físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF	- Se realiza un seguimiento de la ruta presentada, obteniendo la ruta equivalente. - Se asigna la nueva ruta.

ID	Descripción	Estado	Justificación o Solución
EC22	En la función de consistencia de comentarios físico a digital, existe una ruta local de la anterior desarrolladora, Dennise.	SF	■ Se realiza un seguimiento de la ruta presentada, obteniendo la ruta equivalente. ■ Se asigna la nueva ruta.
EC23	En el archivo <i>FiguresDP</i> , existe una ruta local de la anterior desarrolladora, Dennise.	SF	■ Se realiza un seguimiento de la ruta presentada, obteniendo la ruta equivalente. ■ Se asigna la nueva ruta.
EC24	No se encuentra el plugin <i>consistencyLibraryFiguresDP.dll</i> .	SF	■ Luego de una conversación con el jefe de proyectos, se determina la forma de obtener el archivo y su ubicación. ■ Se presenta una ruta local de la desarrolladora, la cual es reemplazada por su equivalente actual y se vuelve a generar el archivo.

Tabla A.3: Errores encontrados en el módulo de búsqueda de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
EB1	La función de diccionario no busca las palabras reconocidas.	SP	■ En una revisión, la pagina de donde se obtuvo la API no existe. Al esta no funcionar y no arrojar errores a parte de la conexión se deduce que no se encuentra mas disponible. ■ Solución propuesta: Es necesario implementar una nueva API para diccionario.
EB2	La función de enciclopedia no muestra resultados con las palabras reconocidas.	SF	■ Se determinan los componentes relacionados al error, encontrando que se utilizo una ruta local de Dania. Se encuentra la ruta correspondiente y se reemplaza.

ID	Descripción	Estado	Justificación o Solución
EB3	No se detecta el cambio de la selección de la API para la enciclopedia.	SF	■ No se detecta el cambio cuando se selecciona la aplicación, sin embargo, se descubre que con el botón 'save settings' estas son implementadas. ■ Este problema genera uno nuevo: "Botón 'save settings'". Este elemento no representa su funcionalidad puesto que por su nombre se entiende que se guarda la configuración actual para su uso en futuras ejecuciones. Sin embargo, este las aplica para su uso inmediato. Solución propuesta: Cambiar a 'Apply settings'.
EB4	Al ser detectado símbolos tales como '(', al utilizar la API de la Wikipedia en la enciclopedia el programa falla.	SS	■ Luego de seguir la ruta de los componentes de la funcionalidad se encuentra la variable de conflicto. Sin embargo, aunque se aplican excepciones e identificadores para la solución, no se encuentra solución. ■ Este error esta relacionado con EB9. Por lo cual se propone la misma solución.
EB5	En caso de no haber conexión a internet, la búsqueda de wikipedia hace fallar el programa.	SF	■ Se busca por la documentación necesaria en la página https://stackoverflow.com/, donde se busca por una funcionalidad que compruebe la conexión a internet. Gracias a esto existe una validación de si esta disponible y se evita el fallo de la aplicación.

ID	Descripción	Estado	Justificación o Solución
EB6	No se reconoce la API para la función de traducción.	SP	■ Luego de revisar la documentación del desarrollador original de la funcionalidad, se determina que se utiliza la API de traducción de Bing. Esta utiliza las credenciales del estudiante por lo cual, al ser utilizadas en la actualidad esas credenciales no son válidas, por lo cual la funcionalidad no sirve. ■ Solución propuesta: Implementar nuevas credenciales y documentar que es necesaria su renovación.
EB7	No se puede utilizar la funcionalidad de 'comandos de voz' para acceder a la búsqueda de diccionario.	SF	■ Se mejora la disposición del proyector, el cual detecta el audio, y se obtiene el comando correspondiente buscando en la documentación pertinente.
EB8	En 'búsqueda de enciclopedia', si la palabra no puede ser encontrada el programa falla.	SP	■ Luego de la aplicación de la metodología, se deduce que el problema es inherente de la API. ■ Soluciones propuestas: Buscar actualizaciones de la API de Wikipedia. Buscar una API alternativa.
EB9	En 'búsqueda de enciclopedia', si la palabra no esta exactamente igual en la página de Wikipedia, la respuesta no es válida.	SP	■ Luego de la aplicación de la metodología, se deduce que el problema es inherente de la API. ■ Soluciones propuestas: Buscar actualizaciones de la API de Wikipedia. Buscar una API alternativa.

ID	Descripción	Estado	Justificación o Solución
EB10	La funcionalidad de 'búsqueda de figuras' no puede ser ejecutada.	SP	■ Luego de la revisión de la documentación se deduce que se utiliza cloud vision como API para la búsqueda. ■ En una inspección más profunda se descubre que la versión de prueba dura tan solo 12 meses, por lo cual ya no es posible utilizarla. ■ Soluciones propuestas: Implementar una nueva API o pagar por el servicio, ingresando las credenciales correspondientes, asociando a esto una documentación pertinente a modo de recordatorio para asegurar su uso.

Tabla A.4: Errores encontrados en la categoría interno-otro de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
EO1	No se pudo encontrar el dll leapC.	SF	■ Se instala <i>leapSDK V2.3.1.31559</i>.
EO2	El programa no puede inicializar debido a referencias de visual studio erróneas.	SF	■ Se revisan una a una las referencias de todos los archivos y se actualizan.
EO3	El programa no puede inicializar debido a falta de dll correspondiente a serilog.	SF	■ Se desinstala, se obtienen los archivos por separado y se integran en el programa.
EO4	El programa no se puede inicializar debido a falta de cámara disponible.	SF	■ Se agrega una excepción y un cuadro de texto que indique al usuario que hacen falta cámaras.

ID	Descripción	Estado	Justificación o Solución
EO5	El programa no tiene un correcto funcionamiento al no encontrarse 2 cámaras disponibles.	SF	Se agrega una excepción y un cuadro de texto que indique al usuario que hacen falta cámaras.
EO6	Los mensajes de error están en un idioma distinto al resto de la aplicación.	SF	Se cambia el idioma de los mensajes desplegados al usuario para que coincida con el resto de la aplicación.
EO7	No se encuentra el paquete Netstandard.library.	SF	- Se recorre la siguiente ruta en Visual Studio: Tools, nuget package manager, package manager settings. - En el parámetro 'nombre' se ingresa 'nuget.org' y en 'source' se ingresa 'api.nuget.org/V3/index.json'.
EO8	No se encuentra Netframework v4.8.	SF	- Se abre el instalador de Visual Studio. - Se selecciona la opción 'Netframework 4.8' y se da click en modificar.
EO9	No se pueden procesar los recursos de las vistas debido a que los archivos están en internet o en una zona restringida, o bien tiene la marca de la web.	SF	- Se identifica la ubicación de cada imagen que presenta el problema. - Se da click derecho a la imagen y se selecciona 'propiedades'. - Se selecciona la opción 'Desbloquear' en la sección de seguridad.
EO10	La interfaz no tiene el tamaño correcto cuando se utiliza el proyector.	SF	Se utiliza Visual Studio para cambiar las dimensiones de despliegue.
EO11	Malas prácticas: Falta comentarios dentro del código para reconocer que hace cada funcionalidad.	SS	Se sugiere la implementación de un manual de usuario.

ID	Descripción	Estado	Justificación o Solución
EO12	Malas prácticas: Variables y funciones no cuentan con nombres representativos.	SS	Se sugiere la implementación de un manual de usuario.

Tabla A.5: Errores encontrados en la categoría Externo de EDDIE.
Fuente: (Autoría propia, 2023)

ID	Descripción	Estado	Justificación o Solución
EE1	Internet del laboratorio inestable.	SP	- Se contacta con el personal de la universidad para reportar el problema. - Se solicita un <i>pendrive</i> de internet al personal de la universidad como solución temporal.
EE2	Problemas de compatibilidad entre el software de control de versiones (Gitkraken) y permisos del código.	SF	Se solicitan los permisos con el jefe de proyectos.
EE3	No se identifica la rama correcta para obtener la última versión del código.	SP	- Se solicita al jefe de proyectos una aclaración para tener en conocimiento cual es la ultima versión. - Se recomienda tener documentación donde se encuentre esta información y sea actualizada constantemente. Un ejemplo de esto es un manual de usuario.
EE4	Falta de conexiones eléctricas en el laboratorio.	SP	Como solución temporal se utiliza un alargador perteneciente al desarrollador.
EE5	Falta de seguridad en el computador del laboratorio.	SF	- Se establece una contraseña para el computador. - Se envía mediante correo electrónico al jefe de proyectos para evitar que esta se pierda.

ID	Descripción	Estado	Justificación o Solución
EE6	No existe un manual de instalación para EDDIE.	SP	La metodología desarrollada y la documentación de esta cuenta con aspectos relacionados a la instalación de la herramienta.
EE7	No existe un manual de uso para EDDIE.	SP	La metodología desarrollada y la documentación de esta cuenta con aspectos relacionados al uso de la herramienta.
EE8	No se reconoce el eyetracker en el computador.	SF	- Se verifica la correcta instalación del software necesario. - Se reconoce que el problema esta en el cable de conexión, por lo cual se solicita uno nuevo.
EE9	Conexión de la cámara digital es inestable.	SP	- Se destina una especial atención a la posición de la cámara. - Es necesario verificar el estado del cable usb de la cámara.
EE10	Utilizando <i>The eye-tracker</i> , no se puede detectar la parte inferior de la proyección.	SP	Solución posible: Utilizar una nueva posición para la el <i>eyetracker</i>.

A.4. Tablas de resultados y documentación de diagnostico

Las tablas que corresponden a la documentación cuentan con todos los aspectos pertinentes hasta la sección 'Nivel de funcionamiento', esto se debe a que solo se presentan los resultados obtenidos por el desarrollador. Además, estas se complementan con sus respectivas tablas de resultados.

Para las tablas que tienen que ver con resultados, estas cuentan con una enumeración por cada prueba realizada, donde se utilizan los siguientes símbolos:

- ✓: La comparativa retorna un resultado coherente con lo esperado.
- ✗: La comparativa retorno un resultado distinto a lo esperado.

A.4.1. Búsqueda en enciclopedia

Tabla A.6: Documentación de la funcionalidad 'Búsqueda en enciclopedia'.
Fuente: (Autoría propia, 2023)

Funcionalidad	Búsqueda en enciclopedia (wikipedia)
Descripción	Se busca el contenido de una palabra en una enciclopedia elegida por el usuario, siendo en este caso Wikipedia. La funcionalidad requiere de un texto previamente encontrado por una cámara digital y las fuentes de información importantes son el texto reconocido y la información que se encuentra en la página de Wikipedia.
Módulo	Búsqueda
Resultado esperado	El despliegue de la primera sección de la enciclopedia elegida, correspondiente al texto destacado por el usuario. Los datos corresponden a texto plano.
Método implementado	Para la selección de los métodos, dada la naturaleza de los datos se implementan métodos no estadísticos. Para la validez se compara con la referencia existente, siendo esta la página de Wikipedia de la palabra correspondiente. En caso de la confiabilidad, se determina el nivel de consistencia realizando una comparación con los resultados obtenidos. Los métodos implementados son los siguientes: Validez: Se ejecuta la funcionalidad con diez palabras distintas, comparando el resultado de cada una con la información entregada en Wikipedia. Para este ultimo se utiliza el formato: 'https://es.wikipedia.org/wiki/' + TextoResaltado. Confiabilidad: Se ejecuta la funcionalidad con diez palabras distintas, diez veces cada una. Se compara si el resultado es el mismo por las diez veces que se ejecuta el resultado.
Pruebas	Tanto para validez como para consistencia se utiliza el siguiente conjunto de palabras: control, arte, lectura, Jerusalén, esponja, pastilla, caramelo, metodología, consistencia y cama. Para esto se utiliza la fuente Times new roman en tamaño 30, como sugiere el jefe de proyectos. Los resultados se encuentran en la Tabla A.7.
Resultados	Validez: 80 % Confiabilidad: 100 %
Comentarios desarrollador	■ Si se busca una palabra que no existe en wikipedia, la aplicación falla. ■ La API no realiza el proceso de corrección que se hace al ingresar el url. Ej: si se busca 'metodologia', no se agregara la tilde como si se hace al buscarlo por url, enviando el mensaje 'No se ha encontrado el termino buscado'.
Nivel de funcionamiento	Funciona parcialmente

Tabla A.7: Resultados de las pruebas aplicadas en la búsqueda en enciclopedia.
Fuente: (Autoría propia, 2023)

					Validez						
	1	2	3	4	5	6	7	8	9	10	
control											

	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
arte	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
lectura	×	×	×	×	×	×	×	×	×	×
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
Jerusalén	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
esponja	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
pastilla	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
caramelo	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
metodologia	×	×	×	×	×	×	×	×	×	×
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
consistencia	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Confiabilidad									
	1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Validez									
	1	2	3	4	5	6	7	8	9	10
cama										

✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Confiabilidad									
1	2	3	4	5	6	7	8	9	10
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

A.4.2. Rastreo Ocular

Tabla A.8: Documentación de la funcionalidad 'Rastreo Ocular'.
Fuente: (Autoría propia, 2023)

Funcionalidad	Rastreo ocular (<i>TheEyeTribe</i>)
Descripción	La funcionalidad identifica que parte de la interfaz está observando el usuario a través de la conexión con un <i>eyetracker</i> , en este caso se utiliza <i>TheEyeTribe</i> . La posición identificada puede ser vista por el usuario a través de una imagen en la interfaz, por ejemplo: la imagen de un ojo que cambia de posición dependiendo de donde este mirando el usuario. La fuente de información importante es el movimiento de la retícula del usuario.
Módulo	Rastreo ocular
Resultado esperado	La posición identificada que corresponde a la parte de la interfaz proyectada que esta viendo el usuario. Los tipos de datos corresponden a datos del tipo coordenadas x,y reflejadas en la pantalla por una imagen a elección.
Método implementado	Para la selección de métodos, considerando las características de los resultados se implementan métodos no estadísticos. Para la validez se realiza una comparativa de miradas con una referencia existente entregada por los <i>softwares</i> de <i>TheEyeTribe</i>, basado en ‘Eye tracking y usabilidad en ambientes informacionales digitales: revisión teórica y propuesta de procedimiento de evaluación’ (Roa-Martínez and Vidotti, 2020). Para la confiabilidad se realiza una comparativa de los resultados obtenidos en la realización de repetir el proceso diez veces. Validez: Se establece un área de estímulo basado en la interfaz ofrecida por <i>TheEyeTribe</i> y una lista de tareas a realizar, basadas en el área definida. Luego de ejecutar las tareas, se comparan los datos con la posición entregada por el <i>software</i> de <i>TheEyeTribe</i>. Confiabilidad: Se ejecuta la lista de tareas definidas diez veces, comparando los resultados de cada tarea entre sí.
Pruebas	Las pruebas se realizan utilizando la secuencia: 9, 2, 4, 3, 1, 7, 8, 5 y 6. Los resultados están en la Tabla A.9.
Resultados	Ver la Tabla A.10.

Comentarios de-sarrollador	■ Las mayores fallas se encuentran en la parte inferior. Esto se debe a que EDDIE implementa el eyetracker de una forma distinta a la que se supone. ■ Se recomienda una refactorización de la interfaz, dejando los botones en la parte superior y, en su defecto, izquierda inferior y central de esta. ■ Se recomienda utilizar la aplicación duplicando las pantallas, puesto que la funcionalidad esta pensada solo para una pantalla.
Nivel de funcionamiento	Funciona parcialmente

Tabla A.9: Resultados de las pruebas aplicadas en el rastreo ocular.
Fuente: (Autoría propia, 2023)

Pos/Iteración	IT1	IT2	IT3	IT4	IT5	IT6	IT7	IT8	IT9	IT10
1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
4	✓	✓	X	✓	X	✓	✓	✓	X	✓
5	X	✓	✓	X	X	X	X	X	X	X
6	✓	✓	X	X	X	X	X	X	X	✓
7	X	X	X	✓	✓	✓	✓	✓	✓	✓
8	X	X	✓	X	X	X	X	X	✓	X
9	X	✓	X	✓	✓	X	✓	X	X	X

Tabla A.10: Resultados finales para diagnostico de rastreo ocular.
Fuente: (Autoría propia, 2023)

Posición	Validez	Confiabilidad
1	100 %	100 %
2	100 %	100 %
3	100 %	100 %
4	70 %	70 %
5	20 %	80 %
6	30 %	70 %
7	70 %	70 %
8	20 %	80 %
9	40 %	60 %

Tabla A.11: Documentación de la funcionalidad 'Rastreo Ocular' utilizando una segunda posición.
Fuente: (Autoría propia, 2023)

Funcionalidad	Rastreo ocular (<i>TheEyeTribe</i>) utilizando posición alternativa
----------------------	---

Descripción	La funcionalidad identifica que parte de la interfaz está observando el usuario a través de la conexión con un <i>eyetracker</i> , en este caso se utiliza <i>TheEyeTribe</i> . La posición identificada puede ser vista por el usuario a través de una imagen en la interfaz, por ejemplo: la imagen de un ojo que cambia de posición dependiendo de donde este mirando el usuario. La fuente de información importante es el movimiento de la retícula del usuario. En este caso específico, se utiliza el <i>eyetracker</i> en la parte delantera de la mesa con una inclinación de 45 grados aproximadamente.
Módulo	Rastreo ocular
Resultado esperado	La posición identificada que corresponde a la parte de la interfaz proyectada que esta viendo el usuario. Los tipos de datos corresponden a datos del tipo coordenadas x,y reflejadas en la pantalla por una imagen a elección.
Método implementado	Para la selección de métodos, considerando las características de los resultados se implementan métodos no estadísticos. Para la validez se realiza una comparativa de miradas con una referencia existente entregada por los <i>softwares</i> de <i>TheEyeTribe</i>, basado en 'Eye tracking y usabilidad en ambientes informacionales digitales: revisión teórica y propuesta de procedimiento de evaluación' (Roa-Martínez and Vidotti, 2020). Para la confiabilidad se realiza una comparativa de los resultados obtenidos en la realización de repetir el proceso diez veces. Validez: Se establece un área de estímulo basado en la interfaz ofrecida por <i>TheEyeTribe</i> y una lista de tareas a realizar, basadas en el área definida. Luego de ejecutar las tareas, se comparan los datos con la posición entregada por el <i>software</i> de <i>TheEyeTribe</i>. Confiabilidad: Se ejecuta la lista de tareas definidas diez veces, comparando los resultados de cada tarea entre sí.
Pruebas	Las pruebas se realizan utilizando la secuencia: 9, 2, 4, 3, 1, 7, 8, 5 y 6. Los resultados están en la Tabla A.12.
Resultados	Ver la Tabla A.13.
Comentarios desarrollador	Se sugiere implementar la nueva posición encontrada. Para ello es necesario utilizar algún elemento externo que mantenga el dispositivo estable.
Nivel de funcionamiento	Funciona totalmente

Tabla A.12: Resultados de las pruebas aplicadas en el rastreo ocular utilizando una segunda posición.

Fuente: (Autoría propia, 2023)

pos/iteración	IT1	IT2	IT3	IT4	IT5	IT6	IT7	IT8	IT9	IT10
1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
4	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
5	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
6	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

pos/iteración	IT1	IT2	IT3	IT4	IT5	IT6	IT7	IT8	IT9	IT10
7	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
8	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
9	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Tabla A.13: Resultados finales para diagnostico de rastreo ocular utilizando una posición alternativa.

Fuente: (Autoría propia, 2023)

Posición	Validez	Confiabilidad
1	100 %	100 %
2	100 %	100 %
3	100 %	100 %
4	100 %	100 %
5	100 %	100 %
6	100 %	100 %
7	100 %	100 %
8	100 %	100 %
9	100 %	100 %

A.4.3. Reconocimiento de texto

Tabla A.14: Documentación de la funcionalidad 'Reconocimiento de texto'.

Fuente: (Autoría propia, 2023)

Funcionalidad	Reconocimiento de texto
Descripción	La funcionalidad reconoce una palabra previamente destacada por el usuario. Esta requiere una cámara digital colocada de forma que reconozca el texto al revés y la fuente de información importante es el documento físico elegido por el usuario.
Módulo	Reconocimiento de texto
Resultado esperado	El equivalente a la palabra resaltada por el usuario. El tipo de resultado esperado es texto plano.
Método implementado	Para la selección de métodos se toma en cuenta que para ser un resultado valido, este tiene que ser equivalente por lo cual se toma un verdadero y falso, usando métodos no estadísticos. Validez: Se utiliza un conjunto de diez palabras de largo distinto y ubicadas en nueve posiciones diferentes. Esto para poder verificar los efectos de la posición y proximidad de las palabras. Las posiciones son utilizadas en una combinación entre superior, medio e inferior y las posiciones izquierda, centro y derecha. Confiabilidad: Se utiliza una palabra y se itera diez veces por cada posición, comparando la igualdad de los datos.
Pruebas	Para la validez se utiliza un conjunto de palabras el cual cumple con tener distintos largos, este conjunto es: Control, Arte, Lectura, Jerusalén, Esponja, Pastilla, Caramelo, Metodología, Consistencia y Cama. En el caso de consistencia solo se utiliza la palabra Control. Los resultados de validez y confiabilidad se encuentran en las Tablas A.15 y A.16, respectivamente. Las cruces rojas representan que el error fue a causa de detectar el botón que activa la funcionalidad de resaltado de texto.

Resultados	Los resultados finales se encuentran en la Tabla A.17.
Comentarios de-sarrollador	■ Existen fallas importantes con respecto al color de la detección. Se sugiere un cambio en el color del botón que permite destacar el texto. ■ Entre más lejos de la cámara se encuentra el texto, más difícil es que esta lo reconozca. Se sugiere una refactorización de la interfaz para adaptarse a este problema. Otra solución posible es utilizar una cámara de mayor resolución para el texto o implementar inteligencia artificial para evitar los errores por baja de calidad.
Nivel de funcionamiento	Funciona parcialmente.

Tabla A.15: Resultados de las pruebas de validez para reconocimiento de texto.
Fuente: (Autoría propia, 2023)

Palabra/Pos	1	2	3	4	5	6	7	8	9
Control	X	✓	✓	✓	✓	✓	X	X	X
Arte	✓	✓	✓	✓	X	✓	X	X	X
Lectura	X	✓	✓	✓	X	✓	X	X	X
Jerusalén	✓	✓	✓	X	X	✓	X	X	✓
Esponja	✓	✓	✓	✓	✓	✓	X	X	✓
Pastilla	✓	X	✓	X	X	X	X	X	X
Caramelo	✓	✓	✓	✓	✓	✓	X	X	X
Metodología	X	✓	✓	✓	X	X	X	X	X
Consistencia	X	✓	✓	X	X	X	✓	✓	✓
Cama	✓	✓	✓	X	X	✓	X	X	X

Tabla A.16: Resultados de las pruebas de consistencia para reconocimiento de texto.
Fuente: (Autoría propia, 2023)

Pos/Iteración	1	2	3	4	5	6	7	8	9	10
Pos1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Pos2	✓	X	✓	✓	✓	✓	X	X	X	X
Pos3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Pos4	X	✓	X	✓	✓	X	✓	X	✓	✓
Pos5	✓	✓	✓	X	X	X	X	X	X	X
Pos6	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Pos7	X	X	X	X	X	X	X	X	X	X
Pos8	X	X	X	X	X	X	X	X	X	X
Pos9	X	X	X	X	X	X	X	X	X	X

Tabla A.17: Resultados finales de las pruebas de reconocimiento de texto.
Fuente: (Autoría propia, 2023)

Posición	Validez	Confiabilidad
1	60 %	100 %

Posición	Validez	Confiabilidad
2	90 %	50 %
3	100 %	100 %
4	60 %	60 %
5	30 %	70 %
6	70 %	100 %
7	10 %	100 %
8	10 %	100 %
9	30 %	100 %

A.5. Derechos de Autor

© @ José Manuel Ibaceta Jaña y Roberto González-Ibáñez @2023



• Algunos derechos reservados. Esta obra está bajo una Licencia Creative Commons Atribución-Chile 3.0. Sus condiciones de uso pueden ser revisadas en:
<http://creativecommons.org/licenses/by/3.0/cl/%7D>.